

Agentic MoE 2026

Emilson de Queiroz Moraes

2026-06-27

Contents

Parte I — Fase 0: Canonicalização + Gate de Segurança	3
RUN 0 — Canonicalização + Gate de Segurança/Supply-chain (BLOCKING)	3
§0.1 — Achados que ALTERAM o set (headline)	3
§0.2 — Correções de licença (gate “state license”)	3
§0.3 — Identidades resolvidas (migrações/renomeações)	3
§0.4 — Tabela de Canonicalização (Deliverable 0)	4
§0.5 — Listas finais por veredito	7
§0.6 — Divergências de estrelas vs v5 (reconciliar na RUN 4)	7
§0.7 — Itens [UNVERIFIED] / pendências para fases seguintes	8
Parte II — Fase 1: Deep Research (a) Substratos L0/L8/L9	8
GRUPO 1a — Substratos Always-On (L0 Guardrails · L8 Memory · L9 Observability)	8
Contexto do grupo	8
PROFILE CARDS	9
L0 — GUARDRAILS	9
L8 — MEMORY	10
L9 — OBSERVABILITY	14
OVERLAPS, COLABORAÇÃO vs COMPETIÇÃO vs INCOMPATIBILIDADES (com evidência)	17
L0 Guardrails — competição estrutural por um recurso único	17
L8 Memory — backends concorrentes (mem0 vs letta vs cognee vs graphiti vs agentmemory)	18
L9 Observability — instrumentação vs plataforma (composição, não competição)	18
Padrão recorrente: OSS-engine ↔ SaaS-platform	19
MATRIZ DE COMPARAÇÃO	19
Fontes	21
Parte II — Fase 1: (b) Conhecimento L4/L5	22
Agentic MoE 2026 — Group 1b: Knowledge Layers (L4 + L5)	22
Nota de Escopo	22
Perfis dos Experts	22
Análise Cross-Group	29
Matriz de Comparação	30
Fontes	31
Parte II — Fase 1: (c) Pipeline L1/L2/L6/L7	33
Agentic MoE 2026 — Phase 1 · Group 1c · BUILD PIPELINE	33
L1 — Spec-Driven Development (SDD)	33
L2 — Workflow & Skill Frameworks	36
L6 — Generative Design	40
L7 — Slides-as-code	43

(2) MATRIZ DE COMPARAÇÃO	44
(3) ANÁLISE CRUZADA DO GRUPO — overlaps, colaboração vs competição vs incompatibilidades (com evidência)	49
Fontes	51
Parte II — Fase 1: (d) Amplificador L3	52
Group 1d — Amplifier: L3 Multi-Agent Orchestration / MoE Engines	52
Sumário Executivo	52
1. Profile Card — ruvnet/ruflo	52
2. Profile Card — bmad-code-org/BMAD-METHOD	55
3. Mini-Cards — Vizinhos	58
4. Padrões MoE / Gating Network — Extração e Reuso	59
5. Análise Comparativa — Ruflo vs. BMAD	61
6. Matriz de Comparação	62
7. Fontes	63
Parte III — Fase 2: N-Tree / MoE Routing Graph	64
Phase 2 — Recursive N-Tree + MoE Routing Graph	64
0. Modelo MoE — o hub como GATING NETWORK	64
1. N-TREE ANINHADA (experts INCLUDED por role/layer)	65
2. EDGE LIST — from → to mecanismo (Axis4) relação (Axis5) evidência	69
3. MERMAID — grafo de roteamento MoE estratificado	73
4. RECEITAS DE COMPOSIÇÃO (substrate-first)	73
5. INCOMPATIBILIDADES DE CAMADA	75
Parte IV — Fase 3: Orchestrator/Hub	79
Phase 3 — Orchestrator/Hub: the MoE Gating Network (middleware Agentic-Tool Orchestrator)	79
(a) ARQUITETURA EM CAMADAS — o pipeline de gating sobre os substratos	79
(b) TOOL ATTENTION / ISO — atenção gateada sobre um pool permanente de summaries	80
(c) MULTI-CRITERIA SCORING (CTS) — ISO × Eisenhower × risk × scope × methodology × reversibility	81
(d) TOOL REGISTRY SCHEMA — YAML por tool (SSOT do gating)	82
(e) MEMORY SUBSTRATE — como o HUB persiste/recorda estado e decisões de roteamento	83
(f) OBSERVABILITY SUBSTRATE — todo route traçado OTel-native; evals alimentam os gates	84
(g) MODEL ROUTER (Switchcraft-style) — modelo capaz mais barato por tool-call via LiteLLM	85
(h) GUARDRAIL & HITL GATES — warn → correct → block; L0 sempre primeiro	85
(i) SECURITY MODEL — supply-chain, tainting, sandbox/egress, failover	86
(j) COMPOSITION RECIPES embutidas no roteamento (presets, substrate-first)	87
(k) EVALUATION — métricas, plano de testes, deployment/scaling	87
(l) REFERENCE IMPLEMENTATION — MCP-server sketch + sequência de roteamento	88
(m) PRAGMATISM FILTER — alternativas over-engineered REJEITADAS (explícito)	90
Parte V — Síntese Final	90
Agentic MoE 2026 — RELATÓRIO FINAL (Síntese das Fases 0-3)	90
1. SUMÁRIO EXECUTIVO (pt-BR, ~1 página)	91
2. PANORAMA MERGED (Fases 0-3, condensado)	92
3. RECONCILIAÇÃO DE ESTRELAS & LICENÇAS (com datas)	94
4. CHEAT-SHEET DE ROTEAMENTO (“qual expert, quando”)	97
5. LIMITAÇÕES & PRÓXIMOS PASSOS	99

Parte I — Fase 0: Canonicalização + Gate de Segurança

RUN 0 — Canonicalização + Gate de Segurança/Supply-chain (BLOCKING)

pt-BR (nota epistêmica): Esta passada **re-derivou** identidade, estrelas, licença e status de cada item via API **ao vivo** (não confiei nos números do v5). Resultado: várias divergências materiais e **3 achados de segurança/identidade que mudam o set INCLUDED**. Star counts são **ordem de grandeza datada (2026-06-27)**; onde a API retornou corpo vazio, uso fonte secundária marcada [sec]. Nada aqui é precisão — é o melhor sinal verificável hoje.

§0.1 — Achados que ALTERAM o set (headline)

- 1. **MemPalace → EXCLUDED/FLAGGED (segurança).** Alegações secundárias (**não provadas**) de compra de estrelas (~42K em repo de ~2 meses), de o projeto ser um “wrapper de ChromaDB” com nome de celebridade (owner milla-jovovich/mempalace), e benchmark “96.6% LongMemEval” não-auditável. Pelo gate (“EXCLUDE or flag any compromised supply chain”), **sai do INCLUDED** até auditoria independente. O v5 o tinha como default de memória — **substituir por mem0**.
- 2. **andrej-karpathy-skills: dono canônico é multica-ai, NÃO forrestchang.** O path forrestchang/andrej-karpathy-skills **404/não resolve**; multica-ai/andrej-karpathy-skills é o repo vivo (**154K, SEM licença**). A atribuição “mirror” do v5 está **invertida**. [!] **Sem licença = “all rights reserved” por padrão →** falha a exigência “state license” do filtro.
- 3. **GSD rug-pull CONFIRMADO por múltiplas fontes secundárias.** gsd-build/get-shit-done (~63K) — token \$GSD (Solana), saque de liquidez (~US\$500K), fundador “TÂCHES” apagou contas ~2026-04-01; pacotes npm abandonados seguem como ameaça viva. **EXCLUDED** confirmado. Sucessor seguro auditado: **open-gsd/get-shit-done-redux** (@opengsd/get-shit-done-redux).

§0.2 — Correções de licença (gate “state license”)

Item	v5 dizia	API/realidade 2026-06-27	Ação
multica-ai/andrej-karpathy-skills	MIT	none (sem LICENSE)	flag OSS-posture (all-rights-reserved)
langfuse/langfuse	MIT	NOASSERTION (source-available/EE-style)	flag (não é OSS permissivo puro)
Arize-ai/phoenix	OSS	NOASSERTION (licença Arize custom)	flag
bmad-code-org/BMAD-METHOD	MIT	NOASSERTION (GitHub não resolve p/ SPDX)	verificar
BerriAI/litellm	MIT	NOASSERTION (cláusula custom)	flag (infra)
ChristopherKahler/base,—/seed		none	flag (cluster)
ruvnet/agent-ic-flow	OSS	none	flag (neighbor)

§0.3 — Identidades resolvidas (migrações/renomeações)

Item	Veredito	Evidência (2026-06-27)
Understand-Anything	Egonex-AI/Understand-Anything canônico (migrou de Lum1104)	README “originally created by Lum1104 · now maintained under EgonexAI”; ~55-66K [sec]
ECC	affaan-m/ECC canônico (MIT)	API: 187,680, não-fork, homepage ecc.tools; atribuição WorldFlowAI do ATH = errada
laminar	renomeado → lmnr-ai/lmnr (~2.8K [sec])	slug laminar não resolve; YC S24
claude-swarm	renomeado → parruda/swarm	API do owner falhou; Ruby gem ativo [sec]
MetaGPT	FoundationAgents/MetaGPT (ex geekan/MetaGPT)	API: 68,402, rename de org
ruflo	ruvnet/ruflo (ex claude-flow)	API: 54,569, criado 2025-06-02

§0.4 — Tabela de Canonicalização (Deliverable 0)

Colunas: stars = valor API ao vivo (sem tag) ou [sec] (secundário, API vazia), datado 2026-06-27 · fork? sempre não salvo indicado · last push = atividade do repo · sec = status do gate.

SUBSTRATOS (always-on)

Item	owner/repo	stars (2026-06-27)	lic	fork?	last push	sec- status	layer	role	verdict
karpthy-claude-md	multica-ai /andrej-karpthy-skills	154,186	none	não	2026-04-20	[!] sem licença	L0	substrate	INCLUDED (flag licença)
forrestcham-variant	forrestchang/andrej-karpthy-skills	—	—	—	—	404	L0	—	EXCLUDED (não existe/mis-attrb)
base	ChristopherKahle/base	870	none	não	2026-04-29	borderline stale	L0	substrate	INCLUDED (cluster <1K)
mem0	mem0ai/mem0	570-59K [sec]	Apache-2.0	não	ativo 2026	clean	L8	substrate	INCLUDED (novo de-fault memória)
letta	letta-ai/letta (ex-MemGPT)	22,891	Apache-2.0	não	2026-05-14	clean	L8	substrate	INCLUDED
cognee	topoteretes/cognee	17,452	Apache-2.0	não	2026-05-22	clean	L8	substrate	INCLUDED
graphiti	getzep/graphiti	26,380	Apache-2.0	não	2026-05-21	clean	L4↔L8	bridge	INCLUDED (BRIDGE)

Item	owner/repo	stars (2026-06-27)	lic	fork?	last push	sec- status	layer	role	verdict
mempalace	MemPalace / millajovovich	~42K [sec]	MIT	não	—	[FLAG] star-manip + claims não-auditáveis [UN-VERIFIED]	L8	—	EXCLUDED/FLAGGED (segurança)
agentmemory	monix00/agentmemory	14,616	Apache-2.0	não	2026-05-20	clean	L8	neighbor	NEIGHBOR
zep	(SaaS; core=graphiti)	—	—	—	—	—	L8	baseline	BASELINE
langfuse	langfuse/langfuse	28,219	NOASSERTION	NOASSERTION	2026-05-30	clean (lic source-available)	L9	substrate	INCLUDED (default obs; flag licença)
openllmetry	traceloop/openllmetry	7,115	Apache-2.0	não	2026-05-29	clean	L9	neighbor	NEIGHBOR
laminar	lmnr-ai/lmnr (renomeado)	~2.8K [sec]	Apache-2.0	não	—	rename	L9	neighbor	NEIGHBOR
opik	comet-ml/opik	18,829	Apache-2.0	não	2026-04-14	stale (~74d)	L9	neighbor	NEIGHBOR
phoenix	Arize-ai/phoenix	9,918	NOASSERTION	NOASSERTION	2026-05-29	clean (lic custom)	L9	neighbor	NEIGHBOR

CONHECIMENTO

Item	owner/repo	stars (2026-06-27)	lic	fork?	last push	sec	layer	role	verdict
graphify	safishamsi/graphify	52,057	MIT	não	2026-05-23	clean	L4	knowledge	INCLUDED
understanding anything	Egonex-AI/Understanding Anything	~55-66K [sec]	MIT [sec]	não	ativo	clean (migrated Lum1104→Egonex)	L4	knowledge	INCLUDED
obsidian-skills	kepano/obsidian-skills	3,143	MIT	não	2026-05-24	clean	L5	knowledge	INCLUDED
obsidian app	(proprietary core)	—	—	—	—	—	L5	baseline	BASELINE

BUILD PIPELINE (ordenado)

Item	owner/repo	stars (2026-06-27)	lic	fork?	last push	sec	layer	role	verdict
spec-kit	github/spec-kit	106,332	MIT	não	2026-05-27	clean	L1	pipeline	INCLUDED
openspec	Fission-AI/OpenSpec	51,175	MIT	não	2026-05-27	clean	L1	pipeline	INCLUDED
gsd (safe)	open-gsd/gsd-core	~4.2K [sec]	MIT	não	rel. 2026-06-12	clean (sucessor seguro)	L1	pipeline	INCLUDED
gsd (original)	gsd-build/getshit-done	~63K	MIT	não	abandonado ~2026-04	[F0AG] \$GSD rug-pull	L1	—	EXCLUDED (segurança)
specdd	—	—	—	—	—	—	L1	lens	LENS (metodologia)
superpowers	websa/superpowers	~147K [sec]	MIT	não	ativo	clean	L2	pipeline	INCLUDED
gstack	garrytan/gstack	101,289	MIT	não	2026-05-22	clean	L2	pipeline	INCLUDED
ecc	affaan-m/ECC	187,680	MIT	não	2026-05-20	clean	L2(+L3+sup)	pipeline (cross)	INCLUDED
cli-anything	HKUDS/CLI-Anything	~42-44K [sec]	Apache-2.0 [confirm]	não	—	needs-confirm	L2	pipeline	INCLUDED (lic a confirmar)
paul	ChristopherKahn/paul	127,600	MIT	não	2026-06-03	clean	L2	pipeline	INCLUDED (cluster)
carl	ChristopherKahn/carl	153,300	MIT	não	2026-04-29	borderline	L2	pipeline	INCLUDED (cluster)
seed	ChristopherKahn/seed	288,000	one	não	2026-06-03	flag licença	L2	pipeline	INCLUDED (cluster)
open-design	nexu-io/open-design	48,662	Apache-2.0	não	2026-05-21	clean	L6	pipeline	INCLUDED
open-codesign	OpenCoworkers/open-codesign	46,993	MIT	não	2026-06-20	clean	L6	pipeline	INCLUDED
impeccable	bbakaus/impeccable	30,051	Apache-2.0	não	2026-05-22	clean	L6	pipeline	INCLUDED
claude-design / frontend-design	Anthropic	—	proprietary	—	—	—	L6	baseline	BASELINE
slidev	slidevjs/slidev	47,037	MIT	não	2026-06-03	clean	L7	pipeline	INCLUDED
frontend-slides	zarazhangru/frontend-slides	19,093	MIT	não	2026-05-26	clean	L7	pipeline	INCLUDED

AMPLIFICADOR + INFRA

Item	owner/repo	stars (2026-06-27)	lic	fork?	last push	sec	layer	role	verdict
ruflo	ruvnet/ruflo	54,569	MIT	não	2026-05-24	clean	L3	amplifier	INCLUDED (MoE exemplar)
bmad-method	bmad-code-org/BMAD-METHOD	47,753	NOASSERTION	NOASSERTION	2026-05-20	clean (verificar lic)	L3	amplifier	INCLUDED (MoE exemplar)
agentic-flow	ruvnet/agentic-flow	762	none	não	2026-06-24	flag licença	L3	neighbor	NEIGHBOR (<1K)
claude-swarm	parruda/swarm (renomeado)	UNVERIFIED	MIT [sec]	não	ativo [sec]	rename	L3	neighbor	NEIGHBOR
claude-squad	smtg-ai/claude-squad	7,673	AGPL-3.0	não	2026-05-18	clean (AGPL)	L3	neighbor	NEIGHBOR
metagpt	FoundationAgents/MetaGPT	68,462	MIT	não	2026-01-21	stale (~5m)	L3	baseline	BASELINE (não Claude-específico)
LangGraph	LangGraph/AutoGen/CrewAI	—	OSS	—	—	—	L3	baseline	BASELINE (refs)
litellm	BerriAI/litellm	42,034	NOASSERTION	NOASSERTION	2026-04-03	stale (~3m)	INFRA	infra-primitive	INFRA (Phase 3 only)

§0.5 — Listas finais por veredito

- **INCLUDED (experts do landscape):** multica-ai/andrej-karpathy-skills[!], ChristopherKahler/base (cluster), mem0, letta, cognee, graphiti (BRIDGE), langfuse[!], graphify, Understand-Anything, obsidian-skills, spec-kit, OpenSpec, gsd-core, superpowers, gstack, ECC, CLI-Anything, paul/carl/seed (cluster), open-design, open-codesign, impeccable, slidev, frontend-slides, ruflo, bmad-method. **(26)**
- **BRIDGE:** getzep/graphiti (L4↔L8).
- **NEIGHBOR:** agentmemory, openllmetry, lmnr, opik, phoenix, agentic-flow, parruda/swarm, claude-squad.
- **BASELINE:** zep, obsidian app, claude-design/frontend-design, MetaGPT, LangGraph/AutoGen/CrewAI, claude-code.
- **LENS:** SpecDD (metodologia SDD).
- **EXCLUDED:** gsd-build/get-shit-done ([FLAG] \$GSD rug-pull), **MemPalace ([FLAG] star-manip/claims)**, forrestchang/andrej-karpathy-skills (404/mis-attrib), klaviyo/graphiti_mcp (fork), goabstract/Awesome-Design-Tools (stale/non-agentic).
- **INFRA (não-Target):** BerriAI/litellm.

§0.6 — Divergências de estrelas vs v5 (reconciliar na RUN 4)

Item	v5 (citado)	API/[sec] 2026-06-27	Δ
spec-kit	~90-97K	106,332	↑
OpenSpec	~56K	51,175	↓
ECC	~163K (abr)	187,680	↑
gstack	~117K	101,289	↓
superpowers	~240K	~ 147K [sec]	↓ ↓

Item	v5 (citado)	API/[sec] 2026-06-27	Δ
graphify	~73K	52,657	↓
understand-anything	~68K	~55-66K [sec]	↓
open-design	~70K	48,662	↓
obsidian-skills	~36K	33,438	↓
ruflo	~38-62K	54,569	[ok]
bmad-method	~43K	47,753	↑
karpathy-skills	~183K	154,186	↓

[!] **Aviso de magnitude:** vários counts (ECC 188K, karpathy 154K, superpowers ~147K, spec-kit 106K, gstack 101K) são **extraordinários** para repos de 2025-2026 — rivalizam os maiores do GitHub. Reporto o **valor da API ao vivo** (fonte primária) datado, **sem editorializar**, mas sinalizo a anomalia de magnitude para verificação humana. Isso reforça por que o gate trata “velocidade de estrela” como sinal, não prova (vide MemPalace).

§0.7 — Itens [UNVERIFIED] / pendências para fases seguintes

- mem0, superpowers, Understand-Anything, CLI-Anything, gsd-core, lmnr, parruda/swarm: estrelas via **secundário** (API retornou corpo vazio) — re-tentar com retry/token.
- CLI-Anything: licença a confirmar (v5 diz Apache-2.0).
- bmad-method/langfuse/phoenix/litellm: licença **NOASSERTION** — confirmar termos reais (EE/custom).
- MemPalace: alegações de segurança são **secundárias/não-provadas** — exigir auditoria antes de qualquer reinclusão.

Parte II — Fase 1: Deep Research (a) Substratos L0/L8/L9

GRUPO 1a — Substratos Always-On (L0 Guardrails · L8 Memory · L9 Observability)

Pesquisa: Ecossistema AI-native / agentic software-engineering (Claude Code & cross-harness) **Grupo:** 1a — substratos (always-on) **Observado:** 2026-06-27 (estrelas = ordem de grandeza datada; valores RUN 0 + refinamentos primários quando disponíveis) **Nota de língua:** prosa em pt-BR; identificadores, comandos e nomes de repositório em en-US. **Disciplina de fontes:** ≥2 fontes independentes por expert; citações ≤15 palavras; itens incertos marcados [UNVERIFIED].

Contexto do grupo

Os **substratos always-on** são as camadas que ficam permanentemente ligadas por baixo de qualquer fluxo agêntico, independentemente da tarefa concreta:

- **L0 — Guardrails:** as regras/princípios injetados no contexto (tipicamente via CLAUDE.md/rules) que governam *como* o agente pensa e edita antes de qualquer ação.
- **L8 — Memory:** a camada de memória persistente que sobrevive entre sessões (preferências, fatos, grafos de conhecimento, estado de agente).
- **L9 — Observability:** a camada de tracing/avaliação que torna o comportamento do agente visível para humanos (traces, evals, prompt-mgmt).

Estas três camadas compartilham uma propriedade arquitetural: **não competem pela tarefa** — elas *envelopam* a tarefa. Por isso podem (e tipicamente devem) coexistir.

PROFILE CARDS

L0 — GUARDRAILS

multica-ai/andrej-karpathy-skills

Identidade & maturidade. Um único CLAUDE.md que codifica princípios de codificação-com-LLM derivados de observações de Andrej Karpathy. ~154,186 no RUN 0; a página de repositório observada em 2026-06-27 mostra **183k / 18.7k forks** (github.com/multica-ai/andrej-karpathy-skills) — ordem de grandeza “centenas de milhares”, entre os artefatos de guardrail mais estrelados do ecossistema. Maturidade de *conteúdo* alta, maturidade de *engenharia* baixa: “28 Commits”, sem releases publicados. Distribuído também como plugin Claude Code e regra Cursor (`.cursor/rules/karpathy-guidelines.mdc`).

Propósito (escopo, casos de uso, persona). Escopo: documento único de princípios. Resolve os pitfalls que Karpathy descreve: “*models make wrong assumptions... don't manage their confusion, don't seek clarifications*” e “*like to overcomplicate code... bloat abstractions*” (README, citações ≤15 palavras). Persona = um *senior-engineer-em-prosa* que força o agente a deliberar antes de codar. Quatro princípios: **Think Before Coding** (não assumir, expor tradeoffs), **Simplicity First** (mínimo código), **Surgical Changes** (tocar só o necessário), **Goal-Driven Execution** (transformar tarefas imperativas em metas verificáveis com tests-first).

Capabilities (artefatos, automação, cobertura de harness). Artefato: 1× CLAUDE.md + CURSOR.md + EXAMPLES.md. Automação: nenhuma (é texto injetado no contexto). Cobertura de harness: **Claude Code** (plugin/marketplace + CLAUDE.md per-project) e **Cursor** (project rule committada). Qualquer harness que leia CLAUDE.md/AGENTS.md herda os princípios por convenção.

Lentes. - *OSS posture*: **License = NONE** (autoritativo: GitHub API license=null — **sem arquivo LICENSE**). O README/footer menciona “MIT”, mas sem um LICENSE válido isso **não constitui concessão OSS** (default = all-rights-reserved). Decisão do operador (RUN 0, confirmada): manter como substrato L0 **com flag de licença**. [!] “Sem licença” ≠ open-source para o filtro “state license”. - *S-SDLC fit*: alto como *política de baseline* — “Surgical Changes” e “tests-first” mapeiam diretamente em práticas de PR review seguro. - *SaaS multi-tenant fit*: N/A (é um arquivo, não um serviço). - *SDD-methodology fit*: parcial — “Goal-Driven Execution” é spec-light (success criteria + verificação), mas não é uma metodologia spec-driven formal. - *MoE expert-profile*: layer=L0; role=guardrail/policy; competence=princípios de qualidade de código; persona=senior-engineer cético; abstraction=prosa-de-regra; authorization=read-only sobre o agente (não executa nada).

SECURITY posture (incl. license). Superfície de ataque mínima: é texto. O risco é de *provenance* — atribuir a Karpathy princípios que ele não revisou linha-a-linha (o README é explícito que são “*observations*” destiladas de um post, não um endosso). License MIT permite uso comercial. Sem código executável = sem risco de supply-chain.

INTEGRATION SURFACE (mecanismos): {filesystem, git-repo, rules} (+ hooks opcional quando instalado como plugin Claude Code que injeta a skill).

ChristopherKahler/base

Identidade & maturidade. “AI builder operating system” — BASE = **Builder’s Automated State Engine**. ~87 (cluster) no RUN 0; página observada mostra **86 / 18 forks**, “50 Commits”, “1 tag” (github.com/ChristopherKahler/base). Maturidade nascente, single-maintainer (Chris Kahler). Distribuído via npm @chrisai/base (npx @chrisai/base --global --workspace).

Propósito (escopo, casos de uso, persona). Transformar Claude Code “*from a per-session tool into a workspace that remembers, maintains itself, and never goes stale*” (README). Resolve o problema do CLAUDE.md que vira “*junk drawer*” e fica obsoleto. Persona = um *gerente-de-estado-de*

workspace para “AI builders” multi-projeto/multi-cliente que saíam da “duct tape phase”. Casos de uso: injeção automática de contexto compacto (Projects, Entities, State, PSMM), monitoramento de saúde por **drift score**, ciclos de manutenção (/base:pulse, /base:groom, /base:audit).

Capabilities (artefatos, automação, cobertura de harness). Artefatos: **Data Surfaces** (JSON estruturado + hook que auto-injeta sumários no contexto); *workspace.json* (manifest single-source-of-truth); *operator.json* (perfil do operador); **PSMM** (Per-Session Meta Memory — loga DECISION/CORRECTION/SHIFT/INSIGHT/COMMITMENT e re-injeta). Automação: 1 servidor **BASE MCP** (20 ferramentas em 5 módulos — Projects/Entities/State/Operator/PSMM) + hooks Claude Code (UserPromptSubmit, SessionStart). Comando-chave /base:audit-claude audita o diretório *.claude/* (dedup por MD5; classes DUPLICATE/DIVERGED/STALE/etc.). Cobertura de harness: **Claude Code only** (Node ≥ 16.7 , Python 3).

Lentes. - *OSS posture*: **License = NONE** (autoritativo: GitHub API license=null — **sem arquivo LICENSE**; o lc-agent confirmou license:null em base/seed). README menciona “MIT — Chris Kahler”, mas sem LICENSE não há concessão OSS válida (all-rights-reserved por padrão). Flag mantida. - *S-SDLC fit*: moderado — o valor é *higiene de configuração* (detectar hooks que disparam em duplicata, config stale apontando para ferramentas inexistentes), o que é uma forma de segurança operacional. - *SaaS multi-tenant fit*: N/A (*workspace-local*, JSON em *.base/data/*, sem multi-tenant). - *SDD-methodology fit*: baixo — é state-management, não spec-driven. - *MoE expert-profile*: layer=L0 (com forte spillover para L8); role=workspace-state/config-hygiene; competence=anti-staleness; persona=operations-manager; abstraction=JSON-surfaces + manifest; authorization=read-only sobre projetos (“*never modifies projects*”), read-write sobre *.base/*.

SECURITY posture (incl. license). Princípio de design #1: “**If it’s not current, it’s harmful.**” Hooks são “*lightweight Python*”, output XML compacto, silenciosos se nada a reportar. Risco: hooks UserPromptSubmit executam código local a cada prompt — superfície de execução não-trivial; o próprio README documenta upgrades v2→v3 que arquivam cópias de hooks “*double-fire-causing*”. Gestão de *rules* propriamente dita é delegada a uma ferramenta separada (CARL), não embutida.

INTEGRATION SURFACE (mecanismos): {MCP, filesystem, git-repo, hooks, rules, memory} (memory via PSMM/surfaces JSON; git apenas leitura para detecção de projetos).

[!] Conflito conhecido (L0): BASE e o “ECC” (outro gerenciador de CLAUDE.md) **competem pelo mesmo recurso** — ambos querem ser o orquestrador do CLAUDE.md/.claude/. BASE inclusive embute /base:audit-claude para detectar exatamente esse tipo de sprawl/duplicação de config. Rodar dois gerenciadores de CLAUDE.md simultaneamente causa hooks duplicados e injeção de contexto redundante (incompatibilidade estrutural, não apenas sobreposição). Ver seção “Overlaps & Conflitos”.

L8 — MEMORY

mem0ai/mem0 — DEFAULT memory substrate

Identidade & maturidade. “Universal memory layer for AI Agents”. RUN 0 ~57–59K [sec]; página observada mostra **58.2k / 6.7k forks**, “2,239 Commits”, **331 releases** (github.com/mem0ai/mem0). Y Combinator S24. Maturidade alta: paper arXiv (2504.19413), benchmark framework open-source separado (mem0ai/memory-benchmarks). É o **substrato de memória default** do grupo.

Propósito (escopo, casos de uso, persona). Camada de memória *universal* e agnóstica de harness. Persona = um *bibliotecário de longo prazo* que lembra preferências de usuário, fatos de sessão e estado de agente. Casos de uso (README): AI assistants, customer support, healthcare, productivity/gaming. **Multi-Level Memory:** retém estado User, Session e Agent com personalização adaptativa.

Capabilities (artefatos, automação, cobertura de harness). Três modos de deploy: **Library** (pip install mem0ai/npm install mem0ai), **Self-Hosted Server** (docker compose up, com auth por default), **Cloud Platform** (zero-ops, app.mem0.ai). CLI dedicada (@mem0/cli / mem0-cli) com

sign-up “as an agent” em <5s. **Novo algoritmo (April 2026):** single-pass ADD-only extraction, entity linking, multi-signal retrieval (semantic + BM25 + entity), temporal reasoning — LoCoMo 91.6, LongMemEval 94.8. Cobertura de harness via **Agent Skills**: Claude Code, Codex, Cursor, Windsurf, OpenCode, OpenClaw (npx skills add ... --skill mem0); diretórios .claude-plugin/, .codex-plugin/, .cursor-plugin/ no repo.

Lentes. - *OSS posture*: **License = Apache-2.0** (exato, README “Apache 2.0 — see the LICENSE file”). - *S-SDLC fit*: alto — auth-on-by-default no self-hosted, ADMIN_API_KEY, controle de chaves. Memória de agente é dado sensível; mem0 trata isso como primeiro-classe. - *SaaS multi-tenant fit*: **forte** — Cloud Platform é multi-tenant zero-ops; filtros por user_id/agent_id são nativos. - *SDD-methodology fit*: indireto (memória apoia qualquer metodologia; não impõe spec). - *MoE expert-profile*: layer=L8; role=memory-substrate-default; competence=long-term personalization + token-efficient retrieval; persona=librarian; abstraction=add/search/API; authorization=read-write sobre o store de memória, isolado por tenant.

SECURITY posture (incl. license). Apache-2.0 (permissivo, patent-grant). Self-hosted com auth obrigatória. **Razão de ser o default (contexto de segurança RUN 0):** MemPalace foi **excluído** no RUN 0 por alegações *secundárias e não comprovadas* de manipulação de estrelas + benchmark não-auditável. mem0, em contraste, abre o **framework de avaliação** (memory-benchmarks) para reprodução independente e publica paper revisável — auditabilidade verificável é o diferencial. (Nota: não se afirma fraude de MemPalace como fato; é a justificativa documentada para preferir o substrato com benchmark reproduzível.)

INTEGRATION SURFACE (mecanismos): {MCP, memory, filesystem, git-repo, rules, TCP} (MCP-exposável via OpenMemory/server; REST/TCP via self-hosted server; skills como rules).

letta-ai/letta — ex-MemGPT

Identidade & maturidade. “Platform for stateful agents: AI with advanced memory that can learn and self-improve over time.” RUN 0 22,891; página observada **23.5k / 2.5k forks**, “7,466 Commits”, **177 releases** (v0.16.8) (github.com/letta-ai/letta). Origem acadêmica forte (MemGPT, UC Berkeley; CITATION.cff). Maturidade alta como *runtime*, não apenas biblioteca.

Propósito (escopo, casos de uso, persona). Diferentemente de mem0/cognee (camadas de memória *plugáveis*), Letta é um **runtime de agente completo** com memória OS-tiered self-editing como peça central. Persona = um *agente-com-sistema-operacional-de-memória* que edita seus próprios memory_blocks (label human/persona). Casos de uso: agentes stateful que aprendem e se auto-melhoram; Letta Code (CLI no terminal) e Letta API (embutir em apps).

Capabilities (artefatos, automação, cobertura de harness). **Letta Code** CLI (npm install -g @letta-ai/letta-code) com skills + subagents pré-construídos para memória avançada/continual learning; **Letta API** com SDKs Python/TypeScript. memory_blocks self-editing; model-agnostic (recomenda Opus 4.5 / GPT-5.2; tem leaderboard próprio). Self-hostável (compose.yaml, Postgres + pgvector via init.sql; otel/ para tracing; WEBHOOK_SETUP.md). Cobertura de harness: é o *próprio harness* (Letta Code), além de expor API para integração.

Lentes. - *OSS posture*: **License = Apache-2.0** (README + LICENSE). Tem AI_POLICY.md, SECURITY.md, PRIVACY.md, TERMS.md — governança madura. - *S-SDLC fit*: alto — SECURITY.md formal, webhooks, OTEL embutido. - *SaaS multi-tenant fit*: forte — app.letta.com (hosted), API-key, deploy in-your-cloud. - *SDD-methodology fit*: baixo-moderado (runtime, não spec-driven). - *MoE expert-profile*: layer=L8 (mas é também um L1/runtime — *boundary expert*); role=stateful-agent-runtime + self-editing-memory; competence=continual learning, memory hierarchy; persona=self-improving-agent; abstraction=memory_blocks + agents API; authorization=read-write sobre próprio estado de memória (self-edit).

SECURITY posture (incl. license). Apache-2.0. Governança de IA explícita (AI_POLICY.md). Self-hostable com Postgres próprio = controle de dados. **Tensão competitiva:** por ser runtime completo, há *lock-in* — usar a memória do Letta implica usar o agente Letta (vs mem0/cognee que são memória

plugável em qualquer agente). O README do agentmemory cita Letta como “*High (must use Letta)*” em framework lock-in.

INTEGRATION SURFACE (mecanismos): {memory, webhooks, TCP, git-repo, filesystem} (API REST/SDK; webhooks nativos; OTEL para observability). MCP não é o mecanismo primário (é runtime-first).

topoteretes/cognee — graph-native control-plane

Identidade & maturidade. “the open-source AI memory platform for agents... persistent long-term memory across sessions with a self-hosted knowledge graph engine.” RUN 0 17,453; página observada **23.3k / 2.2k forks, 121 releases** (v1.2.2, 2026-06-26), 8,426 commits (github.com/topoteretes/cognee). Primary language Python 85%. Maturidade alta e em rápida ascensão.

Propósito (escopo, casos de uso, persona). Memória **graph-native** como *control-plane*: combina vector embeddings + graph reasoning + ontologia “cognitive-science-grounded”. Persona = um *arquiteto de conhecimento* que constrói um grafo de conhecimento auto-hospedado a partir de dados em qualquer formato (“Build Company Brain”). API de quatro operações: remember, recall (busca com auto-routing), forget, improve. **cognee 1.0** roda toda a camada de memória num único Postgres (relações + embeddings + sessões + metadados), “~10% faster than the separate graph-plus-vector setup” em CI.

Capabilities (artefatos, automação, cobertura de harness). Pipeline implícito add + cognify + improve (“remember() runs add + cognify + improve”). **MCP server dedicado** (cognee-mcp, imagem Docker cognee/cognee-mcp, transports HTTP/SSE/stdio). CLI (cognee-cli com remember/recall/forget/-ui) + UI local. **Backends graph:** Postgres (default), Neo4j, Neptune, Kuzu. **Backends vector:** pgvector (default), LanceDB, Qdrant, ChromaDB, Weaviate, Milvus. Cobertura de harness: **plugin Claude Code oficial** (claude plugin marketplace add topoteretes/cognee-integrations → cognee-memory@cognee) com hooks de lifecycle (SessionStart, UserPromptSubmit, PostToolUse, Stop, PreCompact, SessionEnd); **OpenClaw** (@cognee/cognee-openclaw); clients Python/Rust/TypeScript.

Lentes. - *OSS posture:* **License = Apache-2.0** (badge + sidebar). - *S-SDLC fit:* alto — isolamento agêntico por user/tenant, traceability, **OTEL collector** embutido, audit traits. - *SaaS multi-tenant fit:* forte — isolamento de tenant nativo; deploy self-hosted ou Docker. - *SDD-methodology fit:* moderado — ontologia prescrita via tipos é spec-adjacente (modela o domínio antes de ingerir). - *MoE expert-profile:* layer=L8 (com forte componente L4-knowledge — é graph-native); role=memory-control-plane / knowledge-graph; competence=graph reasoning + ontologia + unified store; persona=knowledge-architect; abstraction=remember/recall/cognify; authorization=read-write sobre grafo+vetores, isolado por tenant. - *Nota sobre framing “ECL”/“control-plane”:* o termo exato “Extract-Cognify-Load” / “control plane” **não aparece** no README capturado; o framing mais próximo é o pipeline add → cognify → improve. Tratar “control-plane” como caracterização funcional (correta), não como termo-de-marca verbatim. [UNVERIFIED] quanto ao acrônimo ECL específico.

SECURITY posture (incl. license). Apache-2.0. OTEL + audit traits + tenant isolation = postura defensável para produção. A unificação em Postgres único reduz a superfície (menos serviços = menos vetores).

INTEGRATION SURFACE (mecanismos): {MCP, memory, filesystem, git-repo, hooks, rules, TCP} (MCP dedicado; hooks Claude Code; HTTP/SSE/stdio; graph+vector DBs via TCP).

getzep/graphiti — temporal knowledge graph (BRIDGE L4↔L8)

Identidade & maturidade. “Build Real-Time Knowledge Graphs for AI Agents” / “Build Temporal Context Graphs for AI Agents”. RUN 0 26,380; página observada **27.8k / 2.8k forks, 196 releases** (v0.29.2, 2026-06-08) (github.com/getzep/graphiti). Python 99.4%. Paper arXiv 2501.13956 (“Zep: A Temporal Knowledge Graph Architecture for Agent Memory”). **Tratado como BRIDGE L4↔L8** — é simultaneamente engine de grafo de conhecimento (L4) e substrato de memória de agente (L8).

Propósito (escopo, casos de uso, persona). Grafo de **contexto temporal** onde cada fato tem janela de validade (“Kendra loves Adidas shoes as of March 2026”). Diferente de knowledge graphs estáticos, rastreia *o que é verdade agora vs o que era verdade antes*, com proveniência total até os **episodos** (dados crus). Persona = um *historiador bi-temporal* que invalida fatos em vez de deletá-los. Componentes: Entities (nodes), Facts/Relationships (edges com validade temporal), Episodes (proveniência), Custom Types (ontologia via Pydantic).

Capabilities (artefatos, automação, cobertura de harness). `pip install graphiti-core`. Backends: **Neo4j 5.26 / FalkorDB / Amazon Neptune / Kuzu (deprecated)**. Hybrid retrieval (semantic + BM25 + graph traversal), sub-second latency, fact invalidation automática. **MCP server** (`mcp_server/`) — “Give Claude, Cursor, and other MCP clients powerful context graph-based memory with temporal awareness” (README). REST service (FastAPI, `server/`). **OTEL tracing** (OTEL_TRACING.md). LLM-agnostic (OpenAI default; Anthropic, Gemini, Groq, Azure, Ollama/vLLM/local via OpenAI-compatible). Telemetria anônima opt-out (PostHog).

Lentes. - *OSS posture*: **License = Apache-2.0** (badge + LICENSE + Zep-CLA.md para contribuições). - *S-SDLC fit*: alto — proveniência/lineage de cada fato é auditoria por design; OTEL embutido. - *SaaS multi-tenant fit*: via **Zep** (a infra gerenciada comercial cujo core OSS é o graphiti) — sub-200ms a escala, governança, dashboard. graphiti puro é self-hosted only. - *SDD-methodology fit*: moderado-alto — ontologia *prescrita* via Pydantic models é spec-driven knowledge modeling. - *MoE expert-profile*: layer=L4↔L8 bridge; role=temporal-knowledge-graph-engine; competence=bi-temporal fact management + provenance; persona=bi-temporal-historian; abstraction=episodes/entities/facts; authorization=read-write sobre o grafo.

SECURITY posture (incl. license). Apache-2.0 (+ CLA para contribuidores). SECURITY.md formal. Telemetria explícita e opt-out, com lista clara do que **não** é coletado (“never: API keys or credentials... your actual data”). Proveniência temporal = trilha de auditoria nativa.

INTEGRATION SURFACE (mecanismos): {MCP, memory, git-repo, filesystem, TCP} (MCP server; REST/FastAPI; graph DBs Neo4j/FalkorDB/Neptune via TCP; OTEL export).

Relação BASELINE/NEIGHBOR. **Zep** = SaaS (graphiti é seu core OSS) — relação engine↔plataforma idêntica ao par Graphiti/Zep documentado no README (“Choose Zep [turnkey]... Choose Graphiti [flexible OSS core]”).

rohitg00/agentmemory — coding-agent memory (NEIGHBOR)

Identidade & maturidade. “Persistent memory for AI coding agents... No more re-explaining.” RUN 0 14,818, Apache-2.0. Construído sobre o **iii engine** (github.com/rohitg00/agentmemory). npm @agentmemory/agentmemory; pinado a iii-engine v0.11.2. NEIGHBOR especializado em **memória de coding-agent** (não memória genérica de assistente).

Propósito (escopo, casos de uso, persona). Foco estreito e deliberado: memória para *coding agents*, capturada silenciosamente via hooks. Persona = um *gravador-de-sessão-de-código* que elimina o re-explicar arquitetura a cada sessão. Exemplo canônico do README: Session 1 monta JWT auth → Session 2 pede rate limiting e o agente “*already knows your auth uses jose middleware in src/middleware/auth.ts*”. Diferencial declarado: built-in memory (CLAUDE.md, .cursorrules) “caps out at 200 lines and goes stale”.

Capabilities (artefatos, automação, cobertura de harness). Servidor de memória local (:3111) + real-time viewer (:3113) com **Session Replay** (importa JSONL do Claude Code). **53 MCP tools**,

12 auto hooks, 0 external DBs (SQLite + iii-engine). Busca híbrida **BM25 + Vector + Graph (RRF fusion)**, 4-tier consolidation + decay + auto-forget. Embeddings locais (all-MiniLM-L6-v2, \$0). **Cobertura de harness extremamente ampla** — “any agent that speaks MCP or HTTP”: Claude Code (plugin nativo + 12 hooks + MCP), Codex CLI (plugin + 6 hooks), OpenCode (22 hooks), OpenClaw, Hermes, pi, Cursor, Gemini CLI, Cline, Goose, Kilo Code, Aider (REST), Claude Desktop, Windsurf, Roo Code. Templates de deploy (fly.io, Railway, Render, Coolify).

Lentes. - *OSS posture*: **License = Apache-2.0** (RUN 0). Badge de licença presente (shields.io/github/license). - *S-SDLC fit*: alto para o nicho — audit policy codificada em “every delete path”, governance_delete tool, HMAC secret no deploy, drop de privilégios root→node via gosu. - *SaaS multi-tenant fit*: moderado — self-hosted por default; coordenação multi-agent via “MCP + REST + leases + signals”; deploy templates mas não SaaS gerenciado. - *SDD-methodology fit*: baixo (memória, não spec). - *MoE expert-profile*: layer=L8; role=coding-agent-memory-specialist; competence=hook-driven silent capture + hybrid retrieval; persona=session-recorder; abstraction=remember/observe/smart-search (iii functions); authorization=read-write local, HMAC-gated.

SECURITY posture (incl. license). Apache-2.0. **Dependência crítica de supply-chain**: todo o sistema é “already a running iii instance” — funções, triggers, KV, streams, OTEL são primitivos do iii-engine (binário nativo separado, **não publicado no crates.io**, instalado via script sh/Docker/binário pré-compilado). Isso concentra risco no upstream iii-hq/iii. Viewer fica em loopback; só :3111 é publicado.

INTEGRATION SURFACE (mecanismos): {MCP, memory, hooks, filesystem, git-repo, rules, TCP, webhooks} (53 MCP tools; REST :3111; iii functions via ws://; hooks de lifecycle; triggers HTTP/cron/event).

L9 — OBSERVABILITY

langfuse/langfuse — DEFAULT observability

Identidade & maturidade. “Open source LLM engineering platform: LLM Observability, metrics, evals, prompt management, playground, datasets.” RUN 0 28,219; página observada **27.2k / 2.8k forks**, 7,015 commits (github.com/langfuse/langfuse). Y Combinator W23. **Default de observability** do grupo. Backed por ClickHouse.

Propósito (escopo, casos de uso, persona). Plataforma de **LLMOps/LLM-engineering** para times “collaboratively develop, monitor, evaluate, and debug AI applications” (README). Persona = um *engenheiro-de-observabilidade* que dá visibilidade humana sobre traces de LLM, retrieval, e ações de agente. Cinco pilares: Observability (tracing), Prompt Management (versionado), Evaluations (LLM-as-judge + feedback + manual), Datasets (benchmarks), Playground.

Capabilities (artefatos, automação, cobertura de harness). **OTel-native** — LangfuseSpan-Processor plugado no @opentelemetry/sdk-node; ingere qualquer app que emita spans OTLP. SDKs tipados Python + JS/TS; OpenAI drop-in (from langfuse.openai import openai). Deploy: Docker Compose / VM / **Kubernetes (Helm, preferido prod)** / Terraform (AWS/Azure/GCP). **Regiões de dados incl. HIPAA** (hipaa.cloud.langfuse.com). Cobertura de harness/framework muito ampla: OpenTelemetry, LangChain, LlamaIndex, Vercel AI SDK, LiteLLM (100+ LLMs), CrewAI, AutoGen, Google ADK, Haystack, DSPy, smolagents, Goose, etc. **Integração agêntica direta**: “Langfuse Agent Skill” (github.com/langfuse/skills, npx skills add langfuse/skills), **Cursor Plugin** (cursor.com/marketplace/langfuse), e **Docs MCP Server** (langfuse.com/docs/docs-mcp). Isto confirma o “Claude plugin” mencionado no RUN 0 via o canal de skills/plugin cross-harness.

Lentes. - *OSS posture*: **License = NOASSERTION / source-available** — “This repository is MIT licensed, except for the ee folders” (README). RUN 0 registrou license=NOASSERTION (GitHub não consegue classificar por causa do split MIT-core + EE). Tratar como **MIT-core com pastas Enterprise Edition proprietárias**. - *S-SDLC fit*: alto — prompt management versionado + evals em CI são práticas seguras de SDLC; região HIPAA; página Security & Guardrails. - *SaaS multi-tenant fit*: **forte** — Langfuse Cloud multi-região (EU/US/Japan/HIPAA) + Enterprise tier; self-host com Helm. -

SDD-methodology fit: moderado — datasets + evals estruturados aproximam-se de spec-driven validation. - *MoE expert-profile*: layer=L9; role=observability-default + LLMOps; competence=tracing + evals + prompt-mgmt; persona=observability-engineer; abstraction=traces/observations/spans (OTel); authorization=read-write sobre store de traces, multi-tenant por project.

SECURITY posture (incl. license). Modelo source-available, NÃO 100% OSS — o core é MIT mas as pastas ee/ são proprietárias (Enterprise Edition). Esta é a distinção de licença mais importante do grupo: ao contrário de mem0/letta/cognee/graphiti (Apache-2.0 puro), langfuse tem componentes não-livres. Compliance: região HIPAA dedicada, SOC2/segurança documentados em langfuse.com/security. Self-hostável em minutos = controle de dados.

INTEGRATION SURFACE (mecanismos): {MCP, webhooks, filesystem, git-repo, rules, TCP, hooks} (OTel/OTLP ingest via TCP; Docs MCP server; Agent Skill + Cursor plugin como rules; SDK; ClickHouse backend).

traceloop/openllmetry — OTel instrumentation (NEIGHBOR)

Identidade & maturidade. “Open-source observability for your GenAI or LLM application, based on OpenTelemetry.” RUN 0 7,153 Apache-2.0; página observada **7.1k / 960 forks, 257 releases** (0.60.0) (github.com/traceloop/openllmetry). YC. Python 100%. NEIGHBOR de instrumentação.

Propósito (escopo, casos de uso, persona). Não é uma plataforma — é o **conjunto de instrumentações OTel** que alimenta plataformas. Persona = um *plugue-padrão* que emite spans OpenTelemetry conectáveis a Datadog, Honeycomb, Grafana, e qualquer backend OTel. “*It’s built and maintained by Traceloop under the Apache 2.0 license.*” Suas semantic conventions foram **adotadas pelo OpenTelemetry** (“*Our semantic conventions are now part of OpenTelemetry*”).

Capabilities (artefatos, automação, cobertura de harness). pip install traceloop-sdk + Traceloop.init() (uma linha). Instrumenta LLM providers (OpenAI, Anthropic, Bedrock, Cohere, Gemini, Groq, Mistral, Ollama, etc.), Vector DBs (Chroma, Pinecone, Qdrant, Weaviate, Milvus, LanceDB, Marqo), frameworks (LangChain, LlamaIndex, CrewAI, Haystack, LangGraph, AWS Strands, OpenAI Agents, Agno) e **MCP** (o protocolo). ~21 destinos testados. JS/TS via openllmetry-js separado.

Lentes. - *OSS posture*: **License = Apache-2.0** (puro; GOVERNANCE.md + MAINTAINERS.md). - *S-SDLC fit*: moderado-alto (instrumentação é fundação de observabilidade; sem opinião sobre SDLC). - *SaaS multi-tenant fit*: via Traceloop (plataforma comercial); o SDK em si é vendor-neutral. - *SDD-methodology fit*: N/A. - *MoE expert-profile*: layer=L9; role=OTel-instrumentation-foundation; competence=vendor-neutral span emission; persona=standard-plug; abstraction=OTel spans; authorization=read-only (apenas emite telemetria).

SECURITY posture. Apache-2.0 puro (mais permissivo que langfuse). Não coleta mais telemetria no SDK (≥v0.49.2). Por ser vendor-neutral, reduz lock-in. **Pareamento natural**: openllmetry (instrumentação) → langfuse/opik/phoenix/qualquer-OTel-backend (visualização). É *complemento*, não competidor, das plataformas.

INTEGRATION SURFACE (mecanismos): {filesystem, git-repo, TCP} (emissão OTLP/TCP para qualquer backend; instrumenta MCP como protocolo).

lmnr-ai/lmnr — Laminar, agent-debug (NEIGHBOR)

Identidade & maturidade. “Laminar - open-source observability platform purpose-built for AI agents. YC S24.” RUN 0 ~2.8K [sec] Apache-2.0 (ex-laminar, renomeado para lmnr); página observada **3k / 205 forks, 86 releases** (v0.1.46) (github.com/lmnr-ai/lmnr). TypeScript 72% + **Rust 23%**. NEIGHBOR focado em *debug de agentes*.

Propósito (escopo, casos de uso, persona). Observabilidade *purpose-built for AI agents* (não LLM apps genéricos). Persona = um *depurador-de-agente em tempo real*. Diferenciais: motor real-time em **Rust** para ver traces *enquanto acontecem*, full-text search ultra-rápido sobre spans, **SQL access** a todos os dados (SQL editor embutido), AI monitoring via eventos descritos em linguagem natural (“signals”).

Capabilities (artefatos, automação, cobertura de harness). OTel-native, 1 linha (`Laminar.initialize(...)` / `@observe()` decorator). Auto-traça Vercel AI SDK, Browser Use, Stagehand, LangChain, OpenAI, Anthropic, Gemini. Evals (SDK + CLI, local/CI). Dashboards + data annotation + datasets. **pii-redactor** e gRPC exporter. Self-host fácil (`docker compose up`, UI em :5667); LLM provider configurável (Gemini/OpenAI/Bedrock).

Lentes. - *OSS posture*: **License = Apache-2.0** (badge). - *S-SDLC fit*: alto — PII redactor nativo + SQL auditável + evals em CI/CD. - *SaaS multi-tenant fit*: via `laminar.sh` (managed). Self-host full via `docker-compose-full.yml`. - *SDD-methodology fit*: moderado (evals + datasets). - *MoE expert-profile*: layer=L9; role=agent-debug-observability; competence=realtime tracing + SQL + agent-specific signals; persona=realtime-debugger; abstraction=OTel spans + SQL; authorization=read-write sobre traces.

SECURITY posture. Apache-2.0. PII redactor embutido (`pii-redactor/`) é diferencial de privacidade. Rust no caminho crítico = menor superfície de bugs de memória. **Nota de identidade (RUN 0)**: confirmado o rename `laminar` → `lmnr` — mesmo projeto, não confundir com outras “laminar”.

INTEGRATION SURFACE (mecanismos): {filesystem, git-repo, TCP} (OTel + gRPC exporter via TCP; SQL API; rules via `rules/` dir no repo).

comet-ml/opik — eval+observability (NEIGHBOR)

Identidade & maturidade. “Debug, evaluate, and monitor your LLM applications, RAG systems, and agentic workflows with comprehensive tracing, automated evaluations, and production-ready dashboards.” RUN 0 18,829 Apache-2.0; página observada **19.7k / 1.5k forks, 493 releases** (2.0.73) (github.com/comet-ml/opik). Python 55% + TS 44%. Por **Comet**. NEIGHBOR full-platform.

Propósito (escopo, casos de uso, persona). Plataforma OSS de observability + evaluation “*from prototype to production*”, cobrindo “*RAG chatbots to code assistants to complex agentic systems*”. Persona = um *avaliador-e-monitor* com forte ênfase em evals automatizados. Quatro pilares: Comprehensive Observability (tracing de chamadas LLM + agent activity), Advanced Evaluation (LLM-as-judge, experiment mgmt), **Opik Agent Optimizer** (SDK + otimizadores de prompts/agents), **Opik Guardrails** (AI segura/responsável).

Capabilities (artefatos, automação, cobertura de harness). SDKs Python, TypeScript, **Ruby (via OpenTelemetry)** + REST API. OTel para chamadas suportadas. Self-host: **Docker Compose** + **Kubernetes/Helm**; Cloud via Comet.com. Integra OpenAI, Anthropic, Gemini, Bedrock, Cohere, Groq, Mistral, LiteLLM, LangChain, LangGraph, LlamaIndex, Haystack, CrewAI, AutoGen, AG2, DSPy, Dify, Flowise, Google ADK, BeeAI; validações Guardrails AI. Integração pytest; LLM-as-judge para hallucination detection + moderation. Prompt Playground.

Lentes. - *OSS posture*: **License = Apache-2.0** (puro). - *S-SDLC fit*: **muito alto** — único do grupo a empacotar **Guardrails + Agent Optimizer** + pytest integration juntos; evals como gate de CI. - *SaaS multi-tenant fit*: via Comet.com / Opik Cloud (managed); self-host Docker/K8s. Multi-tenancy enterprise não destacada no README [UNVERIFIED]. - *SDD-methodology fit*: alto — experiments + datasets + evals automatizados. - *MoE expert-profile*: layer=L9 (+ spillover L0-guardrails via Opik Guardrails); role=eval-centric-observability; competence=automated evals + agent optimization + guardrails; persona=evaluator; abstraction=traces + experiments + optimizers; authorization=read-write sobre traces/experiments.

SECURITY posture. Apache-2.0. **Opik Guardrails** torna-o parcialmente um expert L0 também (safe/responsible AI). Sem claims explícitos de SOC2/HIPAA/RBAC no README ([UNVERIFIED] para multi-tenant enterprise).

INTEGRATION SURFACE (mecanismos): {filesystem, git-repo, TCP, rules, hooks} (OTel/REST via TCP; pytest hooks; Guardrails como rules). MCP server dedicado **não** anunciado no README.

Arize-ai/phoenix — OpenInference-based (NEIGHBOR)

Identidade & maturidade. “AI Observability & Evaluation”. RUN 0 9,918 NOASSERTION; página observada **9.7k / 869 forks** (github.com/Arize-ai/phoenix). Python + TS/JS + Java. Por **Arize AI**. NEIGHBOR construído sobre **OpenInference**.

Propósito (escopo, casos de uso, persona). “an open-source AI observability platform designed for experimentation, evaluation, and troubleshooting” (README), “vendor and language agnostic”. Persona = um observador-vendor-agnóstico que roda “practically anywhere... local machine, Jupyter notebook, containerized deployment, or in the cloud”. Capabilities: Tracing (OTel-based), Evaluation (response + retrieval evals), Datasets (versionados), Experiments, Playground, Prompt Management.

Capabilities (artefatos, automação, cobertura de harness). pip install arize-phoenix / conda; Docker Hub (arizephoenix/phoenix); Helm; Jupyter notebook. Construído sobre **OpenTelemetry + OpenInference** (projeto de auto-instrumentação separado, openinference-instrumentation-*). **MCP server** (@arizeai/phoenix-mcp — “unified interface to Phoenix’s capabilities”, badge “MCP Enabled” + deeplink Cursor). **Integração de coding-agent forte:** @arizeai/phoenix-cli é “CLI for fetching traces, datasets, and experiments for use with Claude Code, Cursor, and other coding agents”; ships “**Coding Agent Skills**” em .agents/skills/ (phoenix-cli, phoenix-evals) + CLAUDE.md/AGENTS.md no repo. **Claude Agent SDK** instrumentação dedicada (Python + JS). Cobre OpenAI Agents SDK, LangGraph/LangChain (+ Java), CrewAI, LlamaIndex, DSPy, Haystack, Smolagents, Pydantic AI, Autogen, BeeAI, NVIDIA NeMo.

Lentes. - OSS posture: **License = NOASSERTION** (RUN 0). O README capturado **não** mostra a string SPDX exata ([UNVERIFIED] quanto a ser Elastic vs Apache vs BSD; o sidebar de licença não foi capturado). RUN 0 registra NOASSERTION — tratar como source-available/non-standard até confirmar o arquivo LICENSE. - S-SDLC fit: alto — evals + experiments + Claude Code skills para troubleshooting in-loop. - SaaS multi-tenant fit: via **Arize** (cloud em app.phoenix.arize.com); Phoenix OSS é self-host. - SDD-methodology fit: moderado-alto (datasets + experiments versionados). - MoE expert-profile: layer=L9; role=vendor-agnostic-observability + OpenInference; competence=tracing + evals + coding-agent integration; persona=vendor-neutral-observer; abstraction=OTel/OpenInference spans; authorization=read-write sobre traces.

SECURITY posture (incl. license). **License NOASSERTION** (igual flag de cautela que langfuse, mas pior: a string não é nem confirmável do README — requer leitura do LICENSE + IP_NOTICE + CLA.md presentes no repo). Por ser OpenInference/OTel-based, é vendor-neutral. Relação com Arize: Phoenix é o OSS, Arize é a SaaS comercial parente (mesmo padrão engine↔plataforma de graphiti↔Zep).

INTEGRATION SURFACE (mecanismos): {MCP, filesystem, git-repo, rules, TCP, hooks} (@arizeai/phoenix-mcp; phoenix-cli para coding agents; Coding Agent Skills como rules; OTel/OpenInference via TCP).

OVERLAPS, COLABORAÇÃO vs COMPETIÇÃO vs INCOMPATIBILIDADES (com evidência)

L0 Guardrails — competição estrutural por um recurso único

karpathy-skills vs BASE: sobreposição parcial, não competição direta. karpathy-skills é conteúdo de princípios (estático, prosa); BASE é gestão de estado de workspace (dinâmico,

JSON+hooks+MCP). Podem coexistir: karpathy-skills define *como pensar*, BASE define *o que está no contexto*. Mas ambos escrevem em/sobre `CLAUDE.md/.claude/`.

BASE vs ECC: INCOMPATIBILIDADE estrutural (evidência). O prompt RUN 0 marca BASE como “rules-OS / CLAUDE.md manager (conflicts with other CLAUDE.md managers e.g. ECC)”. A evidência interna corrobora: BASE embute `/base:audit-claude` que detecta exatamente “*duplicated hooks running twice, stale config referencing nonexistent tools, skills duplicated across project dirs*” e o upgrade v2→v3 arquiva cópias de hooks “*double-fire-causing*”. Dois gerenciadores de `CLAUDE.md` rodando juntos produzem hooks que disparam em duplicata e injeção redundante de contexto. **Veredito:** escolher UM orquestrador de `CLAUDE.md` por workspace; rodar BASE + ECC simultaneamente é anti-padrão.

L8 Memory — backends concorrentes (mem0 vs letta vs cognee vs graphiti vs agentmemory)

A competição mais direta do grupo. Há evidência *de primeira mão* num dos próprios experts: o README do agentmemory publica uma tabela comparativa explícita “vs Competitors” (agentmemory vs mem0 vs Letta vs Built-in):

Eixo (do README agentmemory)	agentmemory	mem0	Letta/MemGPT
Type	Memory engine + MCP server	Memory layer API	Full agent runtime
Search	BM25+Vector+Graph (RRF)	Vector+Graph	Vector (archival)
Framework lock-in	None (any MCP)	None	High (must use Letta)
External deps	None (SQLite+iii)	Qdrant/pgvector	Postgres+vector DB

Distinções arquiteturais que determinam competição vs coexistência: - **letta é categorialmente diferente** — é um *runtime de agente completo*, não uma camada plugável. Competir “letta vs mem0” é parcialmente uma falsa dicotomia: usar letta = usar o agente letta (lock-in alto); usar mem0/cognee/agentmemory = plugar memória em *qualquer* agente. **Incompatibilidade prática:** letta não é um “backend de memória” que se acopla a Claude Code do mesmo jeito que mem0 — é um harness rival. - **mem0 vs cognee:** ambos plugáveis, ambos Apache-2.0, ambos com MCP + skills cross-harness. Diferença = **modelo de dados:** mem0 é memory-layer (semantic+BM25+entity, store flexível); cognee é **graph-native** (knowledge graph + ontologia como control-plane). Competem pelo slot “substrato de memória default” — RUN 0 elege **mem0 como default** (benchmark reproduzível/auditável); cognee é a alternativa graph-first quando o caso de uso exige raciocínio sobre relações. - **graphiti é o BRIDGE L4↔L8** — não compete *de frente* com mem0 porque resolve um eixo ortogonal: **temporalidade** (fatos com janela de validade, invalidação automática). Pode *complementar* mem0 (mem0 para preferências, graphiti para fatos que mudam no tempo) OU competir com cognee (ambos graph-native). Diferença cognee vs graphiti: cognee = ontologia cognitive-science + unified Postgres; graphiti = bi-temporal + proveniência por episode. - **agentmemory é nicho** — só coding-agents, hook-driven, 0 DBs externos. Não compete com mem0 no mercado de “memória de assistente genérico”; compete no slot estreito “memória de coding-agent” — onde sua tabela o posiciona contra `CLAUDE.md/.cursorrules` built-in mais do que contra mem0.

Incompatibilidade de backend compartilhado: mem0, cognee, graphiti e agentmemory **todos** podem querer pgvector/Postgres/Qdrant. Rodar múltiplos substratos de memória sobre o mesmo store sem namespacing causa colisão de embeddings — escolher UM substrato primário de memória por agente é a recomendação (embora graphiti-como-bridge + mem0-como-layer seja uma composição válida por serem eixos diferentes).

L9 Observability — instrumentação vs plataforma (composição, não competição)

openllmetry NÃO compete com langfuse/opik/phoenix — ele os ALIMENTA. Evidência: openllmetry é “a set of extensions built on top of OpenTelemetry” cujas “semantic conventions are now

part of OpenTelemetry”; emite OTLP para “Datadog, Honeycomb, and others”. langfuse, opik e phoenix são todos **OTel-native** (langfuse via LangfuseSpanProcessor; phoenix via OpenInference; opik via OTel SDKs). **Pareamento canônico:** openllmetry (instrumentação vendor-neutral) → qualquer backend OTel (langfuse/opik/phoenix/lmnr) para visualização. Esta é a relação de *colaboração* mais limpa do grupo inteiro.

langfuse vs opik vs phoenix vs lmnr: competição real entre plataformas full-stack. Todas fazem tracing + evals + datasets + playground. Diferenciadores: - **langfuse** = default, prompt-management mais maduro, mais integrações de framework, região HIPAA. **Mas:** licença source-available (MIT-core + ee/ proprietário) — a única não-Apache-pura entre as plataformas-default. - **opik** = único com **Guardrails + Agent Optimizer + pytest** empacotados (mais “S-SDLC”); Apache-2.0 puro. - **phoenix** = vendor-agnóstico via OpenInference, mais forte em coding-agent skills (phoenix-cli para Claude Code/Cursor) + Claude Agent SDK instrumentation; **mas** licença NOASSERTION (cautela). - **lmnr** = purpose-built para *agentes* (não LLM apps), realtime Rust, SQL access, PII redactor; menor/mais novo (3k).

Qual observability parecia com qual memory? Não há acoplamento forçado — todos os pares funcionam por serem camadas ortogonais. Pareamentos naturais por afinidade: - **graphiti + langfuse/phoenix** — ambos têm OTEL embutido; graphiti emite traces que langfuse/phoenix ingerem nativamente. - **cognee + qualquer-OTel-backend** — cognee tem OTEL collector embutido. - **mem0 (default) + langfuse (default)** — o par “default×default” recomendado pelo RUN 0; ambos maduros, multi-tenant, com skills cross-harness para Claude Code. - **opik** se o requisito for *evals-como-gate + guardrails* no S-SDLC.

Padrão recorrente: OSS-engine ↔ SaaS-platform

Três experts seguem o mesmo modelo *open-core*: **graphiti↔Zep, phoenix↔Arize, langfuse-core↔langfuse-EE/Cloud**. Implicação de governança: o roadmap do OSS é influenciado pelo produto comercial; features enterprise (governança, SLA, sub-200ms) ficam atrás de paywall ou em pastas ee/. mem0 e letta também têm Cloud, mas mantêm Apache-2.0 puro no core.

MATRIZ DE COMPARAÇÃO

expert layer	role	license	stars+	data-type	harness	MCP?	git-flow	S-SDLC	multi-tenant	memory	observability	security
multicaLO ai/andrej-karpathy-skills	guardrails (princípios de código)	proprietary (README diz MIT; sem código)	154k	arquivo único de princípios	Claude Code, Cursor	não	só rules/repo	médio (policy baseline)	N/A	não	não	mínima (texto); risco=provenar
Christopher DeKorke	state / config-hygiene	proprietary (README diz MIT; sem código)	87	OS de workspace anti-staleness	Claude Code MCP, 20 tools	sim (BASE MCP, 20 tools)	leitura p/ detecção	médio (con-fig hygiene)	não (JSON local)	sim (PSMM/score)	parcial hooks	hooks exec local; [!]con-flito c/ ECC

expert	layer	role	license	stars+	date	type	harness	MCP?	git-flow	S-SDLC	multi-tenant	memory	observability	security
mem0ai/	mem0	memory de-fault	Apache 2.0	58k	2026-06-27	memória universal plugável	Claude Code, Codex, Cursor, Windsurf, Open-Code, OpenClaw	sim	rules/repo/skills	plano (server on)	forte	sim (core)	não	Apache-2.0; benchmark reproduzível
letta-ai/letta	L8(+L1 stateful runtime agent + self-edit mem)	Apache 2.0	23.5k	2026-06-27	runtime de agente har-ness memórias OS-tiered + API	é o próprio agente har-ness (Letta Code)	parcial (não primário)	repo/Claw	alto (SE-CURITY.md, OTEL)	forte (app.letta.com)	sim (self-edit)	OTEL	Apache-2.0; lock-in de runtime	
topoter/	EBs/Log memory control plane / KG	Apache 2.0	23.3k	2026-06-27	memória graph-native + ontologia	Claude Code (cognitive+hooks)	sim	hooks + mcp repo (Docker)	alto (OTEL, tenant iso)	forte (tenant iso)	sim (graph vector)	OTEL	Apache-2.0; unified Postgres	
getzep/	4+L8 temporal knowledge graph	Apache 2.0	27.8k	2026-06-27	grafo de contexto bi-temporal	Claude Cur-sor, MCP clients	sim	repo/Claw (mcp_server)	alto (proveniência)	via Zep (SaaS)	sim (tempo-ral)	OTEL	Apache-2.0; TRACIA.md; lineage nativo	
rohitg007/	Agent coding agent mem-ory (NEIGHBOR)	Apache 2.0	14.8k	2026-06-27	memória hook-driven p/coding agents	B+ (Claude 53 Code, Codex, Cur-sor, Open-Code, Gemini, Cline, Goose, Aider...)	sim	hooks + repo (tools)	alto p/nicho (audit paths, HMAC)	moderado (self-host)	sim (BM25 over + graph)	iii embutido	Apache-2.0; dep iii-engine (não em crates.io)	
langfuse/	langfuse observability de-fault + LLM Ops	MIT-core + LLM Ops propr.)	2026-06-27	2026-06-27	tracing mgmt	OTEL + LangChain MCP + Skill/Cursor plugin	sim	repo/Claw (Docs MCP)	alto (HIPAA, CompAn... versão reg)	forte	não	sim (core)	source-available (ee/não-livre); HIPAA	

expert layer	role	license	stars+date	scope	harness	MCP?	git-flow	S-SDLC	multi-tenant	memory	observability	security
traceloop/	open-orchestrator	Apache 2.0	7.1k · 2026-06-27	instrumentação vendor-back-neutral	qualquer instrumentação	sim	CI	médio-alto (fundação)	via Traceloop	não	sim (emite OTLP)	Apache-2.0 puro; sem telemetria SDK
lmnr-ai/lmnr (Laminar)	L9 agent-debug observ-serv-abilidade (NEIGHBOR)	Apache 2.0	3k · 2026-06-27	observação p/ agentes real-time Rust	Alto, Browser Use, LangChain, OpenAI/Anthropic/Gemini	parcial (OTel/gRPC)	repo	alto (PII redactor, SQL)	via laminar.sh	não	sim (real-time)	Apache-2.0; PII redactor embutido
comet-ml/opik	L9 (+L0) eval-centric observ-serv-abilidade (NEIGHBOR)	Apache 2.0	19.7k · 2026-06-27	observação	OpenAI/Anthropic/Gemini/Google (sem hooks dedicados)	sim	hooks + repo	alto (Guardrails enterprise)	Cloud (AWS, Azure, GCP)	não	sim (eval-first)	Apache-2.0; Guardrails embutido
Arize-ai/phoenix	L9 vendor-agnostic observ-serv-abilidade (NEIGHBOR)	NOASSERTION [UNVERIFIED]	2026-06-27	tracing via OpenInference	Claude Code CLI+skills), Claude Agent SDK, LangGraph...	sim	skills	alto	via Arize (app.phoenix.ai)	não	sim (Open-fer-ence)	NOASSERTION (cautela); vendor-neutral

Legenda: = ordem de grandeza datada (2026-06-27). "RUN0=" indica discrepância com o valor de licença do RUN 0. [!] = ponto de atenção (licença/segurança/conflicto).

Fontes

Fontes PRIMÁRIAS (repo README/docs), via mcp__workspace__web_fetch, observadas 2026-06-27:

L0 Guardrails - <https://github.com/multica-ai/andrej-karpathy-skills> (README: 4 princípios, license MIT, 183k) - <https://github.com/ChristopherKahler/base> (README: BASE MCP 20 tools, hooks, /base:audit-claude, drift score, MIT)

L8 Memory - <https://github.com/mem0ai/mem0> (README: Apache-2.0, novo algoritmo April 2026, modos library/self-hosted/cloud, agent skills cross-harness) - <https://github.com/letta-ai/letta> (README: Apache-2.0, Letta Code CLI, memory_blocks self-edit, OTEL, runtime) - <https://github.com/topoteretes/cognee> (README: Apache-2.0, cognee-mcp, graph+vector backends, Claude Code plugin+hooks, OTEL collector, v1.2.2) - <https://github.com/getzep/graphiti> (README: Apache-2.0, temporal context graph, MCP server, Neo4j/FalkorDB/Neptune, OTEL_TRACING.md, paper arXiv 2501.13956, Zep relationship) - <https://raw.githubusercontent.com/rohitg00/agentmemory/main/README> (README: Apache-2.0, 53 MCP tools, 12 hooks, BM25+Vector+Graph RRF, tabela vs mem0/Letta, iii-engine dependency, 16+ harnesses)

L9 Observability - <https://github.com/langfuse/langfuse> (README: MIT-core+ee/, ClickHouse, YC W23) - <https://langfuse.com/docs/observability/get-started> (docs: OTel-native LangfuseSpanProcessor, Agent Skill github.com/langfuse/skills, Cursor plugin, Docs MCP, região HIPAA) - <https://github.com/traceloop/openllmetry> (README: Apache-2.0, OTel semantic conventions adotadas pelo OTel, ~21 destinos, instrumenta MCP) - <https://github.com/lmnr-ai/lmnr> (README: Apache-2.0, Laminar, Rust realtime, SQL access, pii-redactor, YC S24) - <https://github.com/comet-ml/opik> (README: Apache-2.0, Opik Guardrails + Agent Optimizer, pytest, Docker/K8s, by Comet) - <https://github.com/Arize-ai/phoenix> (README: OpenInference, @arizeai/phoenix-mcp, phoenix-cli para Claude Code/Cursor, Claude Agent SDK instrumentation, Arize relationship)

Fontes secundárias / cross-referência (independentes do repo principal): - Tabela “vs Competitors” do agentmemory (fonte independente sobre mem0 R@5 68.5% LoCoMo, Letta 83.2%, lock-in) - README do graphiti seção “Graphiti and Zep” / “Zep vs Graphiti” (fonte primária sobre a relação engine↔SaaS) - Cross-referência openllmetry↔langfuse/phoenix via convergência OpenTelemetry (semantic conventions compartilhadas, confirmada em ambos os READMEs) - Prompt RUN 0 (valores datados de estrelas/licença/identidade canonicalizada; contexto de exclusão de MemPalace)

Itens [UNVERIFIED]: (1) acrônimo “ECL/Extract-Cognify-Load” / termo “control-plane” verbatim em cognee — não aparece no README (framing funcional confirmado, marca não); (2) string SPDX exata de phoenix — README não a expõe, RUN 0 registra NOASSERTION; (3) claims de multi-tenancy enterprise (SOC2/RBAC) de opik — não no README; (4) discrepâncias de licença L0 (karpathy=MIT no README vs RUN0=NONE; base=MIT no README vs RUN0=none) — resolvidas a favor do README com baixa confiança de provenance.

Parte II — Fase 1: (b) Conhecimento L4/L5

Agentic MoE 2026 — Group 1b: Knowledge Layers (L4 + L5)

Group: 1b — Knowledge Layers

Observed: 2026-06-27

Língua das análises: pt-BR; identificadores, comandos e nomes de repo em en-US

Fase do estudo: Phase 1 — Expert Profile Cards

Nota de Escopo

Este documento cobre a camada de inteligência de codebase (L4) e a camada de PKM pessoal (L5) do mapa MoE. A ponte L4↔L8 via getzep/graphiti (26,380, Apache-2.0) é mencionada como adjacente mas não é perfilada aqui (pertence ao grupo 1a — Substrates).

Perfis dos Experts

EXPERT-L4a — safishamsi/graphify

Identidade e Maturidade

- **Repositório:** <https://github.com/safishamsi/graphify>
- **Stars (observado 2026-06-27):** ~52,657 (RUN 0) — fontes secundárias reportam 56k-69k em maio/junho 2026 com crescimento acelerado sem marketing pago

- **Licença:** MIT
- **Criado em:** 2026-04-03
- **Branch padrão:** v8 (desenvolvimento ativo; main existe mas v8 é o default)
- **Package PyPI:** graphifyy (v0.8.x)
- **Homepage:** <https://graphifylabs.ai>
- **Maturidade:** Early-production — rápida adoção viral; em ativo uso de produção com issues de bugs abertos (e.g., issue #580 sobre eficiência de tokens em sessões Claude Code); segurança de dependências gerenciada ativamente (CVE-2026-48155/48156 corrigidos no changelog v8)

Propósito Graphify resolve o problema de “context window overflow” em repositórios grandes e multi-modais. A proposta central: processar o diretório **uma única vez** no momento de build, produzindo um knowledge graph persistente; sessões subsequentes de IA consultam o grafo pré-indexado em vez de re-ler arquivos brutos. O slogan do repositório é “Turn any folder of code, SQL schemas, R scripts, shell scripts, docs, papers, images, or videos into a queryable knowledge graph.”

Casos de uso principais: - Navegação estruturada de codebases grandes (50k+ linhas) por agentes de IA - Consultas multi-modais: código + papers + imagens + vídeos em um único grafo - PR impact analysis via graphify prs (grafo de blast radius por PR) - Repositórios de pesquisa (papers + experimentos + código conectados) - Onboarding automatizado de novos desenvolvedores

Persona primária: Desenvolvedor sênior ou tech lead gerenciando codebases heterogêneas com múltiplos artefatos (code, SQL, infra, docs); equipes usando multi-agente que precisam de contexto estruturado sem reenviar o repo inteiro ao LLM a cada query.

Capacidades Artefatos produzidos: - Knowledge graph (JSON + NetworkX/GraphML interno) - Visualizações HTML interativas (vis.js) exportáveis - Obsidian Vault + JSON Canvas (export direto para L5!) - Wiki estática agent-crawlable (formato Markdown estruturado) - Neo4j export para grafos de produção - PR dashboard com mapeamento para grafo

Motor de extração (pipeline linear, módulos isolados): 1. detect.py — classifica arquivos por tipo: Code, Doc, Paper, Image, Video 2. extract.py — tree-sitter (23 linguagens incl. Dart, PHP, Zig) + Claude/Whisper para PDFs e áudio 3. build.py / cluster.py — NetworkX + Leiden community detection (sem vector database) 4. analyze.py — centralidade de nós, “God Nodes” (hubs de alto grau), “Surprising Connections” cross-community 5. serve.py / watch.py — MCP server + file watcher incremental (SHA256 semantic cache)

Automação: - File watcher em tempo real: apenas nós afetados são re-processados quando um arquivo muda - Git hooks integration para atualizar grafo em commits - MCP server expõe ferramentas tipadas: query_graph, get_node, get_neighbors, shortest_path

Harness coverage: Claude Code (skill nativo via graphify install), Codex, OpenCode, Cursor, Gemini CLI, Aider, Kiro, e 10+ adicionais

Lentes OSS Posture: MIT puro; sem cloud relay proprietário; todo tráfego vai diretamente máquina → Anthropic/OpenAI com credenciais do próprio usuário. “Open-source and free forever” — graphifylabs.ai. Sem telemetria interna declarada.

S-SDLC fit: Alto. Pipeline de build separado de runtime (grafo pré-compilado antes de sessão de desenvolvimento); integração com git hooks; PR dashboard com blast radius; incremental update via SHA256 cache — adequado para CI/CD gates.

SaaS multi-tenant fit: Baixo por design. Arquitetura é fundamentalmente local/single-tenant. Cada worktree/tenant precisaria de instância de grafo isolada. Não há mecanismo de names-

pace/tenant embutido. Risco: em ambientes multi-tenant onde múltiplos usuários compartilham filesystem, a ausência de controle de acesso ao grafo é um gap.

SDD-methodology fit: Alto para equipes que adotam Spec-Driven Development — o grafo conecta specs (Markdown/PDFs) ao código que as implementa, criando rastreabilidade automática.

MoE Expert Profile: - **Layer:** L4 (Codebase Intelligence) - **Role:** Graph Builder + Query Engine - **Competence:** Structural understanding of heterogeneous repos - **Persona:** Sempre presente; recomendado como skill base para qualquer agente que precise de contexto de codebase - **Abstraction level:** Nó/aresta semântico (acima de AST raw, abaixo de domain model) - **Authorization:** Local-only; sem controle de acesso por usuário

SECURITY posture: - Sem relay server — dados não saem da máquina exceto chamadas diretas a Anthropic/OpenAI com chaves do usuário - Path traversal e label sanitization documentados (DeepWiki §5.2) - URL validation e safe fetch implementados (§5.1) - CVEs de dependência monitorados e corrigidos rapidamente (pypdf, pyjwt, cryptography no changelog v8) - **Risco de primeiro run:** Para projetos com centenas de PDFs/imagens, o Pass 3 (extração semântica via LLM subagents) pode gerar custo API substancial. Cache SHA256 torna runs subsequentes baratos. - **Git-LFS / large artifacts:** Arquivos de vídeo processados via yt-dlp + Whisper localmente; PDFs via pypdf. Sem Git-LFS nativo — arquivos binários grandes devem ser excluídos ou gerenciados externamente.

Integration Surface

Mecanismo	Suporte	Detalhe
MCP	Sim (nativo)	serve.py expõe graph tools via MCP protocol
filesystem	Sim (primário)	Lê qualquer diretório local
git-repo	Sim	Git hooks para atualização pós-commit
git-PR	Sim	graphify prs — blast radius dashboard por PR
git-worktree	Compatível	Cada worktree pode ter grafo isolado; sem automação explícita
memory	Parcial	O grafo persiste em disco como memória estruturada; não é memória conversacional
hooks	Sim	Git hooks integration documentada
A2A	Não (N/A)	—
P2P	Não	—
TCP	Via MCP	MCP server escuta em porta local
webhooks	Não	—
ticket/backlog	Não direto	—
rules	Não	—

Export especial: Graphify exporta diretamente para **Obsidian Vault + JSON Canvas** — esta é a ponte L4→L5 mais concreta do grupo: o grafo de codebase torna-se um vault Obsidian navegável.

EXPERT-L4b — Egonex-AI/Understand-Anything

Identidade e Maturidade

- **Repositório:** <https://github.com/Egonex-AI/Understand-Anything>
- **Stars (observado 2026-06-27):** ~55-66k (RUN 0 range); Star History reporta 64.9k (global rank #279); Open Agent Skill reporta 59k; trendshift.io confirma crescimento acelerado; primeiro #1 GitHub Trending em 23 mai 2026
- **Licença:** MIT
- **Origem:** Criado por Lum1104 (Yuxiang Lin); migrado para org Egonex-AI [CONFIRMADO via GitHub — Lum1104 existe como conta independente, repo agora em Egonex-AI]
- **Homepage:** <https://understand-anything.com>
- **Maturidade:** Early-production; monorepo pnpm com pacotes @understand-anything/core e @understand-anything/dashboard; CI documentado; testes de integração de skills
- **Estrutura:** Claude Code plugin + multi-platform support via .claude-plugin/, .cursor-plugin/, .copilot-plugin/ plugin manifests

Propósito Understand Anything adota o slogan “Graphs that teach > graphs that impress”. O foco diferenciado em relação ao Graphify é **interatividade educativa e exploração visual** em vez de query-efficiency first. O projeto transforma qualquer codebase em um knowledge graph interativo que pode ser explorado, pesquisado e questionado — com dashboard visual rico, guias de onboarding gerados automaticamente, e análise de impacto de diffs.

Casos de uso principais: - Onboarding acelerado de novos devs em codebases desconhecidas - Exploração visual de arquitetura de sistemas legados - Análise de impacto de mudanças (/understand-diff) - Extração de domínio de negócio (/understand-domain — fluxos, entidades, relações) - Análise de knowledge bases no padrão Karpathy LLM Wiki (/understand-knowledge) - Q&A conversacional sobre o codebase (/understand-chat)

Persona primária: Novo membro de equipe, PM técnico, ou desenvolvedor que precisa explorar e *entender* uma base de código (ensino/orientação), não apenas consultá-la eficientemente.

Capacidades Artefatos produzidos: - .understand-anything/knowledge-graph.json (grafo persistente no repo) - Dashboard web interativo (força-dirigida, cores por camada arquitetural) - Guia de onboarding textual gerado por agente - Análise de domínio (domains, flows, steps estruturados) - Tour guiado do codebase (tour-builder agent)

Pipeline multi-agente (agents especializados): - project-scanner.md — descoberta e classificação de arquivos - file-analyzer.md / batch processing — análise em lote com LLM - graph-reviewer.md — validação do grafo montado - architecture-analyzer.md — análise de padrões arquiteturais - tour-builder.md — geração de tour narrativo - domain-analyzer.md — extração de domínio de negócio - assemble-reviewer.md — revisão final de assembly

Tech stack core (@understand-anything/core): - tree-sitter via plugin registry para extração AST multi-linguagem - GraphBuilder + LLM Analyzer para relacionamentos semânticos - Non-code parsers para Markdown, JSON, YAML - Search, persistence e incremental updates - Zustand store no dashboard (state management React)

Comandos disponíveis: | Comando | Função | |———|———| | /understand | Build completo do knowledge graph | | /understand-dashboard | Abre dashboard visual interativo | | /understand-chat | Q&A conversacional sobre o codebase | | /understand-diff | Impacto de mudanças atuais | | /understand-explain [file/fn] | Deep dive em arquivo/função | | /understand-onboard | Guia de onboarding para novos membros | | /understand-domain | Extração de domínio de negócio | | /understand-knowledge [path] | Análise de knowledge base |

Harness coverage: Claude Code (plugin nativo), Cursor, VS Code + GitHub Copilot, Codex, Open-Code, OpenClaw, Antigravity, Gemini CLI, Pi Agent, Vibe CLI, Hermes, Cline, KIMI CLI, Nanobot, Kiro

Lenses OSS Posture: MIT; código aberto; instala localmente via marketplace ou `install.sh/install.ps1`. Dashboard roda localmente (servidor Node.js). Sem SaaS cloud declarado.

S-SDLC fit: Médio-alto. Útil em onboarding de sprint e análise de impacto de PR (`/understand-diff`), mas o foco é compreensão/exploração, não gates automatizados de CI.

SaaS multi-tenant fit: Baixo. Sem mecanismo de namespace por tenant. O grafo `.understand-anything/knowledge-graph.json` é por-repositório, não por-usuário. Em cenários multi-tenant, cada instância precisaria ser isolada por projeto/repositório.

SDD-methodology fit: Médio. `/understand-knowledge` indica capacidade de lidar com knowledge bases, mas sem integração explícita com specs formais ou ticket systems.

MoE Expert Profile: - **Layer:** L4 (Codebase Intelligence) - **Role:** Visual Explorer + Teaching Graph - **Competence:** Codebase comprehension e onboarding acceleration - **Persona:** Invocado na fase de exploração (novo projeto, onboarding, análise de impacto) - **Abstraction level:** Módulo/domain (mais alto que nó AST; voltado para compreensão humana) - **Authorization:** Local-only; sem RBAC

SECURITY posture: - Local-first: grafo salvo em `.understand-anything/` no repositório — cuidado com commit acidental de dados sensíveis extraídos do código (e.g., segredos hard-coded mapeados no grafo) - Chamadas LLM via credenciais do usuário configurado no harness (Claude Code, Cursor, etc.) — sem relay central - Install scripts `install.sh/install.ps1` executam código diretamente: recomenda-se auditar antes de `curl | bash` - Sem menção explícita de CVE tracking ou dependency audits no README principal - **Risco de large artifacts:** Grafo JSON pode crescer em repos grandes; sem menção de Git-LFS para o arquivo de grafo; pode inflar o repo history se commitado - **Risco de build inicial:** Multi-agent pipeline consome tokens proporcionais ao tamanho do projeto; sem estimativa publicada de custo inicial

Integration Surface

Mecanismo	Suporte	Detalhe
filesystem	Sim (primário)	Lê repo local; salva <code>.understand-anything/knowledge-graph.json</code>
git-repo	Parcial	Grafo salvo no repo; sem git hooks explícitos
git-PR	Sim	<code>/understand-diff</code> analisa impacto de mudanças atuais
git-worktree	Compatível	Sem automação explícita
memory	Parcial	Grafo JSON é memória persistente estrutural
MCP	Não documentado	Sem MCP server no README/DeepWiki
A2A	Não	—
P2P	Não	—
TCP	Sim (dashboard)	Servidor Node.js local para dashboard
webhooks	Não	—
ticket/backlog	Não	—
hooks	Parcial	Auto-update & hooks documentados (DeepWiki §6)
rules	Não	—

EXPERT-L5 — kepano/obsidian-skills

Identidade e Maturidade

- **Repositório:** <https://github.com/kepano/obsidian-skills>
- **Stars (observado 2026-06-27):** 35k [CONFIRMADO via GitHub page: “35k stars”, 2.5k forks, 200 watching]; fontes secundárias reportam 33,438–35.9k
- **Licença:** MIT

- **Criado em:** 2026-01-02
- **Autor:** Steph Ango (handle: kepano) — CEO da Obsidian
- **App baseline:** Obsidian (core proprietário; arquivos locais em Markdown aberto)
- **Maturidade:** Estável; 46 commits; zero releases publicados (skills são conteúdo, não software compilado); 16 issues abertas, 15 PRs
- **Spec:** Segue <https://agentskills.io/specification> — interoperável com qualquer harness compatível

Propósito Obsidian Skills é a coleção oficial de 5 agent skills para o Obsidian PKM, publicada pelo próprio CEO da empresa. O objetivo é ensinar AI agents a *operar corretamente* dentro de vaults Obsidian — usando a sintaxe proprietária (wikilinks, callouts, Bases, Canvas) sem improvisação. Resolve o problema de agentes que sabem escrever Markdown genérico mas falham ao usar recursos específicos do Obsidian.

Casos de uso principais: - Criação e edição automatizada de notas no vault por agentes - Construção de bases de dados estruturadas (CRMs, sprint trackers, research databases) via Bases - Diagramas e mapas visuais via JSON Canvas - Clipping web limpo para vault via Defuddle (economia de tokens) - Automação de gestão de plugins e temas via CLI

Persona primária: Knowledge worker ou pesquisador que usa Obsidian como second brain e quer que AI agents criem, organizem e conectem notas no vault sem requer instruções ad hoc a cada sessão.

Capacidades Skills (5 no total):

Skill	Arquivo de formato	Capacidade ensinada
obsidian-markdown	.md	Obsidian Flavored Markdown: [[wikilinks]], callouts > [!note], frontmatter YAML, embeds, tags
obsidian-bases	.base	Obsidian Bases: views, filters, formulas, summaries — structured data dentro do vault
json-canvas	.canvas	JSON Canvas spec (jsoncanvas.org): nodes, edges, groups, connections
obsidian-cli	—	Obsidian CLI: plugin/theme development, vault management
defuddle	—	Extração de Markdown limpo de web pages via Defuddle CLI (remove ads/nav/clutter, salva tokens)

Artefatos produzidos: - Notas .md com sintaxe Obsidian-correta - Arquivos .base com databases estruturadas - Arquivos .canvas com mapas visuais JSON Canvas - Markdown limpo de páginas web (via defuddle)

Automação: - Não são scripts executáveis — são SKILL.md files (context injection para o agente) - O agente, munido das skills, pode executar ações via Obsidian CLI (requer Obsidian instalado localmente) - Sem automação de build/CI própria

Harness coverage: Claude Code (plugin marketplace + manual /.claude), Codex (~/.codex/skills), OpenCode (~/.opencode/skills/obsidian-skills/), qualquer harness compatível com agentskills.io spec

Lenses OSS Posture: MIT; zero dependências de runtime; zero configuração; zero API keys próprias; zero tiers pagos. “Free forever” implícito. Steph Ango é um defensor explícito de formatos abertos (ver filosofia “Files over Apps” da Obsidian).

S-SDLC fit: Baixo-médio. Não é voltado para desenvolvimento de software — é para gestão de conhecimento pessoal. Pode complementar S-SDLC como camada de documentação/wiki se a equipe usa Obsidian para notas de arquitetura ou ADRs.

SaaS multi-tenant fit: Baixo. Obsidian é fundamentalmente local e single-user (vault por pessoa). Obsidian Publish e Sync são serviços pagos mas sem API programática. Não adequado para ambiente multi-tenant sem adaptação significativa.

SDD-methodology fit: Alto em contexto PKM. Se specs vivem no vault Obsidian, o agente munido de obsidian-skills pode criar, editar e ligar specs, tasks, e notas de implementação com sintaxe correta.

MoE Expert Profile: - **Layer:** L5 (PKM — Personal Knowledge Management) - **Role:** Vault Operator / Knowledge Structurer - **Competence:** Obsidian-native content creation e vault management - **Persona:** Ativado quando o vault Obsidian é o destino de notas, docs, ou knowledge graphs - **Abstraction level:** Nota/link/canvas (human-readable, vault-centric) - **Authorization:** Acesso ao vault local do usuário — escopo de acesso é o filesystem onde o vault reside

SECURITY posture: - Skills são puro texto (SKILL.md) — sem código executável nas skills em si - O agente executa ações via Obsidian CLI: risco de escrita não intencional no vault - Defuddle faz fetch de URLs externas — verificar que o agente não seja induzido a fetch de URLs maliciosas via prompt injection em conteúdo web - Vault local: nenhum dado enviado para Obsidian Inc. pelo uso das skills (Obsidian core pode ter sincronização se habilitada, mas é ortogonal às skills) - **Risco de large artifacts / Git-LFS:** Se o vault contém attachments binários (PDFs, imagens) e está sob git, Git-LFS é recomendado. As skills não abordam este aspecto. - **Token cost:** Skills são context injection leve (SKILL.md files); custo de primeiro run é mínimo — nenhum pipeline de indexação pesado

Integration Surface

Mecanismo	Suporte	Detalhe
filesystem	Sim (primário)	Skills operam sobre vault local via filesystem
memory	Sim (semântico)	O vault Obsidian é a memória de longo prazo — notas, links, bases
git-repo	Compatível	Vault pode estar em git; skills não gerenciam git diretamente
MCP	Não direto	Sem MCP server nas skills; Obsidian tem plugins MCP de terceiros
A2A	Não	—
P2P	Não	—
TCP	Não	—
webhooks	Não	—
git-PR	Não	—
git-worktree	Não	—
ticket/backlog	Não direto	Pode criar task lists em .base files, mas sem integração com Jira/Linear
hooks	Não	—

Mecanismo	Suporte	Detalhe
rules	Sim (skill spec)	Skills seguem agentskills.io spec como regras de comportamento

Análise Cross-Group

1. Graphify vs. Understand-Anything: Query-Efficiency vs. Exploration/Teaching

Os dois experts de L4 têm filosofias fundamentalmente distintas:

Dimensão	graphify	Understand-Anything
Objetivo primário	Eficiência de tokens em query (~71.5x redução)	Compreensão interativa e onboarding
Metáfora	“Graph-first retrieval”	“Graphs that teach”
Output central	Grafo queryable via MCP tools	Dashboard visual interativo
Persona	Agente autônomo que precisa de contexto barato	Desenvolvedor humano explorando codebase nova
Tech diferencial	Leiden clustering sem vector DB; SHA256 cache incremental	Multi-agent pipeline (8+ agents especializados); dashboard React com Zustand
Integração MCP	Sim (nativo, documentado)	Não documentado
Export para L5	Sim — Obsidian Vault + JSON Canvas	Não
Custo de build	Alto (Pass 3 para PDFs/imagens/vídeos via LLM)	Moderado (pipeline multi-agente, sem processamento de vídeo/áudio)
Stars (2026-06-27)	~52-69k (RUN 0: 52,657)	~55-66k (RUN 0 range)
Linguagens	23 (tree-sitter + multimodal)	Multi-linguagem via tree-sitter (sem multimodal declarado)

Conclusão chave: Graphify e Understand-Anything são **complementares, não competidores diretos**. Graphify é a escolha para agentes que precisam de contexto eficiente em tokens durante sessões de desenvolvimento contínuo; Understand-Anything é a escolha para a fase de *onboarding* e *exploração* humana. Em um pipeline MoE maduro, Understand-Anything executa primeiro (gera compreensão), e Graphify serve de context provider durante o trabalho.

Incompatibilidade a notar: Graphify tem MCP server nativo enquanto Understand-Anything não — isso favorece Graphify em pipelines de agente autônomo puro. Understand-Anything tem dashboard visual rico que Graphify não oferece como feature nativa.

2. Como L4 alimenta L5 (PKM) e L8 (graphiti)

L4 → L5 (graphify → obsidian-skills): A conexão mais concreta é o **export Obsidian Vault** do Graphify (seção 3.2 no DeepWiki). O grafo de codebase torna-se um vault navegável, onde cada nó vira uma nota .md e as arestas viram `[[wikilinks]]`. Com obsidian-skills instalado, o agente

pode então navegar, editar e enriquecer esse vault com novas notas — criando um loop bidirecional L4↔L5. Esta é a ponte mais acionável do grupo.

Understand-Anything não tem export direto para Obsidian, mas /understand-knowledge [path/to/wiki] pode consumir knowledge bases no padrão Karpathy-wiki, que frequentemente são vaults Obsidian — fluxo inverso (L5 → L4).

L4 → L8 (graphiti — menção como adjacente): Graphiti (getzep/graphiti) é um knowledge graph *temporal* focado em episódios de conversação e memória de agente. Graphify e Understand-Anything são knowledge graphs *estruturais* de código. A composição natural: graphify extrai a estrutura estática do codebase → graphiti adiciona a camada temporal (o que mudou, quando, por qual agente, em qual conversa). Graphiti bridging L4↔L8 significa que agentes podem perguntar tanto “qual é a arquitetura atual” (L4) quanto “o que foi feito nessa parte do código na última semana” (L8).

3. Overlaps e Colaborações

- **Overlap Graphify-Understand-Anything:** Ambos usam tree-sitter para extração AST, ambos produzem JSON knowledge graphs, ambos suportam Claude Code e Cursor. Sem conflito técnico — podem coexistir no mesmo repo.
- **Colaboração Graphify-obsidian-skills:** Export Obsidian Vault (Graphify) + obsidian-skills (agente sabe escrever/editar vault) = pipeline completo de code→PKM.
- **Sinergia Understand-Anything-obsidian-skills:** /understand-knowledge pode ingerir vault Obsidian como knowledge base; obsidian-skills pode criar notas estruturadas com outputs do understand.

Matriz de Comparação

Expert	Layer	Role	Licens	Stars (2026-06-27)	Scope	Harness	MCP?	Git-flow	S-SDLC	Multi-tenant	Memory	Observability	Security
safisham	4	Graphify Builder + Query Engine	MIT	~52,657 (RUN 0)	Code 23 languages	CSQL, Open-Code, Cursor, Gemini CLI, Aider, Kiro, 10+	Yes (server-side)	Yes; worktree-compatible	Alto	Image-based (local-only)	Parcial (grafo persistente)	Via PR dashboard	Forte: CVE tracking, path sanitization, URL validation; sem relay server

Expert	Layer	Role	Licenses	Stars (2026-06-27)	Scope	Harnesses	MCSP?	Git-flow	S-SDLC	Multi-tenant	Memory	Observability	Security
Egonex-AI/Understand-Anything	x4	Visual-Explorer + Teaching Graph	MIT	~55-66k (RUN 0 range)	Code; multi-language via Copilot, sitter	Claude-Code, Open-Code, Gemini-CLI, Cline, Kiro, 15+	Não	Auto-updates; hooks; worktree-compatible	Médio-alto	Baixo (local-only)	Parcial (.understand-anything/graph.json)	Não	Médio: scripts requerem auditoria; sem CVE tracking público
kepano skills	Obsidian	Operator / Knowledge Structurer	MIT	35k (CONFIRMADO)	Obsidian; Mark-Down; Bases; Can-vas, quer CLI, web extraction	Claude-Code, Open-Code, qual-quer agentskills.io	Não	Vault es-tem git; sem geren-ciamento próprio	Baixo-médio	Baixo (single-user vault)	Sim (vault = memória de longo prazo)	Não	Leve: são texto puro; risco de escrita em vault; de-fud-dle fetch de URLs externas

Fontes

Primárias (repositórios e documentação oficial)

1. **safishamsi/graphify — GitHub (primary):** <https://github.com/safishamsi/graphify>
2. **graphify ARCHITECTURE.md (DeepWiki indexed, commit e441454c):** <https://deepwiki.com/safishamsi/graphify/commit/e441454c>
3. **graphify CHANGELOG v8 branch:** <https://github.com/safishamsi/graphify/blob/v8/CHANGELOG.md>
4. **graphifylabs.ai — homepage oficial:** <https://graphifylabs.ai/>
5. **Egonex-AI/Understand-Anything — GitHub (primary):** <https://github.com/Egonex-AI/Understand-Anything>
6. **Understand-Anything README (DeepWiki indexed, commit ba9ba1f7):** <https://deepwiki.com/egonex-ai/understand-anything>

7. **kepano/obsidian-skills — GitHub (primary, star count confirmado):** <https://github.com/kepano/obsidian-skills>
8. **Agent Skills specification:** <https://agentskills.io/specification>

Secundárias (análises independentes, benchmarks)

9. **“Graphify + Claude Code: How I Cut Token Usage by 71x”** — Manav Ghosh, Medium, abr 2026: <https://medium.com/@manavghosh/graphify-claude-code-how-i-cut-token-usage-by-71x-on-a-50k-line-codebase-74868ac67fd1>
10. **“I Ran This Open-Source AI Tool on a Messy Codebase and Got 71x Fewer Tokens”** — Towards AI: <https://towardsai.com/p/machine-learning/i-ran-this-open-source-ai-tool-on-a-messy-codebase-and-got-71x-fewer-tokens-here-is-exactly-what-happened>
11. **“Graphify: Build a Knowledge Graph Without Sending Your Code to Anyone”** — GoPenAI, abr 2026: <https://blog.gopenai.com/graphify-build-a-knowledge-graph-from-your-entire-codebase-without-sending-your-code-to-anyone-1b6924474b50>
12. **Graphify issue #580 (token efficiency in Claude Code):** <https://github.com/safishamsi/graphify/issues/>
13. **Graphify MCP + worktrees guide:** <https://gist.github.com/ashokvarmamatta/344a642e8b5bd286be605>
14. **Star History — Understand-Anything (64.9k, global rank #279):** <https://www.star-history.com/egonex-ai/understand-anything/>
15. **trendshift.io — Understand-Anything trending data:** <https://trendshift.io/repositories/23482>
16. **“Obsidian Skills: Let Your Agent Manage Your Second Brain”** — DEV Community: <https://dev.to/stevengonsalvez/obsidian-skills-let-your-agent-manage-your-second-brain-4fel>
17. **“Obsidian’s CEO Just Taught AI How to Use His Own App”** — Medium: <https://medium.com/@hamzakhan/obsidians-ceo-just-taught-ai-how-to-use-his-own-app-here-s-the-0-way-to-do-it-too-97acbe8cfefe>
18. **Obsidian Skills Review 2026 — vibecoding.app:** <https://vibecoding.app/blog/obsidian-skills-review>
19. **Claude Skills for Obsidian — claudeskills.info:** <https://claudeskills.info/blog/obsidian-claude-skills-guide/>
20. **Graphify CLI Reference — graphify.net:** <https://graphify.net/graphify-cli-commands.html>
21. **DeepWiki: graphify Security Model:** <https://deepwiki.com/safishamsi/graphify/5-security-model>
22. **DeepWiki: Understand-Anything Getting Started:** <https://deepwiki.com/egonex-ai/understand-anything/1.1-getting-started-and-installation>
23. **DeepWiki: kepano/obsidian-skills reference:** <https://deepwiki.com/kepano/obsidian-skills/7-reference>

Análise produzida em 2026-06-27 | Observações de stars datadas (RUN 0) — verificar no momento de uso
Identidades canonicalizadas em RUN 0; este documento pesquisa CAPABILITIES, PURPOSE, INTEGRA-

Parte II — Fase 1: (c) Pipeline L1/L2/L6/L7

Agentic MoE 2026 — Phase 1 · Group 1c · BUILD PIPELINE

Grupo: 1c — Pipeline (ordenado: **L1 Spec** → **L2 Workflow** → **L6 Design** → **L7 Slides**)
Observado em: 2026-06-27 (valores de estrela = ordem-de-magnitude datada do RUN 0; refinados só com evidência primária) **Nota de idioma:** prosa em pt-BR; identificadores, comandos, nomes de repositório e citações curtas em en-US. **Disciplina:** ≥2 fontes independentes por expert (preferência PRIMARY: README/docs/releases). Citações ≤15 palavras. [UNVERIFIED] / [A CONFIRMAR] marcam o que não foi confirmável por fonte primária. Identidades já canonicalizadas — o foco aqui é **capacidade, propósito, superfície de integração, segurança e relações cruzadas**.

Este é o **maior** grupo da Fase 1 (17 experts em 4 sublayers). O pipeline modela a esteira de construção de software AI-native: **especificar** (L1) → **orquestrar o comportamento do agente enquanto codifica** (L2) → **gerar design/UI** (L6) → **gerar slides** (L7).

L1 — Spec-Driven Development (SDD)

A camada onde a intenção humana vira um artefato executável **antes** do código. Eixo central de tensão do sublayer: **formal/template-rich (Spec Kit)** vs **leve/iterativo (OpenSpec)** vs **anti-context-rot por subagentes (gsd-core)**.

L1.1 · github/spec-kit

Identidade & maturidade. Toolkit de SDD da própria GitHub. Tagline: “Build high-quality software faster.” — substituir “vibe coding” por especificações executáveis: “specifications become executable”. Licença **MIT** (confirmada). Linguagem **Python** (CLI specify, requer Python 3.11+); instalação `uv tool install specify-cli --from git+https://github.com/github/spec-kit.git` (uv recomendado, pipx suportado), auto-gestão via `specify self check|upgrade`. Versão fixada não exposta no README (usa placeholder vX.Y.Z; corroboração externa indica faixa ~v0.10.x [UNVERIFIED]). **106,332** (RUN 0; READMEs do grupo não fixam contagem — dinâmica) · MIT. Alta confiança institucional (GitHub-owned), um dos dev-tools de crescimento mais rápido de 2026.

Propósito (escopo · casos · persona). Greenfield “0-to-1”, exploração criativa paralela e brown-field (“Iterative Enhancement”/modernização), com ambição explícita de constraints corporativos e compliance. Persona: equipes que querem “predictable outcomes instead of vibe coding”.

Capacidades. Comandos namespaced `/speckit.*`: ciclo de vida `/speckit.constitution` → `/speckit.specify` → `/speckit.plan` → `/speckit.tasks` → `/speckit.implement`; mais `/speckit.taskstoissues` (tasks → **GitHub Issues**), e opcionais `/speckit.clarify`, `/speckit.analyze` (consistência cross-artefato), `/speckit.checklist` (“unit tests for English”). Artefatos no filesystem: `.specify/memory/constitution.md`, `.specify/scripts/`, `.specify/templates/`, e `specs/NNN-feature-name/` contendo `spec.md`, `plan.md`, `tasks.md`, `research.md`, `data-model.md`, `quick-start.md`, `contracts/`. Automação alta porém human-gated (refinamento multi-passo, clarificação obrigatória antes do plano, checkpoints). **Cobertura de harnesses: “30+ AI coding agents”** — README nomeia Claude Code, Gemini CLI, Cursor CLI, Qwen CLI, opencode, Codex CLI, Qoder, Tabnine, Kiro, Pi, Forge, Goose, Mistral Vibe; Copilot é o default. `specify integration list` enumera.

Lentes. - *OSS posture*: MIT permissivo, mantido pela GitHub; extensões comunitárias carregam disclaimer (“review source before installation”). - *S-SDLC fit*: **Forte** — constitution.md como camada de governança/princípios persistentes; /speckit.checklist e /speckit.analyze agregam gates de qualidade; integração nativa com GitHub Issues/PR. - *SaaS multi-tenant fit*: Indireto — produz o app, não isola tenants; ambição de compliance corporativa é de *processo*, não runtime. - *SDD-methodology fit*: **Canônico/maximalista** — é o template-rich de referência do sublayer (constitution + spec + plan + tasks + implement + checklists). - *MoE expert-profile*: camada **L1 Spec**; papel **spec compiler / lifecycle governor**; competência alta-formal; persona “arquiteto de especificação”; abstração média-alta (texto→implementação); autorização: escreve filesystem + cria branch + Issues + guia PR.

Segurança. MIT, GitHub-trust. Risco da camada de instrução: durante /speckit.implement o agente executa CLI local (dotnet, npm) — risco de execução agêntica padrão, divulgado. Supply-chain: instala via git ref (uv/pipx); extensões comunitárias com disclaimer explícito. Sem incidentes conhecidos.

Superfície de integração. {filesystem, git-branch, git-repo, ticket/backlog (GitHub Issues), git-PR, rules, hooks}. Branch-por-feature (001-...) via create-new-feature.sh; rules = constitution.md + arquivos de instrução do agente (CLAUDE.md); sistema de extensões pode adicionar “commands, hooks, and capabilities”. **git-worktree** apenas via extensões comunitárias (não no core). **MCP não mencionado no README [UNVERIFIED]**.

L1.2 · Fission-AI/OpenSpec

Identidade & maturidade. SDD “for AI coding assistants”; header colapsável: “The most loved spec framework.” Filosofia: “fluid not rigid / iterative not waterfall / easy not complex / built for brownfield.” Licença **MIT** (confirmada). Linguagem **TypeScript/Node** (pacote npm @fission-ai/openspec, requer Node ≥ 20.19.0); instalação npm install -g @fission-ai/openspec@latest + openspec init; refresh via openspec update. **51,175** (RUN 0) · MIT. Recomenda modelos de alto raciocínio (“Codex 5.5 and Opus 4.7”).

Propósito. “AI coding assistants are powerful but unpredictable when requirements live only in chat history.” Adiciona uma camada de spec leve para “agree on what to build before any code is written.” Posiciona-se deliberadamente **mais leve que o Spec Kit**, brownfield-first, escalando de projeto pessoal a empresa.

Capacidades. Workflow /opsx:* — perfil core: /opsx:propose, /opsx:explore, /opsx:apply, /opsx:archive (+ /opsx:sync); perfil expandido (opt-in): /opsx:new|continue|ff|verify|bulk-archive|onboard. Cada mudança vira uma pasta openspec/changes/<change-name>/ com proposal.md, specs/, design.md, tasks.md; no archive → move para openspec/changes/archive/YYYY-MM-DD-<name>/ e atualiza specs. Sem gates rígidos de fase (“update any artifact anytime”); ênfase em “context hygiene” antes de implementar. **Cobertura: “25+ tools”** — Claude Code, Cursor, Copilot, Windsurf, Continue, Gemini CLI, Amazon Q, Cline, RooCode, Kilo, Codex, OpenCode, Qoder, Qwen, Kiro, Pi, etc. (alguns só skill, sem command adapter).

Lentes. - *OSS posture*: MIT, org nomeada (Fission-AI), MAINTAINERS.md público, CI ativo. - *S-SDLC fit*: **Médio-leve** — spec-as-durable-memory vs chat efêmero; sem gates de fase nem PR/branch nativos (é git-agnóstico, organiza pastas). - *SaaS multi-tenant fit*: Indireto (produz o app). - *SDD-methodology fit*: **Leve/iterativo** — o contraponto explícito do Spec Kit; proposal/spec/design/tasks por change-folder. - *MoE expert-profile*: L1 Spec; papel **lightweight change-proposer**; competência médio-formal; persona “facilitador de spec brownfield”; abstração média; autorização: escreve filesystem + regenera guidance do agente.

Segurança. MIT. **Telemetria anônima** (apenas nomes de comando + versão; “No arguments, paths, content, or PII”; auto-desabilita em CI; opt-out OPENSPEC_TELEMETRY=0/DO_NOT_TRACK=1) — divulgada e mínima, mas relevante para ambientes sensíveis. Risco de camada de instrução: openspec update escreve/regenera arquivos de instrução em muitos dirs de config (.claude/, .github/...) —

superfície de escrita ampla, porém transparente e iniciada pelo usuário. npm-distribuído, sem incidentes.

Superfície de integração. {filesystem, git-repo, rules, MCP, memory(leve)}. Filesystem é o mecanismo primário (change-folders + skill/command files por ferramenta); rules = guidance regenerada; **MCP referenciado** (“dedicated notes for Cursor and MCP”) mas profundidade [UN-VERIFIED]; memória = camada de spec como memória durável (file-backed, não protocolo). **Sem** git-branch/PR/worktree/ticket nativos [provavelmente ausentes].

L1.3 · open-gsd/gsd-core

Identidade & maturidade. Tagline: “Git. Ship. Done.” — “a light-weight meta-prompting, context engineering, and spec-driven development system” para Claude Code, OpenCode, Gemini CLI, Kilo, Codex, Copilot, Cursor, Windsurf. Rodapé: “Claude Code is powerful. GSD Core makes it reliable.” Licença **MIT** (confirmada). Linguagem **Node/JS** (npm @opengsd/gsd-core); instalação npx @opengsd/gsd-core@latest (escolhe runtime + global/local) → /gsd-new-project. **Branch default next.** ~4.2K [sec] (RUN 0; o sibling get-shit-done-redux ~583; o original tinha 65k+) · MIT. **É o sucessor SEGURO do GSD** — ver postura de segurança.

Propósito. Combate o **context rot** — “the quality degradation that accumulates as an AI fills its context window” — rodando pesquisa/planejamento/execução pesados em **subagentes de contexto fresco**, mantendo a sessão principal enxuta. Três modos de falha visados: context bloat, ausência de memória compartilhada entre sessões, e nada verificando que o código funciona.

Capacidades. Loop de 5 fases (uma por vez, por milestone): **Discuss** (→ CONTEXT.md) → **Plan** (pesquisa/decompõe/verifica caber em contexto fresco) → **Execute** (waves paralelas; cada executor recebe 200k tokens limpos) → **Verify** (percorre o construído, diagnostica + corrige antes de “done”) → **Ship** (cria o PR, arquiva a fase). Comando de entrada /gsd-new-project; demais nomes em docs/COMMANDS.md [UNVERIFIED além do entry]. Artefatos: STATE.md (“the spine of the system”: milestone/fase/decisões/blockers/progresso) + CONTEXT.md; “structured artifacts survive session boundaries.” Orquestração a mais sofisticada do sublayer (thin orchestrator + subagentes frescos + verify-before-done). Cobertura provider-flexível (público + local), contraste deliberado com o gsd-plugin Claude-only.

Lentes. - *OSS posture*: MIT, fork comunitário (open-gsd) com migração bit-for-bit e atribuição preservada. - *S-SDLC fit*: **Forte na orquestração** — verify-before-ship + PR auto-gerado + worktree isolated; memória cross-sessão. - *SaaS multi-tenant fit*: Indireto. - *SDD-methodology fit*: **SDD + context-engineering** — leve no core, pesado na orquestração por subagentes. - *MoE expert-profile*: L1 Spec; papel **context-rot orchestrator**; competência alta (orquestração); persona “shipper disciplinado anti-bloat”; abstração média-alta; autorização: filesystem + worktree + PR + subagent dispatch (A2A-like).

Segurança — A HISTÓRIA DE SUPPLY-CHAIN (headline do sublayer). O open-gsd/gsd-core atual é o **fork SEGURO**; o **original gsd-build/get-shit-done por “TÂCHES” está EXCLUÍDO/comprometido**. Confirmado por múltiplas fontes independentes (anúncio fixado do mantenedor do fork + firma de segurança + imprensa cripto): - **Rug-pull confirmado**: o criador lançou um token Solana \$GSD na esteira do hype (venceu o Bags Hackathon), drenou liquidez, vendeu tokens e **deletou contas X + GitHub**, sumindo com ~US500K (2026-04). *Mantenedor*: **"the`GSDtoken... has been publicly linked to a rug-pull."** - ***Fork (get-shit-done-redux→open-gsd)*** em 2026-05-22: espelhou 394 branches/229 tags/77 issues/17 PRs, preservou código + MIT + atribuição, removeu badges de token e links sociais mortos. Versionamento resetado para v1.0.0 (quebra limpa do 1.42.x). - ***Ameaça NPM ativa e contínua (a parte mais séria):*** **"The scammer still holds the administrative keys to the original NPM package registry."** Como o GSD requer permissões profundas deshell/bash, um update malicioso aos pacotes ORIGINAIS poderia comprometer máquinas locais (chaves privadas, env vars). ***Pacotes npm originais permanecem vivos.*** - ***Remediação:*** desinstalar originais (npm uninstall -g get-shit-done-cc,@gsd-build/sdk),

limpar ~/npm/_npx/, apagar pastas gsd não-Markdown em claude; usar apenas @opengsd/gsd-core; pinar versões; sandboxar o agente. - **O código do fork em si:** auditoria independente — “No backdoors or active malicious payloads”; o risco está inteiramente nos pacotes ORIGINAIS. (Identidade legal do fundador reportada na imprensa cripto mas [UNVERIFIED].)

Superfície de integração. {filesystem, git-branch, git-PR, git-worktree, memory, rules, subagent-orchestration (A2A-like)}. Memória = STATE.md/CONTEXT.md cross-sessão (file-backed); Ship cria PR com body gerado; worktrees isolados (re-tentativa em worktree estreito ao rejeitar). **Sem** MCP/P2P/ticket/webhooks/TCP [UNVERIFIED]. Mecanismo definidor: orchestrator → subagentes de contexto fresco reportando de volta.

LENS — SpecDD: “SpecDD” é um **rótulo de metodologia SDD**, não um Target separado. É coberto integralmente por spec-kit + OpenSpec neste grupo; não recebe profile card próprio.

L2 — Workflow & Skill Frameworks

A camada que governa “**como o agente deve se comportar enquanto codifica**” — injetando instruções always-on (CLAUDE.md), hooks, skills, regras e “instincts”. É aqui que mora a **colisão de camada de instrução** mais aguda do grupo: múltiplos gerenciadores always-on não coexistem bem (ver síntese ao final).

L2.1 · obra/superpowers

Identidade & maturidade. “a complete software development methodology for your coding agents, built on top of a set of composable skills.” Licença **MIT** (confirmada). Autor: **Jesse Vincent / Prime Radiant** (release ~out/2025; ~v5.1.0 em mai/2026 [UNVERIFIED-sec]). Distribuído como **plugin Claude Code** (marketplace oficial superpowers@claude-plugins-official ou obra/superpowers-marketplace); **não edita CLAUDE.md manualmente** — entrega skills (skills/*/SKILL.md) + instruções iniciais que garantem o uso. ~147K [sec] (RUN 0; nenhum README do trio fixa estrelas — [UNVERIFIED]) · MIT.

Propósito. Impede o agente de “just jumping into trying to write code”; impõe SDLC spec-first/plan-first/TDD para “work autonomously for a couple hours... without deviating from the plan.” Persona: qualquer usuário de coding-agent querendo metodologia disciplinada (sem “teatro” de personas).

Capacidades. Workflow de 7 fases, cada uma uma skill auto-disparada: **brainstorming** (refinamento socrático de spec) → **using-git-worktrees** → **writing-plans** (tarefas de 2–5 min com paths exatos) → **subagent-driven-development / executing-plans** (subagente fresco por tarefa, revisão em dois estágios) → **test-driven-development** (“Deletes code written before tests”) → **requesting-code-review** (“Critical issues block progress”) → **finishing-a-development-branch**. Biblioteca ~14 skills (systematic-debugging, verification-before-completion, dispatching-parallel-agents, writing-skills...). Cobertura: **6 harnesses** — Claude Code, Codex (CLI+App), Cursor, OpenCode, Copilot CLI, Gemini CLI.

Lentes. - *OSS posture:* MIT limpo. - *S-SDLC fit:* **Muito forte** — TDD obrigatório, code-review com severidade, worktree-first, verification-before-completion; é literalmente uma metodologia SDLC. - *SaaS multi-tenant fit:* N/A (governa comportamento, não produz tenant). - *SDD-methodology fit:* Forte (brainstorm→plan→execute mapeia spec→plan→implement). - *MoE expert-profile:* L2 Workflow; papel **SDLC methodologist (skills)**; competência alta; persona “engenheiro disciplinado TDD”; abstração média; autorização: filesystem + worktree + branch + PR via skills.

Segurança. MIT. Risco de instrução **baixo-a-moderado** — injeta camada always-on “use your skills” + workflows obrigatórios (“Mandatory workflows, not suggestions”), mas **sem hooks executando shell por turno, sem daemon de memória, sem MCP bundled**. Menor footprint = menor superfície de colisão do trio. **Como injeta comportamento:** via **skill-trigger instructions**

(preâmbulo “check for relevant skills before any task” + frontmatter auto-ativável) — por **ativação de skill, não por edição de CLAUDE.md nem hooks por turno.**

Superfície de integração. {filesystem, git-repo, git-worktree, git-branch, git-PR, rules(skill-preamble), skills/SKILL.md}. **Sem** MCP/hooks/memory-store/browser/ticket/A2A/P2P/TCP/web O mais enxuto dos três.

L2.2 · garrytan/gstack

Identidade & maturidade. “turns Claude Code into a virtual engineering team” — “23 opinionated tools that serve as CEO, Designer, Eng Manager, Release Manager, Doc Engineer, and QA.” Licença **MIT** (“Free forever”). Autor: **Garry Tan** (President & CEO, Y Combinator). Versionamento inconsistente no doc (refs a v0.19 e a v1.43.0.0+). **Instalação invasiva:** clona em ~/.claude/skills/gstack, roda ./setup, e **escreve uma seção ## gstack + ## Skill routing no seu CLAUDE.md**; “team mode” (./setup --team) commita .claude/ + CLAUDE.md no repo (auto-update silencioso horário). **101,289** (RUN 0) · MIT.

Propósito. Permitir que um builder solo “ship like a team of twenty” via sprint estruturado Think→Plan→Build→Review→Test→Ship→Reflect com especialistas por papel. Persona: founders/CEOs, iniciantes em Claude Code, tech leads — o “exato setup do Garry Tan”.

Capacidades (23 especialistas + power tools). Plan: /office-hours (YC Office Hours), /plan-ceo-review, /plan-eng-review, /plan-design-review (“AI Slop detection”), /autoplan, /spec. Build/review: /review (auto-fix bugs óbvios), /investigate (“no fixes without investigation”; para após 3 falhas), /codex (segunda opinião OpenAI). Design: /design-shotgun (4–6 mockups GPT-Image + taste memory), /design-html. QA/browser: /qa (“opens a real browser”, gera regressões), /browse (Chromium real), /pair-agent (agentes cross-vendor compartilham um browser via scoped tokens + ngrok). Ship/ops: /ship (PR + coverage audit), /land-and-deploy, /canary, /cso (OWASP+STRIDE). Safety: /careful, /freeze, /guard. Memória: /learn (cross-session), GBrain. iOS: /ios-qa (iPhone real via USB). Cobertura: **10 agentes** (Claude Code primário, Codex, OpenCode, Cursor, Factory Droid, Slate, Kiro, Hermes, GBrain, OpenClaw via ACP).

Lentes. - OSS posture: MIT. - S-SDLC fit: **Forte e amplo** — /cso (OWASP/STRIDE), /ship (coverage gate), QA com browser real, scanner de segredos próprio. - SaaS multi-tenant fit: N/A diretamente (mas GBrain tem tiers de trust read-write/read-only/deny por repo). - SDD-methodology fit: Médio (tem /spec//autoplan, mas é sprint-de-personas, não SDD formal). - MoE expert-profile: L2 Workflow; papel **virtual eng-team (personas)**; competência alta-ampla; persona “time de eng em uma pessoa”; abstração média; autorização: filesystem + worktree + PR + browser + memória + MCP(GBrain) + A2A(/pair-agent).

Segurança. MIT. **Ships sua própria stack de segurança** para browser/prompt-injection (classificador ML 22MB + Haiku transcript check + canary token + combiner 2-de-3; kill switch GSTACK_SECURITY_OFF=1) + scanner de segredos defense-in-depth. Risco de instrução **moderado-a-alto de footprint:** reescreve seu CLAUDE.md, commita gstack em repos compartilhados (team mode), roda **auto-update silencioso horário** (consideração de supply-chain) e persiste estado via /learn. **Como injeta comportamento:** via **seções explícitas no CLAUDE.md** (## gstack com “Never use mcp_claude-in-chrome_* tools” + ## Skill routing) + lógica de roteamento de skills — **editando CLAUDE.md diretamente, não via hooks por turno.** As diretivas “never use tool X” são o elemento de maior colisão quando empilhado.

Superfície de integração. {filesystem, git-repo, git-worktree, git-branch, git-PR, rules(CLAUDE.md), memory(GBrain), MCP(registra GBrain), browser(Chromium/CDP), A2A(/pair-agent), TCP/ngrok+Tailscale}. Ticket: /spec deduplica issues, /ship fecha issue de origem. **Não** instala hooks de ciclo de vida estilo ECC — comportamento via CLAUDE.md + skill-routing.

L2.3 · affaan-m/ECC

Identidade & maturidade. *“The harness-native operator system for agentic work. From an Anthropic hackathon winner.” / “The agent harness performance optimization system. Skills, instincts, memory, security, and research-first development.”* Licença **MIT** (*“This repo is MIT-licensed forever”*; camada comercial ECC Pro \$19/seat/mo + GitHub App). Autor: affaan-m. Versão **v2.0.0-rc.1** (Alpha abr/2026; protótipo Rust em ecc2/). Homepage **ecc.tools**. **CROSS-LAYER**: também traz **NanoClaw** (orquestração) + **AgentShield** (segurança). **187,680** [sec] (RUN 0; README hardcode “182K+” como prosa de marketing, busca live dá 222K+ — [UNVERIFIED]) · MIT.

Instalação (maior número de superfícies do grupo). **CLAUDE.md** + **AGENTS.md** (universal cross-tool); skills ~/.claude/skills/ecc/; **hooks** ~/.claude/hooks/hooks.json; rules ~/.claude/rules/ecc/; ~/.claude/agents/ + ~/.claude/commands/; e configs por harness (.codex/config.toml, opencode.json, .cursor/, .github/copilot-instructions.md, .gemini/GEMINI.md, .zed/settings.json). Via plugin (/plugin install ecc@ecc) ou npx ecc-install.

Propósito. Otimizar performance/confiabilidade do harness (custo de token, memória cross-sessão, verificação, paralelização, segurança) + camada de operador reutilizável. Seis pilares: **skills · instincts · memory optimization · continuous learning · security scanning · research-first development**. Persona: power users rodando 1+ harnesses para produtos reais.

Capacidades. **232 skills** (Claude Code) — search-first, security-scan (AgentShield), continuous-learning-v2, tdd-workflow, eval-harness, verification-loop. **Instincts** (padrões aprendidos com confidence scoring, auto-extraídos; /instinct-status|import|export, /evolve, /prune 30d TTL, /promote). **Memória:** hooks de persistência (session-start.js/session-end.js/pre-compact.js) + **SQLite state store** + **memory MCP server**. **NanoClaw v2** — “model routing, skill hot-load, session branch/search/export/compact/metrics” (por trás de /model-route, /loop-start, /harness-audit). **AgentShield** (repo separado + ecc-agentshield): “1282 tests, 98% coverage, 102 static analysis rules” — escaneia CLAUDE.md/settings/MCP/hooks/agents/skills em 5 categorias (secrets[14 patterns], permission audit, **hook injection analysis**, **MCP server risk profiling**, agent config review); modo --opus roda red-team/blue-team/auditor; grade A-F, exit 2 gateia builds. **60 subagentes** + /multi-* + PM2 + tmux. Cobertura **mais ampla: 7+** (Claude Code, Codex, Cursor, OpenCode, Gemini-exp, Zed, Copilot, + Antigravity/Qwen/Grok-manual).

Lentes. - *OSS posture*: MIT; GitHub App comercial pede acesso a repo. - *S-SDLC fit*: **Muito forte** — AgentShield é um scanner de postura de segurança AI dedicado (102 rules), ironicamente auditando a mesma superfície que o ECC cria. - *SaaS multi-tenant fit*: Parcial (Pro \$19/seat; GitHub App por repo). - *SDD-methodology fit*: Médio (research-first + tdd-workflow skills, mas é operator-layer, não SDD). - *MoE expert-profile*: L2 Workflow (cross-layer p/ segurança+orquestração); papel **harness operator system**; competência alta/maximalista; persona “operador de harness research-first”; abstração média-baixa (runtime/hooks); autorização: **a mais ampla** — hooks por turno + MCP(~14) + filesystem + worktree + PR + ticket(parcial) + memória.

Segurança. MIT. AgentShield é a história (contagens acima). Risco de instrução **o MAIS ALTO do trio** — injeta comportamento por turno via **múltiplos canais always-on simultâneos**: (1) rules always-on (rules/ecc/, alwaysApply:true, copilot-instructions “always injected”); (2) preâmbulo CLAUDE.md + AGENTS.md por sessão; (3) **SessionStart hook injeta até 8000 chars** (ECC_SESSION_START_MAX_CHARS); (4) instincts; (5) **hooks Pre/PostToolUse/Stop rodando shell em cada tool call**. **Auto-reconhece a colisão**: o README alerta “Duplicate hooks file detected” e “duplicate execution and cross-platform hook conflicts” + erosão de contexto (200k→~70k por descrições MCP). **Como injeta comportamento**: o mais agressivo — CLAUDE.md+AGENTS.md + rules/ always-on + SessionStart hook + Pre/PostToolUse/Stop hooks + instincts. **Único do trio que instala hooks.json e escreve AGENTS.md além de CLAUDE.md**.

Superfície de integração. {MCP(~14: GitHub, Context7, Exa, Memory, Playwright, Supabase, Vercel...), filesystem, git-repo, git-worktree, git-branch, git-PR, ticket/backlog(parcial: work-items sync-github, Linear), memory(hooks+SQLite+MCP), rules(always-on), hooks(Pre/PostToolUse/Stop/SessionStart/End/PreCompact), browser(Playwright
Não nomeado: A2A, P2P, TCP, webhooks.

L2.4 · HKUDS/CLI-Anything

Identidade & maturidade. “*Making ALL Software Agent-Native*” / “*Today’s Software Serves Humans. Tomorrow’s Users will be Agents.*” Licença **Apache-2.0 — CONFIRMADA** (arquivo LICENSE completo lido; Copyright [2026] [HKUDS CLI-Anything Team]) → **a licença era load-bearing e está confirmada; redistribuição comercial OK.** Autor: HKU Data Science Lab (HKUDS). Ships pacote PyPI cli-anything-hub + plugin Claude Code. ~**42-44K** [sec] (RUN 0; fonte datada mais dura = 40K em 2026-05-24, faixa 42-44K plausível hoje — [UNVERIFIED exato]) · Apache-2.0.

Propósito. A maioria do software desktop instalado não expõe interface estruturada que agentes possam dirigir (GIMP sem REST, Blender não fala MCP). CLI-Anything gera **CLIs agent-native** a partir do código-fonte. **Requer acesso ao código-fonte** — degrada em software só-binário (admitido nos docs). Persona: devs construindo agentes que dirigem ferramentas desktop reais; equipes auditando quanto da stack é dirigível por agentes.

Capacidades — pipeline de 7 fases. **Analyze** (mapeia GUI→APIs) → **Design** (grupos de comando, estado, formatos) → **Implement** (**CLI Click** com REPL, JSON output, undo/redo) → **Plan Tests** (TEST.md) → **Write Tests** → **Document** → **Publish** (gera setup.py, instala no PATH); meio-passo gera SKILL.md para registries (SkillHub/ClawHub). Ships **40+ CLIs pré-construídas + 2,330 testes** (1,732 unit/579 e2e). Refinamento iterativo /cli-anything:refine. Entrada via plugin (/cli-anything <path>).

Lentes. - *OSS posture*: **Apache-2.0 — a melhor postura de reuso do grupo** (grant de patente explícito). - *S-SDLC fit*: Médio — gera testes (TEST.md) por harness, mas testa comportamento da CLI, não correção do output da aplicação. - *SaaS multi-tenant fit*: N/A. - *SDD-methodology fit*: Baixo (é tooling-generation, não SDD). - *MoE expert-profile*: L2 Workflow (tool-builder); papel **software→agent-native CLI generator**; competência alta-especializada; persona “engenheiro de tooling agent-native”; abstração baixa-média (gera código); autorização: lê código-fonte + escreve setup.py/CLI + instala no PATH.

Segurança. **Apache-2.0** (a mais forte). Risco: emite harnesses executáveis (setup.py no PATH) + SKILL.md agent-discoverable a partir de código arbitrário — **código de install/exec gerado de repos não confiáveis é a superfície de supply-chain a auditar.**

Superfície de integração. {filesystem, git-repo(lê source), skill-registry(SKILL.md), CLI/REPL/JSON} + **CLI-Hub** (cli-anything-hub; cli-hub list|search|install|launch). **NÃO usa/wrappa MCP** (mas o JSON-output mode é desenhado para wrap MCP downstream). Sem A2A/P2P/PR/worktree/ticket/webhooks/TCP.

L2.5 · ChristopherKahler suite (paul · carl · seed · base)

Quatro ferramentas independentemente instaláveis (sem dependências duras) que compõem **um único lifecycle: SEED (idear) → BASE (lembrar) → CARL (comportar) → PAUL (construir).** Autor: Chris Kahler (Chris AI Systems). Todas verificadas via GitHub API (estrelas/licenças/datas autoritativas).

PAUL — Plan-Apply-Unify Loop. “*Structured AI-assisted development for Claude Code.*” Lema: “*In-session context over subagent sprawl.*” **MIT** (API). **976** (API exato, RUN 0), criado 2026-01-28, npm paul-framework, 26 slash-commands. Loop: **PLAN** (Objective + Acceptance Criteria Given/When/Then + Tasks + Boundaries DO-NOT-CHANGE) → **APPLY** (Execute/Qualify loop; status DONE/DONE_WITH_CONCERNS/NEEDS_CONTEXT/BLOCKED; anti-rationalization) → **UNIFY** (fechamento obrigatório: SUMMARY.md, plan-vs-actual, atualiza STATE.md; “*Never skip UNIFY*”). Artefatos em .paul/ (PROJECT.md, ROADMAP.md, STATE.md, config.md). Subagentes só para discovery/research. Integração: {filesystem(.paul/), rules(via CARL), memory(STATE.md), git-repo}; MCP via CARL. **vs GSD (citado verbatim):** “*PAUL takes a different approach from GSD.*” — tabela: GSD = *Parallel subagents / Optional closure / Static prompts*; PAUL = *In-session*

context / Mandatory UNIFY / CARL dynamic loading; “different philosophy. PAUL prioritizes quality over speed-for-speed’s-sake.” Doc completo PAUL-VS-GSD.md. MoE: L2; papel **in-session Plan-Apply-Unify orchestrator**; autorização filesystem+rules+memory.

CARL — Context Augmentation & Reinforcement Layer. “Dynamic rules for Claude Code... Rules that load when relevant, disappear when not.” MIT (API). **353** (API exato), criado 2026-01-28, npm carl-core, **v2** (carl.json single-file). **Injeção JIT de regras:** hook escaneia cada prompt → casa recall keywords → carrega só os domain rules que casam (Domains, Star-Commands *command, Context Brackets FRESH/MODERATE/DEPLETED/CRITICAL). Roda em **toda interação** via ~/.claude/hooks/carl-hook.py. Integração: {hooks, MCP(carl-mcp server, 15 tools carl_v2_*), rules, memory, filesystem}. **É o único dos 4 Kahler com MCP server + hook engine reais** — o substrato de runtime dos outros. Risco: hook que **injeta regras silenciosamente em cada prompt** por keyword é mecanismo de instrução poderoso (mitigado por transparência “Explicit Over Magic”). MoE: L2; papel **JIT rule injector / behavioral substrate**.

SEED — typed project incubator. “Your AI project incubator... produce structured plans ready for building.” Licença: license: null na API (claim MIT só na prosa do README; arquivo LICENSE ausente/vazio) → **risco real de higiene legal: scanners e revisão conservadora tratam como all-rights-reserved.** A premissa do brief (“NO license”) está **substancialmente correta no nível de máquina.** **288** (API exato), criado 2026-03-18, npm @chrisai/seed, **markdown puro, zero deps** (sem hooks/MCP). Comandos: /seed, /seed graduate (→ apps/{name}/ com git+README), /seed launch (graduate + PAUL headless), /seed status, /seed add-type. 5 project types com rigor adaptativo. Integração: {filesystem(markdown), git-repo(graduate=git init), memory/session}; integra BASE v2 + PAUL v1.4. MoE: L2; papel **typed ideation→PLANNING.md incubator; risco de licença = o mais alto do sub-sub-grupo.**

BASE — Builder’s Automated State Engine. “Turn Claude Code from a per-session tool into a workspace that remembers.” Camada de **knowledge-graph + proactive context injection** (README diz Rust; metadado de linguagem do repo = JavaScript — core Rust [PARTIALLY UNVERIFIED]). **Licença null na API** (mesma lacuna do seed — risco de reuso). **87** (API exato), criado 2026-03-17. SEED registra projetos no grafo do BASE na ideação/graduação (base project add). MoE: L2; papel **memory substrate (knowledge graph).**

Encadeamento (fluxo concreto): SEED ideação → PLANNING.md → /seed launch gradua p/ repo git + registra no grafo BASE + inicializa PAUL headless → durante o build PAUL, o hook do CARL injeta as 14 regras de loop-enforcement JIT e as mantém fora do contexto fora de .paul/. “SEED + PAUL” é o par marketado; BASE = substrato de memória; CARL = substrato comportamental. (Siblings AEGIS/Skillsmith fora do pipeline de 4.)

L6 — Generative Design

A camada que gera UI/protótipos/design a partir de prompts. Eixo: **studio completo (open-design, open-codesign)** vs **layer de qualidade (impeccable)**; e **local-first/BYOK** como postura comum (alternativas ao Claude Design proprietário).

BASLINE (não-OSS): Anthropic **claude-design / frontend-design** (proprietário) é o produto de referência que todo este sublayer espelha. Notavelmente, **impeccable, open-design e open-codesign descendem da lineage frontend-design** e usam DESIGN.md como arquivo de contexto de marca.

L6.1 • nexu-io/open-design

Identidade & maturidade. “The open-source alternative to Claude Design. Local-first, web-deployable, BYOK at every layer.” Licença **Apache-2.0** (confirmada; guizang-ppt/+html-ppt/ retêm MIT original). Org nexu-io. Auto-descrição: “an early implementation”; releases até **v0.11.0**

“The Bazaar”. Sem instalador brew/npm — só clone + pnpm (pnpm tools-dev run web; Node ~24). **48,662** (RUN 0; só badge dinâmico — [UNVERIFIED]) · Apache-2.0.

[!] DRIFT DE VERSÃO (refinamento crítico do RUN 0): o brief especifica **19 skills / 71 design systems**, mas isso corresponde a um **fork congelado** (RavenTress/nexu-open-design), **não** ao upstream live. O README nexu-io/open-design **buscado hoje (2026-06-27)** diz **31 skills / 72 (ou 129, internamente contraditório) design systems**; a social-preview do GitHub anuncia **259+ skills / 142+ design systems**. → **Figuras 19/71 do brief estão [SUPERSEDED] no upstream.** O repo itera muito rápido.

Propósito. Studio local-first/web-deployable que transforma 12+ CLIs de coding-agent num “design engine”. Persona: designers/devs querendo output estilo Claude Design sem lock-in Anthropic, na própria máquina, com qualquer modelo. *“trained, by the prompt stack, to behave like a senior designer.”*

Capacidades. Produz: protótipos web (landings/dashboards/pricing/docs), **protótipos mobile** (frames pixel-accurate iPhone/Pixel/iPad), **decks/slides** (deck mode), **imagens** (gpt-image-2), **vídeos** (seedance-2.0 + HyperFrames HTML→MP4), áudio, e office docs. **Exports: HTML · PDF · PPTX · ZIP · Markdown + PNG + MP4** (HTML/PDF/PPTX/MP4 do brief confirmados). Skills/design-systems: ver drift acima (cada system = 1 DESIGN.md com schema 9-seções). **od CLI: o bin od existe** (package.json single bin), mas o README **não documenta subcomandos od funcionais** — entrada real é pnpm tools-dev; od init/od skills install são roadmap-only. (Um snapshot de busca menciona od mcp install <agent> — **NÃO está no README buscado, [UNVERIFIED/mais novo].**) Loop agêntico completo (Read/Write/Bash/WebFetch), self-critique 5-dimensões pré-emit. **Cobertura: Claude Code** (argv claude -p ... --permission-mode bypassPermissions) + 11 CLIs auto-detectadas (Codex, Devin, Gemini, OpenCode, Cursor, Qwen, Copilot, Hermes, Kimi, Kiro, Pi). **BYOK multi-model** via proxy OpenAI-compatible.

Lentes. - *OSS posture:* Apache-2.0 (enterprise-friendly). - *S-SDLC fit:* Baixo (é design-gen, não SDLC); **mas [!] postura de segurança fraca** — ver abaixo. - *SaaS multi-tenant fit:* Web-deployable (Vercel) mas local-first por design; SQLite por máquina. - *SDD-methodology fit:* N/A. - *MoE expert-profile:* L6 Design; papel **local-first design studio (multi-surface)**; competência alta-ampla; persona “senior designer com filesystem”; abstração média; autorização: **bypass total de permissões** + Bash/Write/WebFetch em filesystem real.

Segurança. Apache-2.0; fortemente local-first (SQLite local + .od/ gitignored). BYOK em localStorage + daemon state/ (sem vault encriptado claimado; SSRF guard na borda). **[!] RISCO-CHAVE:** cada adapter de agente é spawnado com **permissões totalmente bypassadas** — bypassPermissions (Claude), --permission-mode dangerous (Devin), --yolo (Gemini/Qwen), --dangerously-skip-permissions (OpenCode), --allow-all-tools (Copilot) — com Bash/Write/WebFetch irrestritos. **Sem mitigação de injeção de instrução documentada; builds não assinados.** Default materialmente mais arriscado que o loop permissionado do open-codesign.

Superfície de integração. {filesystem(cwd real, SQLite .od/app.sqlite), git(só clone + flags git-check por CLI), memory(SQLite), rules(prompt-stack anti-slop – regras de design, não access-control), hooks(só flag Hermes --accept-hooks), ACP(Devin/Hermes/Kimi/Kiro)}. **NÃO presente:** git-worktree/branch/PR como features, web-hooks, A2A, P2P, TCP. **MCP: o README buscado NUNCA menciona MCP** (string ausente em 742 linhas) — integração = stdio CLI-spawn + ACP + SSE proxy. (Snapshot de busca alega od mcp install — [UNVERIFIED/mais novo].)

Relação (a) — BORROWS de open-codesign (CONFIRMADO & CITADO): open-codesign é “the UX north star and our closest peer”; open-design “borrow its streaming-artifact loop, its sandboxed-iframe preview pattern... its five-format export list”. Divergência: “they are a desktop Electron app... we are a web app + local daemon that delegates to your existing CLI.”

L6.2 · OpenCoworkAI/open-codesign

Identidade & maturidade. “Your prompts. Your model. Your laptop.” (“Turn prompts into polished artifacts — locally, openly”). Licença **MIT** (“fork it, ship it, sell it”). Org OpenCoworkAI. **v0.2.0** (“Agentic Design”, 2026-05-09) atual. **App desktop** — brew install --cask opencoworkai/tap/open-codesign (macOS), Scoop (Windows), ou DMG/EXE/ApplImage; **não notarizado/assinado** (xattr -cr documentado). **6,993** (RUN 0; só badge — [UNVERIFIED]) · MIT.

Propósito. App desktop (Electron) MIT, alternativa open-source ao Claude Design, local-first, BYOK-any-model — “without subscription lock-in, cloud-only workflows, or being forced onto a single provider.” Time-to-first-artifact ~3 min.

Capacidades. Produz: prompt → protótipo **HTML ou JSX/React** (iframe sandboxed, React 18+Babel vendored); **slide decks**; assets de marketing; **imagens AI**; **UI kits** (decompose-to-ui-kit → ui_kits/<slug>/ com components/tokens/manifest p/ handoff a coding-agent). **12 skill modules** built-in (slide decks, dashboards, landing pages, SVG charts, glassmorphism, pricing, chat UIs, data tables...) + 15 demos. **Exports: HTML · PDF · PPTX · ZIP · Markdown** (a lista exata de 5 formatos que open-design pega emprestada). Agente: tools ask/scaffold/skill/preview/gen_image/tweaks/todos/done (permissionados: read/write/edit/bash/grep/find). **Self-contained** — bundla próprio runtime (@mariozechner/pi-ai + pi-coding-agent), **não** delega a CLIs externas. **BYOK: 20+ modelos** + OAuth ChatGPT Plus/Pro/Team p/ Codex; importa configs Claude Code/Codex existentes.

Lentes. - *OSS posture*: MIT (sem grant de patente — levemente mais fraco que Apache-2.0). - *S-SDLC fit*: Baixo (design-gen); **loop permissionado = default mais seguro que open-design**. - *SaaS multi-tenant fit*: N/A (app desktop local). - *SDD-methodology fit*: N/A. - *MoE expert-profile*: L6 Design; papel **lean self-contained design studio**; competência alta-focada; persona “design agent de bolso”; abstração média; autorização: loop permissionado local (gated por permission UI).

Segurança. MIT, **fully local Electron** (“Nothing leaves your machine unless your chosen model route requires it”). Keys em ~/.config/open-codesign/config.toml + OAuth store; diagnostics auto-redige keys. **Loop permissionado (mais seguro que o bypass do open-design)**. Instaladores não assinados (signing previsto v0.5), mas **ships SHA256SUMS + SBOM CycloneDX por release** (boa higiene de supply-chain).

Superfície de integração. {filesystem(workspace folders, tools permissionados), memory(DESIGN.md “shared memory” + JSONL session history), rules(SKILL.md taste + DESIGN.md tokens)}. **NÃO presente**: MCP, git/worktree/branch/PR, A2A, P2P, webhooks, TCP, hooks. É GUI desktop (sem CLI).

Relação (d) — é a FONTE de UX que open-design pega emprestada. Divergência: open-codesign = Electron self-contained (bundla pi-ai), mais **enxuto e polido em surgical-edit/tweaks** (open-design lista isso como gap de roadmap); open-design = web+daemon delegando ao seu CLI, **mais amplo** (mais skills/design-systems/mobile/video, 12 harnesses).

L6.3 · pbakaus/impeccable

Identidade & maturidade. “Design guidance for AI coding agents.” (“1 skill, 23 commands, live browser iteration, and 41 deterministic detector rules”). Licença **Apache-2.0**. Autor **Paul Bakaus** (@pbakaus; impeccable.style; npm impeccable). Origem: partiu do skill frontend-design da Anthropic. Instalação npx impeccable skills install + /impeccable init (ou plugin Claude Code). **30,054** (RUN 0; não no README — [UNVERIFIED]) · Apache-2.0.

Propósito. É uma **LAYER de linguagem-de-design/qualidade — NÃO um studio completo**. Problema: modelos treinados nos mesmos templates SaaS produzem os mesmos “tells” (Inter everywhere, gradientes purple-to-blue, cards-in-cards). Persona: quem usa coding-agent para UI frontend e quer que não pareça “AI slop”.

Capacidades. Não produz artefatos próprios — melhora a UI que o host gera. Saídas: `PRODUCT.md` + `DESIGN.md`, critiques, lint reports. **1 skill, 23 commands** (craft, init, document, extract, shape, critique, audit, polish, bolder, quieter, distill, harden, onboard, animate, colorize, typeset, layout, delight, overdrive, clarify, adapt, optimize, live; pináveis a atalho). **Detector: 41 regras determinísticas** (rodam **sem LLM, sem API key**) via `npx impeccable detect src/ (dir/HTML/URL via Puppeteer)` — pega AI-slop (purple gradients, bounce easing) + qualidade geral (line length, touch targets). `/impeccable live` itera variantes visuais no browser. **Cobertura: a mais ampla — 11 harnesses** (Cursor, Claude Code, OpenCode, Pi, Gemini CLI, Codex, Copilot, Kiro, Trae, Rovo Dev, Qoder). **Sem modelo/BYOK próprio** — herda o do host.

Lentes. - *OSS posture*: Apache-2.0. - *S-SDLC fit*: **Médio-forte como quality gate** — o detector determinístico key-free é, na prática, um linter de design plugável em CI. - *SaaS multi-tenant fit*: N/A. - *SDD-methodology fit*: N/A. - *MoE expert-profile*: L6 Design; papel **cross-harness design-quality layer**; competência alta-focada; persona “diretor de arte/QA de design”; abstração média; autorização: lê/escreve `PRODUCT.md/DESIGN.md` + escaneia `dirs/URL`; **executa nenhuma operação privilegiada própria**.

Segurança. Apache-2.0. Roda local dentro do host; **o CLI determinístico não precisa de API key nem LLM** (forte privacidade p/ linting). Sem componente cloud. **Menor superfície de risco do sublayer** — ship instruções/commands em harnesses; risco teórico é o de skill-trust usual (prompts steeram o agente), mas não traz keys nem executa privilégio.

Relação (e) — LAYER vs STUDIO (com evidência). impeccable **não gera nada** — oferece audit/critique/polish/harden; um studio (open-design/open-codesign) *produz* o artefato e dona o loop/modelo/export/preview. **Tiers complementares, não competidores**; ambos descendem de frontend-design e usam `DESIGN.md`. Edge do impeccable vs studio: breadth (11 harnesses) + linter determinístico key-free (que nenhum studio oferece).

L7 — Slides-as-code

Eixo: **ferramenta standalone madura (slidev, 2021, não-agent-native)** vs **skill agêntico (frontend-slides)** vs **deck mode de um studio (open-design)**.

L7.1 · slidevjs/slidev

Identidade & maturidade. “Presentation slides for developers.” Licença **MIT © 2021 Anthony Fu (@antfu)**. Criado em 2021 — **predata a era dos agentes**; é uma ferramenta estabelecida (presenter mode, editor, VSCode ext; sem menção a AI/skills/LLMs no README). Pacote `@slidev/cli`; `npm init slidev` (Node ≥ 20.12). **47,037** (RUN 0; só badge — [UNVERIFIED]) · MIT.

Propósito. Framework **slides-as-code** Markdown standalone para devs que querem escrever talks em Markdown no próprio editor com code highlighting, live coding e componentes Vue. **NÃO agent-native por origem**.

Capacidades. Decks a partir de Markdown. Stack: **Vue 3 + Vite + UnoCSS + Shiki + Monaco** (Vue/Vite/markdown confirmado). Features: themable (theme gallery), componentes Vue embutidos, **Presenter Mode** (controle pelo celular), drawing (Drauu), LaTeX (KaTeX), Mermaid, recording (RecordRTC). **Exports: PDF, PNGs, PPTX** (confirmado exato). Tool human-driven (Vite hot-reload); **sem agent loop, sem modelo, sem BYOK**. **Cobertura de harness: nenhuma** — um agente *pode* escrever Markdown Slidev, mas Slidev não ship integração de harness.

Lentes. - *OSS posture*: MIT; autor OSS estabelecido (ecossistema Vite/Vue). - *S-SDLC fit*: N/A (build tool de slides). - *SaaS multi-tenant fit*: N/A. - *SDD-methodology fit*: N/A. - *MoE expert-profile*: L7 Slides; papel **standalone slides-as-code framework (human-first)**; competência alta-madura; persona “dev autor de talks”; abstração baixa (build tool); autorização: filesystem/projeto npm.

Segurança. MIT; **fully local Node/Vite**, sem cloud/keys/modelo. Superfície mínima (supply-chain npm padrão). **Sem risco de instrução/hook** — não executa instruções LLM.

Superfície de integração. {filesystem(Markdown+projeto), git-repo(projeto npm, hack-ável via Vite plugins)}. **NÃO presente:** MCP, A2A, P2P, harness, worktree/branch/PR/ticket, hooks, webhooks, memory, rules, TCP. É build tool, não server/agente. CLI = @slidev/cli (dev-server/build, não agent-CLI).

L7.2 · zarazhangrui/frontend-slides

Identidade & maturidade. “A Claude Code skill for creating stunning, animation-rich HTML presentations.” Licença **MIT**. Autor @zarazhangrui (“Vibe Coding”). Instalação via plugin marketplace Claude Code (/plugin install frontend-slides@frontend-slides) ou cópia manual a ~/.claude/skills/frontend-slides/; invoca /frontend-slides. **19,093** (RUN 0; não no README — [UNVERIFIED]) · MIT.

Propósito. Skill agêntico de slides — skill Claude Code que transforma conteúdo em deck HTML estilizado por/para o agente. Persona: **não-designers** que não sabem articular preferência estética. Dois casos: criar do zero, ou converter PowerPoint existente em web.

Capacidades — como funciona como skill agêntico (content → deck). Produz **HTML self-contained** (CSS/JS inline, **zero dependências**). Fluxo agêntico “show, don’t tell”: (1) pergunta sobre conteúdo, (2) pergunta o *feeling* (impressed/excited/calm), (3) **gera 3 previews de estilo para comparar**, (4) constrói o deck no estilo escolhido, (5) abre no browser. PPT via scripts/extract-pptx.py (python-pptx). **Arquitetura progressive-disclosure (design-chave de skill):** SKILL.md conciso (~180 linhas) + arquivos on-demand (STYLE_PRESETS.md com 12 presets, viewport-base.css, animation-patterns.md, scripts). Cita harness-engineering OpenAI: “give the agent a map, not a 1,000-page instruction manual.” 12 presets anti-AI-slop (“bye-bye, purple gradients on white”). **Exports: PDF** (export-pdf.sh, Playwright 1920×1080) + **deploy a URL live** (deploy.sh, Vercel). **Sem export PPTX** (importa PPTX, exporta HTML/PDF). **Cobertura: só Claude Code** (CLI requerido); herda o modelo do Claude.

Lentes. - OSS posture: MIT. - S-SDLC fit: N/A. - SaaS multi-tenant fit: N/A. - SDD-methodology fit: N/A. - MoE expert-profile: L7 Slides; papel **agent-native slides skill (Claude Code)**; competência média-focada; persona “designer de slides para não-designers”; abstração média; autorização: filesystem (HTML+scripts) + deploy opcional Vercel.

Segurança. MIT; roda dentro do Claude Code local; outputs HTML estático (“will work in 10 years”). Deploy Vercel opt-in é o único toque cloud. Sem keys próprias. Baixo risco; caveat de skill-trust usual + deploy.sh/Playwright auto-install executam shell (nota menor de supply-chain).

Superfície de integração. {filesystem(HTML único+scripts), git-repo(plugin marketplace)}. **NÃO presente:** MCP, A2A, P2P, worktree/branch/PR/ticket, hooks, webhooks, memory, rules, TCP. É **skill** (instruções+scripts) dentro do Claude Code, não server.

(2) MATRIZ DE COMPARAÇÃO

Expert Layer		Role	License	Stars (RUN 0, 2026-06-27)	Scope	Harness	ACP?	git-flow	S-SDLC	Multi-tenant	Memory	Observability	Security
github/spec-kit	Spec	Spec compiler / life-cycle governor	MIT	106,332	Constitutional brownfield	Don→Claude Code, Copilot default, Cursor, Gemini, Codex...)	Não	branch+for commit	Forces in PR	Retor+ (specification)	Filesystem (specify)	Mélio (analyze cross-artefato)	MIT; GitHub-trust; ext disclaimer
Fission-1 AI/OpenSpec	L1 Spec	Lightweight change-proposer	MIT	51,175	proposed change (Claude folders; brownfield first	25archi (Claude Code, Copilot, Windsurf..)	Sem (ref-eren-ciado; pro-fun-di..)	git-agnóstico (pastas)	Médio-Indireto	Indireto	Spec-as-memory (file)	Baixo	MIT; teleme-tria anôn-ima opt-out
open-gsd/gsd-core	L1 Spec	Context rot orchestrator	MIT	~4.2K [sec]	Discussão sub-agentes context fresco	Claude Code, OpenCode, Gemini, Codex, Cursor...	Não [UNVERIFIED]	worktree (verify-before-ship)	For branch indireto	PR	STATE cross-sessão ify step)	Mélio (ver-ify step)	LINEAGE rug-pull — usar SÓ fork; npm original vivo & perigoso
obra/superpowers	SDLC Workflow	method-ologist (skills)	MIT	~147K [sec]	brainstorm→branch; TDD; worktree	Don→Claude Code, Codex, Cursor, OpenCode, Copilot, Gemini)	Não	worktree	Muito forte (TDD+review gates)	Multi-PR	Não (stateless)	Mélio (ver-ification skill)	MIT; menor footprint de instrução do trio

Expert	Layer	Role	License	Stars (RUN 0, 2026- 06- 27)	Scope	Harness	ACP?	git- flow	S- SDLC	Multi- tenant	Memory	Observability	Security
garrytan/gstack	Workflow	eng-team (personas)	MIT	101,289	Think→ sprint; 23 per-sonas; Open-browser, Cur-sor, Kiro...)	Slip (Claude Code, Codex, GBrain)	Sim (registra)	worktree fecha issue	Forbanc (/cso OWASP+ cov-er-age gate)	Parc (GBrain+STRIDE)	PR; Sim (GBrain+Supabase)	Sim (bench-mark, checkp)	MIT; ree-CLAUDE.md + auto-update horário + scanner próprio
affaan/ECCm	L2 Work-flow (cross: sec+orq)	Harness oper-ator (system)	MIT	~187,620 [sec]	202 skills+in	7+ (Claude Code, Codex, Cur-sor, Open-Code, Zed, Copilot...)	Sim (servers)	worktree + Agents	Multi-branch for Shie (Agents 102 rules)	Parc (GBrain+Supabase)	PR; Sim (GBrain+Supabase)	Sim (audit, met-rics, eval-harness)	MIT; risco de in-strução: CLAUDE.md +
HKUDS/CLI-Anything	Work-flow (tool-builder)	Software native CLI generator	Apache 2.0 [ok]	42-44K [sec]	7-fase pipeline; 40+ CLIs; CLI-Hub	Gera CLIs qual-quer agente (JSON/REPL)	Não (JSON p/ wrap downstream)	lê source (git-repo)	Médio (gera TEST.md)	N/A	Não	Testes (2,330)	Apache-2.0; código ger-ado de repo não-confiável = supply-chain a auditar
ChristopherKahler/paul	Workflow session	Plan-Apply-Unify orchestrator	MIT	976	PLAN/APPLY; manda-tory closure	PLAN/APPLY; CARL (via CARL)	UNIFY; CARL	.paul/ filesystem	Médio- fonte (UNIFY+AC+boas práticas)	N/A	STATE.mn cross-serials)	Médio (Qual-idade loop)	MIT; in-strução pe-sada mas é o produto

				Stars (RUN 0, 2026- 06- 27)	Scope	Harness	LLM?	git- flow	S- SDLC	Multi- tenant	Memory	Observability	Security
Expert Layer	Role	License											
Christopher Kähler	Workflow rule in- jec- tor (be- hav- ioral substrate)	MIT	353	Domain rules just- in- time	Claude Code	Sim (carl- mcp, 15 tools)		file system	Médio (rule gating)	N/A	Decision	Transparência "Ex- plicit Over Magic"	Seletiva não hook in- jeta re- gras si- len- ciosa- mente por prompt Licença ausente na máquina = risco le- gal de reuso (o maior do grupo) Licença null (mesma la- cuna do seed)
Christopher Kähler	Workflow rule in- cubator	MIT- claim- only [!]	288	Typed ideation graduate	Claude Code →PAUL	Não N/A		git init (graduate)	Baixo	N/A	Session persistence	Baixo	Licença ausente na máquina = risco le- gal de reuso (o maior do grupo) Licença null (mesma la- cuna do seed)
(base — sub- strato)	L2 Workflow	Memory sub- strate (knowl- edge graph)	null [!]	87	Knowledge graph + con- text injection	Claude Code	—	—	N/A	N/A	Knowledge graph		Licença null (mesma la- cuna do seed)
nexu- io/open design	L6 Design	Local- first multi- surface de- sign studio	Apache 2.0	48,662	web/mod- HTML/PTX	Claude Code + 11 CLIs (BYOK multi- model)	Não N/A README [UN- VERI- FIED snap- shot diz sim)	clone v4 flags git- check	Baixo	Web- deploya- ble/ mas local- first	SQLite (local)	Stream todos/ calls	Apache- 2.0; [!] by- passPer- mis- sions to- tal, builds não assi- na- dos

Expert	Layer	Role	License	Stars (RUN 0, 2026- 06- 27)	Scope	Harness	MCP?	git- flow	S- SDLC	Multi- tenant	Memory	Observability	Security
OpenCode	Design	self-contained design studio	MIT	6,993	prompt kit; 5 exports	Self-contained (bundla pi-ai); BYOK 20+	Não	slides/PDF/Baixo	(loop per-mis-sion-ado)	N/A (desk-top local)	DESIGN + JSONL	Take counter agent panel	MIT; loop per-mis-sion-ado + SBOM/SHA256 (melhor higiene)
pbakaus	Design	Design-quality layer	Apache 2.0	30,054	Quality LAYER (não stu-dio); 41-rule linter	11 harnesses (a mais ampla)	Não	git-submod	Médio-forte (lin-ter key-free p/ CI)	N/A	Não	Lint re-ports determi	Apache-2.0; menor risco do sub-layer (sem keys, sem priv-ilé-gio)
slidevjs	Slides	Standard slides-as-code (human-first)	MIT (©2021)	47,037	Markdown/Vue/Vite/PNG/PPTX	Nenhuma (não GPTX native)	Não	projeto npm	N/A	N/A	Não	Present mode (human)	MIT; sem risco de instrução LLM (build tool puro)
zarazhang	Slides	native slides skill (Claude Code)	MIT	19,093	content deck zero-dep; PDF+Vercel	HTML Claude Code	Não	plugin marketplace	N/A	N/A	Não	Browser open preview	MIT; baixo risco (de-ploy.sh shell = nota menor)

Nota sobre estrelas: nenhum README do grupo fixa contagem (badges dinâmicos); os valores são RUN 0 (2026-06-27) ordem-de-magnitude. Marcados [sec] = baseados em fontes secundárias com dispersão alta (superpowers ~147K vs busca live ~180K; ECC README hardcoda “182K+” vs busca 222K+; gstack 66-108K). **Tratar todos como [UN-VERIFIED] no dígito exato.**

(3) ANÁLISE CRUZADA DO GRUPO — overlaps, colaboração vs competição vs incompatibilidades (com evidência)

(a) OpenSpec vs Spec Kit — leve/iterativo vs formal/template-rich

Competição direta, mesma vaga (a camada de spec do projeto). O próprio OpenSpec se posiciona contra o Spec Kit: framing de “Thorough but heavyweight... OpenSpec is lighter” e filosofia “*fluid not rigid, iterative not waterfall.*” **Evidência de divergência:** Spec Kit é phase-gated (constitution→specify→plan→tasks→implement, com clarify obrigatório e checklist/analyze como gates), escreve .specify/+specs/NNN/, cria branch+Issues+PR; OpenSpec é **git-agnóstico** (organiza openspec/changes/<name>/ folders, “*update any artifact anytime*”, sem gates). **Incompatibilidade prática:** rodar ambos no mesmo repo gera **dois sistemas de spec concorrentes** (dois diretórios, duas convenções de comando /speckit.* vs /opsx.*, dois conjuntos de arquivos de guidance do agente) — escolha mutuamente exclusiva na prática. **Quando cada um:** Spec Kit p/ enterprise/compliance/greenfield com governança; OpenSpec p/ brownfield iterativo leve. gsd-core ocupa um terceiro nicho (anti-context-rot por subagentes), ortogonal aos dois mas ainda escrevendo seu próprio STATE.md/CONTEXT.md — três SSOTs de spec não convergem.

(b) Superpowers vs GStack vs ECC — a COLISÃO da camada de instrução (“como o agente deve se comportar?”)

A incompatibilidade mais aguda do grupo. Os três governam comportamento always-on, mas por **mecanismos diferentes e parcialmente sobrepostos** — instalar mais de um cria conflito real:

Mecanismo de injeção	superpowers	gstack	ECC
Edita/escreve CLAUDE.md	Não (instruções de plugin)	Sim (## gstack + ## Skill routing)	Sim (CLAUDE.md + AGENTS.md)
Arquivos rules/ always-on	Só skill-preamble	Seções CLAUDE.md	Sim (rules/ecc/, alwaysApply, copilot-instructions)
Hooks (hooks.json, por-turno)	Não	Não (roteado por CLAUDE.md)	Sim (Pre/PostToolUse/Stop/SessionStart..)
SessionStart context injection	Não	Não	Sim (≤8000 chars)
Diretivas “Never use tool X”	Não	Sim (bane mcp__claude-in-chrome__*)	Implícito via routing
Store de memória persistente	Não	Sim (GBrain)	Sim (SQLite+MCP)
Bundla MCP	Não	só GBrain	Sim (~14)
Scanner de segurança próprio	Não	Sim (ML browser/injection)	Sim (AgentShield, 102 rules)
Footprint / risco de colisão	O menor	Médio-alto	O maior

Riscos de colisão co-instalados (com evidência): (i) **contenção de escrita em CLAUDE.md** entre gstack e ECC (ambos editam; ECC ainda adiciona AGENTS.md); (ii) **gstack bane mcp__claude-in-chrome__*** enquanto ECC e outros bundlam/esperam tooling MCP de browser — **conflito direto de política de ferramenta**; (iii) o **próprio README do ECC alerta “Duplicate hooks file detected” / “duplicate execution”** — pioraria com qualquer outro instalador de hooks; (iv) **erosão de contexto empilha** — ~14 MCP servers do ECC + preâmbulo SessionStart 8000-char + seções

CLAUDE.md do gstack + skill-preamble do superpowers competem pela janela (ECC documenta 200k→~70k); (v) três camadas de instrução independentes assertando workflows “mandatory” = **precedência imprevisível** em TDD/worktree/review. **Conclusão: escolher um, não empilhar.** superpowers é o mais “convivente” (skill-activation, sem hooks/CLAUDE.md-edit); gstack e ECC são os mais invasivos e os mais mutuamente incompatíveis. (Nota: a suite Kahler — CARL especialmente — é um quarto injetor always-on, com hook por prompt + MCP; CARL+ECC empilhariam dois hook-engines.)

(c) PAUL vs GSD — filosofia (in-session+UNIFY vs subagentes+closure-opcional)

Antagonismo filosófico explícito e documentado (PAUL nomeia GSD diretamente). Citação verbatim: “PAUL takes a different approach from GSD.” — tabela contrastando GSD (Parallel subagents / Optional closure / Static prompts) vs PAUL (In-session context / Mandatory UNIFY / CARL dynamic loading); “PAUL prioritizes quality over speed-for-speed’s-sake.” **A ironia:** gsd-core **também** evoluiu para resolver o mesmo problema (context rot) — mas pela rota **oposta** que PAUL rejeita (subagentes de contexto fresco). São duas teses concorrentes sobre o mesmo mal: **PAUL = manter contexto na sessão + fechamento obrigatório; GSD = expulsar trabalho pesado para subagentes frescos.** Incompatíveis por design (não se compõem); um usuário escolhe a filosofia.

(d) Open Design vs Open CoDesign — borrow explícito (north star vs derivado amplo)

Colaboração/lineage, não competição hostil — mas ocupam a mesma vaga (alternativa OSS local-first ao Claude Design). **Evidência citada:** open-design declara open-codesign “the UX north star and our closest peer” e lista o que pega emprestado (streaming-artifact loop, sandboxed-iframe preview React 18+Babel, live agent panel, lista de 5 exports). **Divergência arquitetural:** open-codesign = **Electron self-contained** (bundla pi-ai, loop **permissionado**, mais polido em surgical-edit/tweaks, +SBOM); open-design = **web+daemon delegando ao seu CLI, mais amplo** (mobile/vídeo/áudio, 12 harnesses, mais design-systems) **mas com default de segurança pior** (bypass total de permissões). **Trade-off claro:** open-codesign = lean+seguro+polido; open-design = amplo+multi-harness+arriscado. Coexistem (origem vs derivado), mas para um mesmo job o usuário escolhe um.

(e) slidev (standalone) vs frontend-slides (agêntico) vs open-design slide export

Overlap apenas no output (emitir um deck); posicionamentos não-competem.

Eixo	slidev (tool standalone)	frontend-slides (skill agêntico)	open-design deck mode (feature de studio)
Natureza	Framework Vue/Vite, human-authored	Skill Claude Code (agent-native)	Um modo de um studio completo
Input	Markdown (você escreve)	Conteúdo + “feeling” (agente entrevista)	Prompt → agente gera
Output	Slides Vue-rendered	HTML single-file zero-dep	HTML decks (guizang-ppt)
Dependências	Toolchain Node/Vue/Vite/UnoCSS	Zero (CSS/JS inline)	Via daemon do studio + seu CLI
Export	PDF / PNG / PPTX	PDF + Vercel URL (sem PPTX out)	HTML / PDF / PPTX / ZIP / MD
Agent-native?	Não (2021, predata agentes)	Sim (skill propositado)	Sim (studio)
Anti-AI-slop	Themes (não slop-focused)	Explícito (12 estilos curados)	Sim (DESIGN.md anti-patterns)

Diferenciação: slidev é o polo **maduro/human-first** (runtime mais rico — presenter mode, live coding, LaTeX, Mermaid — mas exige Vue/Vite e não é agent-native); frontend-slides é o **skill agent-native leve** (single-file HTML zero-dep, UX conversacional “3 previews”, restrito a Claude Code); open-design deck mode é **uma fatia de um studio multi-superfície** (HTML+PPTX real, 12 harnesses). **Síntese:** quem já vive num agente Claude Code e quer um deck rápido → frontend-slides; quem quer controle/runtime de apresentação maduro → slidev; quem já usa o studio open-design p/ design geral → o deck mode reusa o mesmo pipeline. Não há canibalização — são três tiers (framework / skill / studio-feature) que **convergem só no formato de saída**.

Síntese de pipeline (o fio que liga L1→L2→L6→L7)

O grupo 1c é uma **esteira**, não um conjunto de concorrentes: **L1** decide *o que* construir (spec), **L2** decide *como o agente se comporta* construindo, **L6** gera a *face* (UI/design), **L7** gera a *narrativa* (slides). As incompatibilidades reais são **intra-camada** (dois specs, dois injetores de instrução, dois studios, dois geradores de slides competem pela mesma vaga), enquanto a composição **inter-camada** é natural (ex.: spec-kit gera spec → superpowers executa com TDD → open-design produz UI → frontend-slides apresenta). A maior armadilha operacional do grupo é **empilhar L2** (superpowers+gstack+ECC+CARL todos always-on) — colisão de CLAUDE.md/hooks/rules garantida.

Fontes

L1 — Spec-Driven Development - github/spec-kit: <https://raw.githubusercontent.com/github/spec-kit/main/README.md> · <https://github.com/github/spec-kit> - Fission-AI/OpenSpec: <https://raw.githubusercontent.com/Fission-AI/OpenSpec/main/README.md> · <https://raw.githubusercontent.com/Fission-AI/OpenSpec/main/docs/supported-tools.md> - open-gsd/gsd-core: <https://raw.githubusercontent.com/open-gsd/gsd-core/next/README.md> - GSD rug-pull (supply-chain): <https://github.com/open-gsd/get-shit-done-redux/discussions/109> · <https://shieldguard.io/scam-alert-the-get-shit-done-gsd-ai-agent-rug-pull-active-npm-threat/> · <https://aiweekly.co/alerts/get-shit-done-creator-rug-pulls-gsd-token-vanishes> · <https://cryptoadventure.com/bag-of-hackathon-winner-gsd-hit-by-alleged-rug-pull-after-founder-exit/> · <https://intellectia.ai/news/crypto/gsd-token-allegedly-rugpulled-after-founder-exit> · <https://github.com/gsd-build/get-shit-done> (ORIGINAL — comprometido, não instalar)

L2 — Workflow & Skill Frameworks - obra/superpowers: <https://raw.githubusercontent.com/obra/superpowers/main/README.md> · <https://github.com/obra/superpowers/> - garrytan/gstack: <https://raw.githubusercontent.com/garrytan/gstack/main/README.md> · <https://github.com/garrytan/gstack> - affaan-m/ECC: <https://raw.githubusercontent.com/affaan-m/ECC/main/README.md> · <https://ecc.tools/> · <https://github.com/affaan-m/ECC/releases/tag/v2.0.0> - HKUDS/CLI-Anything: <https://raw.githubusercontent.com/HKUDS/CLI-Anything/main/README.md> · <https://raw.githubusercontent.com/HKUDS/CLI-Anything/main/LICENSE> · <https://www.developersdigest.tech/blog/trending-cli-anything-2026-05-24> · <https://github.com/HKUDS/CLI-Anything> - ChristopherKahler/paul: <https://raw.githubusercontent.com/ChristopherKahler/paul/main/README.md> · <https://api.github.com/repos/ChristopherKahler/paul> - ChristopherKahler/carl: <https://raw.githubusercontent.com/ChristopherKahler/carl/main/README.md> · <https://api.github.com/repos/ChristopherKahler/carl> - ChristopherKahler/seed: <https://raw.githubusercontent.com/ChristopherKahler/seed/main/README.md> · <https://api.github.com/repos/ChristopherKahler/seed> · <https://api.github.com/repos/ChristopherKahler/seed/licenses> (vazio = sem LICENSE) - ChristopherKahler/base: <https://api.github.com/repos/ChristopherKahler/base> (license:null)

L6 — Generative Design - nexu-io/open-design: <https://raw.githubusercontent.com/nexu-io/open-design/main/README.md> · <https://github.com/nexu-io/open-design> · <https://github.com/nexu-io/open-design/releases> · <https://github.com/RavenTress/nexu-open-design> (fork congelado: “19 Skills / 71 Design Systems”) - OpenCoworkAI/open-codesign: <https://raw.githubusercontent.com/OpenCoworkAI/open-codesign/main/README.md> · <https://github.com/OpenCoworkAI/open-codesign> - pbaka.us/impeccable: <https://raw.githubusercontent.com/pbaka.us/impeccable/main/README.md> · <https://impeccable.style> - BASELINE: Anthropic claude-design / frontend-design (proprietário; lineage comum de impeccable/open-design/open-codesign)

L7 — Slides-as-code - slidevjs/slidev: <https://raw.githubusercontent.com/slidevjs/slidev/main/README.md>

· <https://sli.dev> - [zarazhangrui/frontend-slides: https://raw.githubusercontent.com/zarazhangrui/frontend-slides/main/README.md](https://raw.githubusercontent.com/zarazhangrui/frontend-slides/main/README.md)

Phase 1 · Group 1c (Build Pipeline) · 17 experts · L1 Spec / L2 Workflow / L6 Design / L7 Slides · observed 2026-06-27. Prosa pt-BR, identificadores en-US. Compilado de 4 sweeps de evidência primária (READMEs/LICENSE/GitHub API/releases) + corroboração secundária. ~40 URLs distintas.

Parte II — Fase 1: (d) Amplificador L3

Group 1d — Amplifier: L3 Multi-Agent Orchestration / MoE Engines

Group: 1d — Amplifier **Phase:** 1 (Expert Profiling) **Observed:** 2026-06-27 **Language note:** Prose em pt-BR; identificadores, nomes de repo, comandos e URLs em en-US. **Author:** Claude-Analyst (subagent via maos research pipeline)

Sumário Executivo

Este documento perfila os dois exemplares MoE (Mixture-of-Experts) de L3 incluídos no mapa N-Tree do Hub: **ruvnet/ruflo** (ex claude-flow, runtime de swarm infrastructure-heavy) e **bmad-code-org/BMAD-METHOD** (framework de desenvolvimento agile persona-driven). Quatro vizinhos recebem mini-cards. A seção de comparação explora quando orquestração pesada é overkill, e a seção MoE extrai os padrões de gating reutilizáveis para o Hub.

1. Profile Card — ruvnet/ruflo

1.1 Identidade e Maturidade

Campo	Valor
Repositório	https://github.com/ruvnet/ruflo
Nome anterior	claude-flow (renomeado para ruflo em 2025)
Stars (2026-06-27)	~61.700
Forks	~7.200
Releases	1.547 (latest v3.14.3, Jun 27, 2026)
Commits	7.079
Criado	2025-06-02 (como claude-flow)
Licença	MIT
Linguagens	TypeScript 82,6% · JavaScript 10,5% · Shell 4,1% · Rust 0,6%
Powered by	Cognitum.One agentic architecture
Suporte enterprise	ruv.io

Maturidade: Alta. 1.547 releases em ~1 ano indicam cadência de shipping contínua. 470 issues abertas e 210 PRs ativos sinalizam ecossistema vivo mas com débito de manutenção crescente. O projeto se auto-descreve como “the leading agent meta-harness for Claude.”

1.2 Propósito — Escopo e Persona

Ruflo é um **meta-harness de execução**: envolve Claude Code e Codex adicionando 100+ agentes especializados, memória vetorial persistente, loops de auto-aprendizado, comunicação federada entre máquinas e guardrails de segurança enterprise. O lema arquitetural é “Agent = Model + Harness” — o modelo escreve código, o harness dá ferramentas, memória, loops, sandboxes e controles.

Persona alvo: Equipes de engenharia que já usam Claude Code e querem coordenação multi-agente sem montar a infraestrutura do zero. Um `npx ruflo init` instala o “sistema nervoso” completo.

Casos de uso primários: - Swarms paralelos de codificação, testes, revisão e documentação - Análise de segurança e CVE remediation autônoma - Federação multi-máquina/multi-organização com zero-trust - Rastreamento de custo/token e otimização de provider - Automação de ADRs, DDD scaffolding e SPARC methodology

1.3 Capacidades

Topologia de Orquestração O roteador central (MCP Server + Router + 27 Hooks) recebe tarefas do Claude Code ou CLI e as despacha para a camada de Swarm Coordination. Três topologias de consenso são suportadas:

Hierárquica (Queen-led) – Raft consensus
Mesh (peer-to-peer) – Gossip protocol
Adaptativa – Byzantine fault tolerance

A **Queen** é o coordenador primário do swarm hierárquico: recebe a tarefa, seleciona topologia via roteador inteligente (“89% accuracy” claimed), instancia sub-agentes como workers e coordena via HNSW-indexed AgentDB com sub-ms retrieval.

Roster de Agentes (100+) 35 plugins agrupados em domínios:

Domínio	Plugins-chave
Core & Orquestração	ruflo-core, ruflo-swarm, ruflo-autopilot, ruflo-loop-workers, ruflo-workflows, ruflo-federation
Memória & Conhecimento	ruflo-agentdb, ruflo-rag-memory, ruflo-rvf, ruflo-ruvector, ruflo-knowledge-graph
Inteligência & Aprendizado	ruflo-intelligence, ruflo-graph-intelligence, ruflo-daa, ruflo-ruvllm, ruflo-goals
Qualidade & Testes	ruflo-testgen, ruflo-browser, ruflo-jujutsu, ruflo-docs
Segurança	ruflo-security-audit, ruflo-aidefence
Arquitetura & Metodologia	ruflo-adr, ruflo-ddd, ruflo-sparc, ruflo-metaharness, ruflo-arena
DevOps & Observabilidade	ruflo-migrations, ruflo-observability, ruflo-cost-tracker
Extensibilidade	ruflo-agent, ruflo-plugin-creator
Domain-specific	ruflo-iot-cognitum, ruflo-neural-trader, ruflo-market-data

Primitivas de Coordenação

- **Hooks:** 27 hooks no orchestration layer (PreToolUse, PostToolUse, SessionStart, UserPrompt-Submit, etc.)
- **MCP Server:** 314 ferramentas MCP expostas; Web UI usa ~210 simultaneamente com parallel tool calling

- **Background Workers:** 12 workers auto-ativados por palavras-chave (ultralearn, optimize, audit, document, refactor, test)
- **GOAP A* Planner:** goal.ruv.io — objetivos em linguagem natural → planos executáveis com state-space search
- **Federation:** 9 MCP tools + 10 CLI commands para ciclo de vida completo (federation_init, federation_send, federation_trust, federation_audit)

Memória e Auto-Aprendizado

Componente	Descrição
AgentDB	Banco vetorial HNSW-indexed (~1,9x mais rápido que brute-force em N=20k; recall@10 ~0,99)
SONA	Self-Optimizing Neural Architecture — padrão Micro-LoRA, EWC++ (continual learning sem catastrophic forgetting), 300x faster pattern retrieval
ReasoningBank	Armazenamento de trajetórias bem-sucedidas com reward scores; cross-agent knowledge sharing
Learning Loop	4 fases: RETRIEVE (top-k MMR) → JUDGE (LLM-as-judge) → DISTILL (estratégias) → CONSOLIDATE (deduplication + pruning)
ruvector	GPU-accelerated search; Graph RAG; 103 ferramentas

Harness Coverage Plugin marketplace: 33 plugins nativos para Claude Code + 21 npm plugins. Path A (plugin install) vs Path B (CLI init com daemon, MCP server, hooks completos).

1.4 Lentes de Análise

OSS Posture: MIT, totalmente aberto. Trademark “ruflo” não registrado no repo (verificar). Comunidade ativa via Agentics Foundation Discord.

S-SDLC Fit (Secure Software Development Lifecycle): Alto — possui ruflo-security-audit (CVE scanning), ruflo-aidefence (prompt injection blocking, 14-type PII detection pipeline), path traversal prevention, HIPAA/SOC2/GDPR audit trails. MetaHarness grading (1-100) para avaliação de prontidão.

SaaS Multi-Tenant Fit: Médio-alto. Federation com mTLS + ed25519 e trust scoring (0,4×success + 0,2×uptime + 0,2×threat + 0,2×integrity) apoia isolamento de tenants. Mas a ausência de documentação explícita de multi-tenancy e o modelo de npx init local indicam design primário single-tenant/on-prem.

SDD-Methodology Fit: Muito alto — ruflo-ddd scaffold domain-driven design (contextos, aggregates, events); ruflo-adr rastreia decisões de arquitetura; ruflo-sparc guia metodologia 5-phase.

MoE Expert Profile:

Dimensão	Ruflo
Layer	L3 — Amplifier (infra de orquestração)
Role	Runtime coordinator + swarm dispatcher
Competência	Task routing inteligente, consensus protocols, self-learning memory
Persona	Infrastructure engineer que quer “sistema nervoso” plug-and-play
Abstração	Meta-harness sobre Claude Code / Codex CLI

Dimensão	Ruflo
Authorization	Behavioral trust scoring (federation); AIDefence (runtime)

1.5 Segurança

Superfície de ataque expandida (runtime de orquestração):

- 1. **Prompt injection via agentes externos:** AIDefence detecta e bloqueia; pipeline de 14 tipos de PII com políticas por trust-level (BLOCK, REDACT, HASH, PASS)
- 2. **Federation hijacking:** Mitigado por mTLS + ed25519 challenge-response; sem API keys ou segredos compartilhados; audit trail bilateral; downgrade imediato por comportamento malicioso
- 3. **Privilege escalation de sub-agentes:** WASM sandbox local (rvagent) + Anthropic Claude Managed Agents para isolamento
- 4. **Supply chain via plugins:** 35 plugins npm-based; sem evidência de pinning de versão ou SCA automatizado no CI — risco MEDIUM [UNVERIFIED que SCA está implementado]
- 5. **Benchmark inflation:** Claims de “89% routing accuracy” e “1.953x faster than LangGraph” são self-reported sem auditoria independente publicada [UNVERIFIED]

Licença: MIT — compatível com uso comercial; ausência de copyleft.

1.6 Superfície de Integração

Tags aplicáveis: MCP filesystem git-repo git-branch git-worktree memory hooks webhooks TCP

- **MCP:** MCP server primário (npx ruflo@latest mcp start); 314 ferramentas; Web UI com parallel tool calling
- **filesystem:** .claude/, .claude-flow/, CLAUDE.md, agent definitions em .agents/
- **git-repo / git-branch / git-worktree:** ruflo-jujutsu analisa git diffs, scoring de risco, sugestão de revisores
- **memory:** AgentDB HNSW + ReasoningBank SQLite + ruvector
- **hooks:** 27 hooks no orchestration layer; hook system nos background workers
- **TCP / federation:** WireGuard mesh layer opcional (ADR-111); QUIC transport via agentic-flow (projeto irmão)
- **webhooks:** [UNVERIFIED — não documentado explicitamente no README principal]

2. Profile Card — bmad-code-org/BMAD-METHOD

2.1 Identidade e Maturidade

Campo	Valor
Repositório	https://github.com/bmad-code-org/BMAD-METHOD
Stars (2026-06-27)	~49.600 (README badge: 49,6k; página de find: 48,8k em sessão anterior — flutuação normal)
Forks	~5.700
Releases	37 (latest v6.9.0, Jun 22, 2026)
Commits	1.936
Criado	2025-04-13
Licença	MIT (verificado — LICENSE file confirma “MIT License, Copyright (c) 2025 BMad Code, LLC”)
Linguagens	JavaScript 65,4% · HTML 21,3% · Python 10,6% · Shell 0,8%

Campo	Valor
npm	npx bmad-method install
Empresa	BMad Code, LLC

Nota crítica sobre licença: A task spec citava NOASSERTION. Isso foi **refutado** pela leitura direta do arquivo LICENSE — o projeto usa MIT padrão. A GitHub UI mostrando NOASSERTION em alguns contextos é um artefato de parsing do campo SPDX quando há trademark riders combinados com MIT; o texto legal completo é MIT inequívoco. TRADE-MARK separado cobre “BMad™”, “BMad Method™”, “BMad Core™”.

Maturidade: Alta e em ascensão rápida. 37 releases em ~14 meses, v6.9.0 como release mais recente. 49,6k stars em ~14 meses indica adoção viral. 1.936 commits e 5,7k forks mostram ecossistema ativo.

2.2 Propósito — Escopo e Persona

BMAD-METHOD é um **framework de metodologia ágil AI-driven** — não é um runtime de orquestração de baixo nível, mas um sistema de coordenação de alto nível via **personas de agentes especializados**. O foco é o processo: planejamento, arquitetura, implementação e QA, guiados por personas de domínio que colaboram como uma equipe humana virtual.

Tagline: “Breakthrough Method for Agile AI-Driven Development” — agentes atuam como “expert collaborators who guide you through a structured process.”

Personas alvo: - Desenvolvedores individuais e pequenas equipes que querem estrutura de planning sem overhead de infraestrutura - Times que usam Claude Code, Cursor ou outros AI IDEs - Projetos de complexidade variável (bug fixes a sistemas enterprise — “scale-domain-adaptive”)

Dois modos de uso: 1. **IDE-native** (Claude Code, Cursor, etc.): instala arquivos CLAUDE.md, agentes e workflows no projeto 2. **Web Bundles** (Gemini Gems, ChatGPT GPTs): planning em subscrição flat-rate, artefatos trazidos para o IDE

2.3 Capacidades

Topologia de Orquestração — Modelo Persona Handoff O BMAD não tem um runtime de swarm explícito. A orquestração ocorre via **handoff de personas** dentro de sessões de IDE:

```

Usuário → BMad Orchestrator (gating agent)
      ↓
  Seleciona persona conforme fase do projeto:
  [Analyst] → [PM] → [Architect] → [Developer] → [QA] → [DevOps]
      ↓
  "Party Mode": múltiplas personas na mesma sessão
  (discussão multi-perspectiva intra-sessão)

```

O **BMad Orchestrator** é o gating network: recebe o contexto do projeto, avalia a fase atual (brainstorming / planning / architecture / implementation / testing) e faz handoff para a persona correta. O skill bmad-help funciona como oracle de roteamento: “bmad-help I just finished the architecture, what do I do next?”

Roster de Agentes (12+ personas especializadas)

Persona	Competência
BMad Orchestrator	Coordenação geral, roteamento de fase, handoff entre personas
Analyst	Brainstorming, PRFAQ, market research, problem framing
Product Manager (PM)	PRD, product brief, backlog, user stories
Architect	System design, tech decisions, ADRs, API contracts

Persona	Competência
Developer	Implementation, code review, pair programming
UX Expert	UX specs, design thinking, accessibility
QA/Tester	Test strategy, automation, risk-based testing
DevOps	CI/CD, infrastructure, deployment
Scrum Master	Sprint planning, retrospectives, velocity
Technical Writer	Documentation, API docs
Business Analyst	Requirements elicitation, stakeholder mapping
Security Engineer	Threat modeling, security review

Party Mode: invoca múltiplas personas na mesma sessão para debate e validação cruzada — análogo ao padrão MoE com múltiplos experts ativos simultaneamente.

Módulos Oficiais (ecosystem extensor)

Módulo	Código	Purpose
BMad Method (BMM)	bmm	Core — 34+ workflows
BMad Builder (BMB)	bmb	Criação customizada de agentes e workflows
Test Architect (TEA)	tea	Risk-based test strategy, enterprise
Game Dev Studio (BMGD)	gds	Unity, Unreal, Godot
Creative Intelligence Suite (CIS)	cis	Innovation, brainstorming
Whiteport Design Studio (WDS)	wds	Strategic UX/Design-first planning
BMad Automator (experimental)	automator	Epic builder, Claude/Codex only

Scale-Domain-Adaptive Intelligence O framework detecta automaticamente a complexidade do projeto e ajusta a profundidade de planning: - Bug fix / pequena feature → workflow curto, menos artefatos - Sistema enterprise → planning completo, todos os artefatos (PRFAQ, PRD, ADR, test strategy)

Harness Coverage Instalável via `npx bmad-method install` com suporte a: - Claude Code (primary) - Cursor - Outros AI IDEs (via instalação de arquivos) - Gemini / ChatGPT (via web bundles para planejamento)

Possui `.claude-plugin/` — compatível com o sistema de plugins do Claude Code.

2.4 Lentes de Análise

OSS Posture: MIT genuíno com trademark rider. Totalmente gratuito, sem gated content ou Discord pago. Sponsor via [buymeacoffee](#). Suporte corporativo via contact@bmadcode.com.

S-SDLC Fit: Médio. Foco em processo, não em runtime security. Não há equivalente do AIDefence/ruflo-security-audit. O módulo Test Architect (TEA) cobre risk-based testing e release gates. Sem evidência de CVE scanning, PII detection ou prompt injection blocking nativos.

SaaS Multi-Tenant Fit: Baixo. BMAD é inerentemente single-project/single-session. Não há mecanismos de isolamento de tenant, quotas ou autenticação de multi-tenant. Para SaaS, seria preciso envolver o framework em uma camada de orquestração externa.

SDD-Methodology Fit: Muito alto — este é o ponto forte. O framework é uma metodologia SDD (Structure-Driven Development) completa: cada fase do ciclo de vida tem a persona correspondente, workflows documentados e artefatos esperados.

MoE Expert Profile:

Dimensão	BMAD
Layer	L3 — Amplifier (metodologia de desenvolvimento)
Role	Process orchestrator + persona gating
Competência	Estruturação de processo, handoff de fase, geração de artefatos
Persona	Time de desenvolvimento querendo estrutura sem overhead de infra
Abstração	Framework de agentes sobre AI IDE (Claude Code / Cursor)
Authorization	Sem mecanismo runtime; autorização implícita via persona handoff

2.5 Segurança

Superfície de ataque (framework de metodologia):

1. **Prompt injection via artefatos de planning:** Os agentes consomem PRDs, ADRs e specs como contexto — documentos maliciosos poderiam injetar instruções [RISCO BAIXO em uso interno, MÉDIO em contextos multi-stakeholder]
2. **Supply chain npm:** `npx bmad-method install` baixa e executa código de npm; sem SCA explícito documentado [UNVERIFIED]
3. **Trademark misuse:** TRADEMARK.md documenta restrições de uso — risco legal para derivativos que usam “BMad” no nome
4. **Web Bundles (Gemini/ChatGPT):** Artefatos de planning podem vaziar IP da empresa para terceiros via serviços externos [risco de data governance]
5. **Husky hooks:** `.husky/` no repo indica git hooks — potencial vetor se o repo for comprometido

Licença MIT (confirmado): compatível com uso comercial; sem copyleft; trademark restrictions separados.

2.6 Superfície de Integração

Tags aplicáveis: `filesystem` `git-repo` `memory` `rules` `hooks`

- **filesystem:** Instala arquivos CLAUDE.md, agentes (.md), workflows no diretório do projeto
- **git-repo:** `.github/workflows/` para CI/CD; `.husky/` para git hooks
- **memory:** Sem memória vetorial persistente nativa — estado é contextual (dentro de sessão de IDE)
- **rules:** CLAUDE.md como sistema de rules para o AI IDE
- **hooks:** Husky git hooks; Claude Code plugin hooks via `.claude-plugin/`
- **MCP:** `.claude-plugin/` confirma compatibilidade com sistema MCP do Claude Code [integração via plugin manifest]
- **ticket/backlog:** Artefatos de PM (PRD, user stories) são o “backlog” — sem integração direta com Jira/Linear [UNVERIFIED se módulo externo existe]

3. Mini-Cards — Vizinhos

3.1 ruvnet/agent-flow

Stars (2026-06-27): ~762 | **Licença:** Sem licença explícita (NO LICENSE) | **npm:** `agent-flow`

Experimento de orquestração adjacente do mesmo autor (rUv), predecessor direto do ruflor. Versão mais enxuta com 66 agentes, 213 MCP tools, SONA neural architecture (Micro-LoRA, EWC++, Flash Attention 2,49x-7,47x), AgentDB alpha com 5 mecanismos de atenção (flash, multi-head, linear, hyperbolic, MoE) e GraphRoPE. Topology support: ring, mesh, hierarchical, star. Sem licença formal — **NÃO usar em produção sem clareza legal**. Funciona como incubadora de features que migram para ruflor.

3.2 parruda/swarm

Stars (2026-06-27): ~1.700 | **Licença:** MIT | **Linguagem:** Ruby 98,8%

Renomeado de claude-swarm; agora abriga três gems separadas: swarm_sdk (single-process Ruby orchestration), swarm_cli (REPL interativo) e swarm_memory (memória persistente com FAISS + ONNX embeddings). v2 elimina dependência do Claude Code CLI — todos os agentes rodam em um único processo Ruby via RubyLLM, com chamadas de método diretas em vez de MCP inter-process. Suporta múltiplos providers. Node workflows com DAG de dependências. 12 hook events. Integração Rails documentada. Ideal para automação geral, data processing e customer support em stacks Ruby. Última release: swarm_sdk v2.7.15, Fev 2026.

3.3 smtg-ai/claude-squad

Stars (2026-06-27): ~7.500 | **Licença:** AGPL-3.0 | **Linguagem:** Go 89,1%

Terminal manager para múltiplos agentes AI (Claude Code, Codex, Gemini, Aider) em paralelo. Usa tmux para sessões isoladas e git worktrees para evitar conflitos de arquivo. **Não é um orquestrador de alto nível** — é uma ferramenta de gerenciamento de processos com TUI. Suporta auto-accept mode (-y). Cada sessão roda em branch própria. Última release v1.0.17, Mar 2026. AGPL-3.0 implica copyleft se modificado e distribuído — verificar compatibilidade antes de embed em produto comercial.

4. Padrões MoE / Gating Network — Extração e Reuso

4.1 Como Ruflo Roteia Agentes (Queen Topology)

O gating network do ruflo opera em dois níveis:

Nível 1 — Topology Selection (macro-routing):

Router recebe tarefa

- Avalia: paralelismo necessário? coordenação de consenso? latência?
- Seleciona: Hierarchical (Queen-Raft) | Mesh (Gossip) | Adaptive (Byzantine)

Nível 2 — Agent Dispatch (micro-routing):

Queen recebe tarefa + contexto HNSW

- SONA Micro-LoRA classifica domínio da tarefa
- MoE Attention routes para top-K agentes especializados
- AgentDB recupera padrões similares do passado (ReasoningBank)
- Workers executam em paralelo ou sequência
- Resultado consolidado via consensus protocol

Estado mantido: - AgentDB: vetores de tarefas + outcomes (HNSW, SQLite) - ReasoningBank: {sessionId, task, input, output, reward[0-1], success, critique, tokensUsed, latencyMs} - Trust scores de federation: histórico de comportamento por agente remoto

Padrão reutilizável para o Hub: O ciclo RETRIEVE→JUDGE→DISTILL→CONSOLIDATE (4 fases do learning loop) é um primitivo de memória episódica que qualquer HUB pode adotar sem reimplementar. O gating 89% accuracy via intelligent routing aponta para classificação semântica de tarefa + lookup HNSW como mecanismo central.

4.2 Como BMAD Roteia Agentes (Persona Handoff)

O gating network do BMAD é **stateless e declarativo**:

Usuário descreve situação/necessidade

- bmad-help analisa: "Em qual fase do SDLC você está?"

- Mapeia fase → persona responsável
- Handoff contextual (persona assume a sessão)
- [opcional] Party Mode: múltiplas personas ativas para debate

Gating signal: A fase do projeto (brainstorm / requirements / architecture / implementation / testing / deployment) é o sinal de gating. Não há vetor de embedding — é roteamento simbólico/declarativo.

Estado mantido: - Artefatos gerados (PRD, ADR, specs) persistem no filesystem do projeto - Sem memória episódica entre sessões (cada sessão de IDE começa fresh)

Padrão reutilizável para o Hub: O mapeamento fase_do_projeto → persona_ativa é um esquema de gating hierárquico de baixo custo computacional, extremamente interpretável. Para contextos onde o overhead de HNSW é desnecessário (projetos simples, budgets pequenos), este padrão é preferível.

4.3 NanoClaw (ECC — Grupo 1c) como Terceiro Exemplar

O NanoClaw (de grupo 1c) representa o extremo oposto: gating **ultra-leve** via regras de pattern matching sobre o tipo de token de código. Não há LLM intermediário no roteamento — o compilador/linter decide qual “expert” (transform) aplicar. Reutilizável em contextos onde a decisão de routing pode ser determinística (e.g., roteamento por tipo de arquivo, linguagem ou tag de domínio).

4.4 Tabela Comparativa de Gating Patterns

Dimensão	Ruflo (Queen/HNSW)	BMAD (Persona Handoff)	NanoClaw (Rule-based)
Mecanismo de routing	MoE Attention + HNSW semantic search	Fase do SDLC → persona map	Pattern matching determinístico
Custo computacional	Alto (embedding + vector search)	Baixo (simbólico)	Mínimo (O(1) lookup)
Adaptabilidade	Auto-learning (reward signals)	Estático (versão do framework)	Estático (regras)
Interpretabilidade	Médio (black-box MoE)	Alto (persona explícita)	Muito alto (regra visível)
Estado entre sessões	Sim (AgentDB persistente)	Não (filesystem apenas)	N/A
Ideal para	Tarefas complexas, ambíguas, alta paralelização	Processo de desenvolvimento estruturado	Transformações determinísticas

4.5 Recomendações DRY para o Hub

O Hub **NÃO deve reimplementar** os seguintes padrões — deve referenciá-los e integrá-los:

- Learning Loop 4-fases (RETRIEVE→JUDGE→DISTILL→CONSOLIDATE):** Adotar do ruflo como primitivo de L8 Memory. Implementação via AgentDB/ReasoningBank pode ser consumida via MCP tools.
- Persona-to-Phase mapping table:** Extrair do BMAD como esquema de gating declarativo de baixo custo. Útil como fallback quando contexto é insuficiente para routing semântico.
- Behavioral Trust Scoring formula:** $0,4 \times \text{success} + 0,2 \times \text{uptime} + 0,2 \times \text{threat} + 0,2 \times \text{integrity}$ do ruflo é diretamente reutilizável para o Hub avaliar confiabilidade de sub-agentes externos.

4. **Queen → Topology Selection:** O padrão de selecionar topologia (hierárquica vs. mesh vs. adaptativa) com base no tipo de tarefa é um meta-routing reutilizável — o Hub pode implementá-lo como primeira camada do gating antes de despachar para L3 experts.

5. Análise Comparativa — Ruflo vs. BMAD

5.1 Posicionamento

Dimensão	Ruflo	BMAD
Natureza	Infrastructure-heavy swarm runtime	Persona-driven agile methodology
Abstração	Meta-harness (baixo nível de infra)	Framework de processo (alto nível de método)
Curva de adoção	Alta — requer entender plugins, MCP, federation	Baixa — <code>npx bmad-method install</code> + conversa
Overhead operacional	Alto (daemon, MCP server, 27 hooks, workers)	Baixo (apenas arquivos no projeto)
Memória persistente	Sim (AgentDB, HNSW, SQLite)	Não (filesystem + contexto de sessão)
Multi-provider	Sim (Claude, GPT, Gemini, Ollama, Cohere)	Primariamente Claude Code / Cursor
Federation	Sim (zero-trust, mTLS)	Não
Security runtime	Forte (AIDefence, CVE, PII)	Fraco (sem runtime security)
Stars momentum	Muito alto (~62k, ex <code>claude-flow</code>)	Muito alto (~50k, crescendo rapidamente)
Licença	MIT	MIT

5.2 Quando Orquestração Pesada é Over-Kill

Ruflo traz overhead significativo que não se justifica em todos os cenários:

Contextos onde ruflo é over-kill: - Projeto solo, budget limitado de tokens: os 27 hooks + daemon + MCP server adicionam latência e custo sem ganho - Tarefa simples e bem-definida (ex: “refatore esta função”): não precisa de swarm, Queen, ou consensus protocol - Prova de conceito ou protótipo: o tempo de setup (`npx ruflo init` + configuração de federation) supera o valor entregue - Ambiente sem persistência (CI efêmero): AgentDB e ReasoningBank perdem valor sem sessões longas - Equipe sem expertise em TypeScript/Node: 35 plugins com dependências npm são difíceis de auditar

Contextos onde BMAD é over-kill: - Tarefa única sem processo de planning (ex: debug de bug específico) - Ambiente que já tem orquestração de processo (e.g., Linear + Jira) — BMAD duplicaria artefatos - Contexto onde memória entre sessões é crítica — BMAD não persiste aprendizado

Regra prática para o Hub: Usar BMAD (ou padrão equivalente de persona handoff) quando o gating pode ser simbólico e o projeto tem fases claras de SDLC; reservar ruflo (ou seu learning loop) quando há necessidade de memória adaptativa cross-session, paralelização real de sub-agentes ou security runtime.

5.3 L3 sobre os Substratos — Dependências de Camada

L3 (Amplifier) não é autossuficiente. Ambos os experts dependem implícita ou explicitamente de:

Camada	Dependência em Ruflor	Dependência em BMAD
L8 — Memory	AgentDB HNSW, ReasoningBank (explícito e profundo)	Filesystem de artefatos (superficial)
L9 — Observability	ruflor-observability (structured logs, traces, metrics); cost-tracker	Nenhum nativo; depende do AI IDE
L0 — Guardrails	AIDefence, ruflor-security-audit, input validation	Nenhum nativo — delega ao IDE/LLM

Implicação para o Hub: Um HUB que usar ruflor como L3 expert DEVE provisionar L8 (AgentDB ou equivalente) e L9 (observability stack) para que o sistema funcione. BMAD pode ser integrado mais levemente mas entrega menos valor sem L8.

6. Matriz de Comparação

Expert	Layer	Role	Licença	Stars (2026-06-27)	Escopo	Harness	MCP?	Git-flow	S-SDLC	Multi-tenant	Memory	Observability	Security
ruvnet/ruflor	L3	Amplifier	Swarm MIT	~61.700	Infra de orques-tração multi-agente, Claude Web UI + multi-provider	Claude Code (plu-gin + CLI), Codex, Claude Web UI	Sim — MCP server primário, 314 tools	git-branch, git-worktree, ruflor-jujutsu	Alto (CVE, PII, AIDefence, HIPAA/SOC2/GDPR)	Médio (fed-era-tion zero-trust sem multi-tenant explícito)	Sim — AgentDB HNSW (+ Dea) + son-ing-Bank SQLite	Sim — ruflor-observability (logs/traces/metrics) + cost-pipeline tracker	Forte: AIDefence, PII pipeline
bmad-code-org/BMAD-METHOD	L3	Amplifier	Process MIT	~49.600	Metodologia ágil AI-driven, Claude Code / Cur-sor / Gem-sini / ChatGPTs	Claude Code (pri-mary), Cur-sor, Gem-sini, ChatGPTs	Sim — via .claude-plugin (plu-gin manifest)	git-repo (husky hooks, GitHub tions)	Médio (Test Ar-chitect, TEA, sem run-time secu-rity nativo)	Baixo (single-project/session) de artefatos ape-nas, sem vetor	Nao (del-ega ao AI IDE)	Nao (del-ega ao AI IDE)	Fraco: sem run-time secu-rity; MIT + trade-mark; supply-chain npm [UN-VERI-FIED SCA]

Expert	Layer	Role	Licença	Stars (2026-06-27)	Escopo	Harness	MCP?	Git-flow	S-SDLC	Multi-tenant	Memory	Observability	Security
ruvnet/agent-flow	Agentic Amplifier	Incubador de features ruflo	Nea	~762	66 agentes MCP tools, SONA+alpha	Claude Code (via MCP)	Sim — 213 tools	N/A	Baixo	Baixo	Sim — AgentDB alpha + Reasoning-Bank SQLite	Parcial	SEM LICENÇA — não usar em produção
parruda/swarm	Swarm Amplifier	Ruby orchestration SDK + memory	MIT	~1.700	Automatizador geral, a Ruby ecosys-tem, single-process	Agnotista (v2 re-move process Claude Code)	Nao	git-repo (GitHub Actions CI)	Baixo	Baixo	Sim — Swarm-Memory FAISS + ONNX	Parcial (cost tracking, structured logging)	Médio: MIT, CVE/PII scanner
smtg-ai/claude-squad	L3 Amplifier	Multi-agent terminal manager	AGPL-3.0	~7.500	Gerenciador de processos AI em paralelo (tmux + worktrees)	Claude Code, Codex, Gemini, Aider	Nao	git-worktree (nativo design)	Baixo	Baixo	Nao	Nao	Médio: AGPL-3.0 (copy-left risco comercial); sem runtime security

7. Fontes

1. GitHub — ruvnet/ruflo README (observado 2026-06-27): <https://github.com/ruvnet/ruflo>
2. GitHub — ruvnet/ruflo full README (file salvo, 855 linhas, lido via subagent): dados de stars, plugins, architecture, Queen topology, federation, security
3. GitHub — bmad-code-org/BMAD-METHOD (observado 2026-06-27): <https://github.com/bmad-code-org/BMAD-METHOD> — stars 49.600
4. GitHub raw — bmad-code-org/BMAD-METHOD/main/README.md: <https://raw.githubusercontent.com/bmad-code-org/BMAD-METHOD/main/README.md> — conteúdo completo do README
5. GitHub raw — bmad-code-org/BMAD-METHOD/main/LICENSE (verificado 2026-06-27): <https://raw.githubusercontent.com/bmad-code-org/BMAD-METHOD/main/LICENSE> — confirma MIT; refuta NOASSERTION
6. GitHub raw — bmad-code-org/BMAD-METHOD/main/AGENTS.md: <https://raw.githubusercontent.com/bmad-code-org/BMAD-METHOD/main/AGENTS.md>
7. GitHub — bmad-code-org/BMAD-METHOD/blob/main/bmad-modules.yaml (observado 2026-06-27): lista oficial de módulos (TEA, BMB, CIS, BMGD, WDS, automator)

8. GitHub — ruvnet/agent-flow (observado 2026-06-27, full file 2769 linhas): <https://github.com/ruvnet/agent-flow> — stars 697 (página), 762 (from file); licença ausente; 66 agentes, 213 tools, SONA, AgentDB alpha
9. GitHub — parruda/swarm (observado 2026-06-27): <https://github.com/parruda/swarm> — stars 1.700; MIT; SwarmSDK v2, SwarmMemory, SwarmCLI; Ruby 98,8%
10. GitHub — smtg-ai/claude-squad (observado 2026-06-27): <https://github.com/smtg-ai/claude-squad> — stars 7.500; AGPL-3.0; Go; tmux + git worktrees
11. docs.bmad-method.org (tentativa de fetch — retornou vazio; conteúdo obtido via README e bmad-modules.yaml): <https://docs.bmad-method.org>
12. ruvnet/ruflo releases page (indireta via README): v3.14.3 como latest Jun 27 2026; 1.547 releases totais
13. ruvnet/ruflo AgentDB benchmark (self-reported, UNVERIFIED independent audit): ~1,9x speedup N=20k, recall@10 ~0,99
14. ruvnet/ruflo routing accuracy claim (self-reported, UNVERIFIED): “89% accuracy” intelligent routing
15. ruvnet/ruflo benchmark vs LangGraph/AutoGen/CrewAI (self-reported, UNVERIFIED): <https://gist.github.com>

Documento gerado por pipeline de pesquisa multi-agente (maos research harness) Observação primária: 2026-06-27 | Próxima revisão recomendada: 2026-09-27 (TTL 90 dias) PROV: agent-moe-2026/20260627-01d-amplifier.md | fase=research | status=FRESH

Parte III — Fase 2: N-Tree / MoE Routing Graph

Phase 2 — Recursive N-Tree + MoE Routing Graph

Título: Agentic MoE 2026 — N-Tree recursiva dos experts INCLUDED, modelada como **grafo de roteamento Mixture-of-Experts** (hub = gating network). **Fase:** 2 — N-Tree / MoE (SÍNTESE; sem nova pesquisa web — fundamentada nos arquivos 00 + 01a-d). **Observado:** 2026-06-27 (estrelas = ordem-de-magnitude datada do RUN 0; licenças/identidades canonicalizadas no arquivo 00). **Nota de idioma:** prosa em pt-BR; identificadores, comandos, nomes de repositório e tags de mecanismo em en-US. **Depende de:** 20260627-00-canonicalization.md + 20260627-01a-substrates.md + 01b-knowledge.md + 01c-pipeline.md + 01d-amplifier.md. **Escopo do span:** os ~26 experts INCLUDED + a BRIDGE graphiti (27 nós). NEIGHBOR/BASELINE são referenciados apenas quando uma aresta os exige, mas a árvore propriamente dita abrange só os INCLUDED.

0. Modelo MoE — o hub como GATING NETWORK

O Hub (multi-agent-os) é o **gating network**: recebe um intent do operador e decide **quais experts ativar e em que ordem**. Cada expert é um **nó tagueado por (layer, role)**. Diferente de um MoE neural (gating denso aprendido), este gating é **simbólico + estratificado**:

- **ALWAYS-ON SUBSTRATES (L0 · L8 · L9)** — não competem pela tarefa; **envelopam** toda rota. São ativados incondicionalmente (densamente), como um “residual” sempre somado.
- **TOP-K ESPARSO no pipeline (L1→L2→L6→L7) + KNOWLEDGE (L4·L5)** — o gating escolhe **um** expert por slot (top-1 dentro da camada), porque os concorrentes intra-camada são mutuamente exclusivos (dois specs, dois injetores de instrução, dois studios não convergem).
- **L3 AMPLIFIER** — gating de **segundo nível** (meta-routing): quando ligado, ruflo/bmad re-roteiam para os próprios sub-experts. É **opcional** e caro; só dispara em alta ambiguidade/paralelização.

Spine de roteamento (uma frase): substrate-check (L0 guardrails + L8 memory recall + L9 trace-on) → KNOWLEDGE (L4 grafo de codebase / L5 PKM) → L1 spec → L2 workflow (TDD/worktree) → L6 design → L7 slides, com **L3 como amplificador opcional** que pode envolver L1-L7 quando a tarefa exige swarm/persona-handoff.

Eixos de análise por nó: **Axis1 IDENTITY** · **Axis2 PURPOSE** · **Axis3 FLOW-TOPOLOGY** · **Axis4 SHARING-MECHANISM (por aresta)** · **Axis5 RELATION (por aresta)**.

1. N-TREE ANINHADA (experts INCLUDED por role/layer)

Legenda compacta por folha: [ID] Axis1 ident · PURPOSE Axis2 · TOP0 Axis3 · MECH Axis4 (mecanismos nativos do nó) · REL Axis5 (relação dominante no grafo). '= ordem-de-magnitude datada 2026-06-27 (RUN 0).[!]' = flag de licença/segurança do arquivo 00.

HUB = GATING NETWORK (multi-agent-os)

A. ALWAYS-ON SUBSTRATES (densos · envelopam toda rota)

L0 – GUARDRAILS (como o agente pensa/edita ANTES de agir)

karpthy-claude-md (multica-ai/andrej-karpthy-skills)

- Axis1: arquivo único de princípios; CLAUDE.md/skill; ~154–183K; **license=NONE [!]** (README diz MIT, sem LICENSE → all-rights-reserved); maturidade-de-conteúdo alta / engenharia baixa.

- PURPOSE: forçar deliberação (Think-Before-Coding, Simplicity-First, Surgical-Changes, Goal-Driven); persona = senior-engineer cético; read-only sobre o agente.

- TOP0: nó-folha distributivo/outward; injeção horizontal (todo harness que lê CLAUDE.md h

- MECH: {filesystem, git-repo, rules, hooks(se plugin)}.

- REL: complementar a tudo (política de baseline); sobrepõe parcialmente base (ambos tocam base (ChristopherKahler/base) – BASE = Builder's Automated State Engine

- Axis1: "AI builder OS"; npm @chrisai/base; 86–87; **license=null [!]**; nascente, single maintainer. (Nó aparece também no cluster Kahler em L2 como substrato de memória/knowledge-graph.)

- PURPOSE: anti-staleness do workspace (drift score, PSMM, Data Surfaces); injeta contexto only sobre projetos, read-write sobre .base/.

- TOP0: hub-and-spoke local (BASE MCP, 20 tools, 5 módulos); inward (lê projetos) + outward

- MECH: {MCP, filesystem, git-repo, hooks, rules, memory}.

- REL: **INCOMPATÍVEL com ECC** (dois gerenciadores de CLAUDE.md → hooks em duplicata; o pr claude detecta esse sprawl).

L8 – MEMORY (estado persistente entre sessões)

mem0 (mem0ai/mem0) – **DEFAULT de memória**

- Axis1: "universal memory layer"; ~58K; **Apache-2.0**; YC S24; benchmark reproduzível (m benchmarks) – razão de ser default vs MemPalace excluído.

- PURPOSE: memória plugável agnóstica de harness (User/Session/Agent); add/search; persona

- TOP0: hub central de memória; bi (write na captura / read no recall); multi-tenant.

- MECH: {MCP, memory, filesystem, git-repo, rules, TCP}.

- REL: competitivo com cognee/letta pelo slot "memória default"; complementar a graphiti (e letta (letta-ai/letta, ex-MemGPT)

- Axis1: runtime de agente stateful; ~23.5K; **Apache-2.0**; origem MemGPT/Berkeley; gover

- PURPOSE: memória OS-tiered self-editing como peça central de um **runtime completo** (Let

- TOP0: boundary-expert (L8+L1 runtime); central mas com lock-in (usar a memória = usar o agente).

- MECH: {memory, webhooks, TCP, git-repo, filesystem} (MCP não-primário).

- REL: **incompatível na prática como "backend plugável" (é harness rival); competitivo c

cognee (topoteretes/cognee) – graph-native control-plane
 · Axis1: "open-source AI memory platform"; ~23.3K; **Apache-2.0**; v1.2.2; unified Postgres.
 · PURPOSE: memória graph-native (remember/recall/forget/improve) + ontologia; "Company Br
 · TOP0: hub-and-spoke (cognee-mcp, HTTP/SSE/stdio); bi; multi-backend graph+vector.
 · MECH: {MCP, memory, filesystem, git-repo, hooks, rules, TCP}.
 · REL: competitivo com mem0 (slot default) e com graphiti (ambos graph-native); complementar a L9.

L9 – OBSERVABILITY (visibilidade humana sobre o comportamento)
 langfuse (langfuse/langfuse) – **DEFAULT de observability**
 · Axis1: LLM0ps OTel-native; ~28K; **NOASSERTION [!]** (MIT-core + pastas `ee/` proprietárias – único não-Apache-puro entre defaults); YC W23; região HIPAA.
 · PURPOSE: tracing + evals + prompt-mgmt + datasets + playground; persona = engenheiro de tenant por project.
 · TOP0: sink central; inward (ingere spans OTLP de qualquer rota); cross-cutting horizontal.
 · MECH: {MCP, webhooks, filesystem, git-repo, rules, TCP, hooks}.
 · REL: complementar a TUDO (cross-cut); alimentado por openllmetry (NEIGHBOR); par default

B. KNOWLEDGE (contexto de codebase L4 / PKM L5 – consultados antes de especificar)

L4 – Codebase Intelligence
 graphify (safishamsi/graphify)
 · Axis1: knowledge graph queryable; ~52.7K; **MIT**; branch v8; CVEs corrigidos.
 · PURPOSE: indexar repo 1x → grafo persistente (token-efficiency ~71x); multimodal (code/S radius.
 · TOP0: build-once hub + MCP query server; inward (lê filesystem) → outward (serve contexto tenant.
 · MECH: {MCP, filesystem, git-repo, git-PR, hooks, memory(grafo), TCP}.
 · REL: complementar a understand-anything (query vs teaching); **ponte L4→L5** via export C understand-anything (Egonex-AI/Understand-Anything)
 · Axis1: knowledge graph interativo/educativo; ~55–66K; **MIT**; migrou Lum1104→Egonex-AI.
 · PURPOSE: onboarding + exploração visual ("graphs that teach"); /understand-diff, /understand-domain; persona = novo dev/PM.
 · TOP0: pipeline multi-agente (8+ agents) → dashboard; inward; **sem MCP server** (gap vs g
 · MECH: {filesystem, git-repo, git-PR, memory(JSON graph), hooks(parcial), TCP(dashboard)
 · REL: complementar a graphify (roda 1º p/ compreender; graphify serve durante o build).

L5 – PKM (Personal Knowledge Management)

obsidian-skills (kepano/obsidian-skills)
 · Axis1: 5 agent skills oficiais do Obsidian (CEO kepano); 35K; **MIT**; agentskills.io sp
 · PURPOSE: ensinar o agente a operar vaults Obsidian corretamente (wikilinks, Bases, Canva
 · TOP0: nó-folha terminal; outward (escreve vault); o vault É a memória de longo prazo.
 · MECH: {filesystem, memory(vault), git-repo, rules(skill spec)}.
 · REL: complementar/dependente de graphify (consome export Obsidian → loop L4↔L5).

C. BUILD PIPELINE (ordenado L1→L2→L6→L7 – top-1 esparso por slot)

L1 – Spec (intenção → artefato executável ANTES do código)
 spec-kit (github/spec-kit)
 · Axis1: SDD da GitHub; 106K; **MIT**; CLI `specify`; "30+ agents".
 · PURPOSE: constitution→specify→plan→tasks→implement; phase-gated, template-rich, compliance-ready; persona = arquiteto de spec.

- **TOP0**: pipeline linear gated; outward (escreve specs/ + branch + Issues + guia PR); inflexão
- **MECH**: {filesystem, git-branch, git-repo, ticket/backlog(GitHub Issues), git-PR, rules, hooks}.
- **REL**: ****mutuamente exclusivo com OpenSpec**** (dois modelos de spec no mesmo repo não convergem)
 - openspec (Fission-AI/OpenSpec)
 - **Axis1**: SDD leve; 51K; ****MIT****; npm; "25+ tools"; telemetria anônima opt-out.
 - **PURPOSE**: propose→explore→apply→archive; git-agnóstico (change-folders); brownfield-first/iterativo.
- **TOP0**: linear sem gates rígidos; outward (pastas openspec/changes/); "update any artifact"
- **MECH**: {filesystem, git-repo, rules, MCP(referenciado), memory(spec-as-memory)}.
- **REL**: ****competitivo/incompatível com spec-kit****; ortogonal-mas-concorrente a gsd-core (3 SSOTs de spec).
 - gsd-core (open-gsd/gsd-core) – fork SEGURO
 - **Axis1**: SDD anti-context-rot; ~4.2K; ****MIT****; branch next; ****sucessor seguro do GSD original**** (pull \$GSD) [!]**.
 - **PURPOSE**: Discuss→Plan→Execute→Verify→Ship via subagentes de contexto-fresco; STATE.md/CONTEXT.md cross-sessão.
 - **TOP0**: thin-orchestrator → subagentes frescos (A2A-like); verify-before-ship; cria PR.
 - **MECH**: {filesystem, git-branch, git-PR, git-worktree, memory, rules, A2A(subagent-orchestration)}.
 - **REL**: ****antagonismo filosófico explícito com PAUL**** (subagentes vs in-session); [!] npm ORIGINAL vivo/perigoso.

L2 – Workflow (como o agente se COMPORTA construindo – always-on)

- superpowers (obra/superpowers)
- **Axis1**: metodologia SDLC via skills; ~147K; ****MIT****; plugin; menor footprint do trio.
 - **PURPOSE**: brainstorm→worktrees→plans→subagent-dev→TDD→code-review→finish-branch; persona = engenheiro disciplinado TDD.
 - **TOP0**: 7 fases skill-disparadas; ****injeta por ativação de skill**** (sem hooks/edição de CLAUDE.md)
 - **MECH**: {filesystem, git-repo, git-worktree, git-branch, git-PR, rules(skill-preamble)}.
 - **REL**: ****colisão de instrução com gstack/ECC**** (escolher um); o mais "convivente".
 - gstack (garrytan/gstack)
 - **Axis1**: "virtual engineering team" 23 personas; 101K; ****MIT****; autor Garry Tan (YC).
 - **PURPOSE**: Think→Ship com especialistas por papel (/cso OWASP+STRIDE, /qa browser real, /pair-agent cross-vendor).
 - **TOP0**: sprint de personas; ****reescreve CLAUDE.md**** (`## gstack`+`## Skill routing`) + auto-update horário; A2A via /pair-agent.
 - **MECH**: {filesystem, git-repo, git-worktree, git-branch, git-PR, rules(CLAUDE.md), memory(GBrain), MCP(GBrain), browser, A2A, TCP(ngrok)}.
 - **REL**: ****incompatível com ECC**** (contenção de CLAUDE.md; gstack ****bane `mcp\_\_claude-in-chrome`**** que outros esperam).
 - ECC (affaan-m/ECC)
 - **Axis1**: harness operator system; ~187K (README diz 182K+) [!]; ****MIT**** (+Pro \$19/seat); tr
 - **PURPOSE**: otimizar perf/confiabilidade do harness (232 skills, instincts, memory, security first); persona = power-user.
 - **TOP0**: ****o mais invasivo**** – CLAUDE.md+AGENTS.md + rules always-on + SessionStart(≤8000 chars) + Pre/PostToolUse/Stop hooks + instincts.
 - **MECH**: {MCP(~14), filesystem, git-worktree, git-branch, git-PR, ticket/backlog(parcial: GitHub/Linear), memory(hooks+SQLite+MCP), rules(always-on), hooks, browser}.
 - **REL**: ****incompatível com base**** (dois gerenciadores de CLAUDE.md) ****e com gstack**** (política o README do ECC alerta "Duplicate hooks file detected").
 - cli-anything (HKUDS/CLI-Anything)

- Axis1: gera CLIs agent-native a partir do código-fonte; ~42–44K; ****Apache-2.0 [ok]**** (LICENSE confirmada – era load-bearing).
- PURPOSE: "making ALL software agent-native"; pipeline 7-fases; 40+ CLIs + CLI-Hub; requer código-fonte.
- TOPO: tool-builder; inward (lê source) → outward (escreve setup.py + instala no PATH).
- MECH: {filesystem, git-repo(lê source), skill-registry(SKILL.md), CLI/REPL/JSON}.
- REL: complementar (não colide com instrução always-on); supply-chain a auditar (exec gerado de repo não-confiável).
- paul (ChristopherKahler/paul) – Plan-Apply-Unify
 - Axis1: dev in-session; 976; ****MIT****; npm paul-framework; 26 commands.
 - PURPOSE: PLAN(AC Given/When/Then)→APPLY(anti-rationalization)→UNIFY(closure obrigatório session over subagent sprawl).
 - TOPO: loop in-session; subagentes só p/ discovery; MCP via carl.
 - MECH: {filesystem(.paul/), rules(via carl), memory(STATE.md), git-repo}.
 - REL: ****antagonismo explícito com gsd-core**** (doc PAUL-VS-GSD.md); dependente de carl (runtime).
- carl (ChristopherKahler/carl) – Context Augmentation & Reinforcement Layer
 - Axis1: regras dinâmicas JIT; 353; ****MIT****; npm carl-core; v2 (carl.json).
 - PURPOSE: hook escaneia cada prompt → carrega só domain-rules que casam; substrato de runtime dos outros Kahler.
 - TOPO: hook por-prompt (always-on); o único Kahler com MCP server + hook engine reais.
 - MECH: {hooks, MCP(carl-mcp, 15 tools), rules, memory, filesystem}.
 - REL: ****empilha 2 hook-engines se co-instalado com ECC****; dependência runtime de paul/seed
- seed (ChristopherKahler/seed) – typed project incubator
 - Axis1: incubadora de ideação; 288; ****license=null [!]**** (claim MIT só na prosa; LICENSE a risco legal de reuso, o maior do cluster).
 - PURPOSE: ideação tipada → PLANNING.md; /seed graduate → apps/{name}/ + git; markdown puro
 - TOPO: nó-fonte (genesis); outward (graduate = git init + registra no grafo BASE → PAUL he
 - MECH: {filesystem(markdown), git-repo(graduate), memory/session}.
 - REL: dependente de base (registro) + paul (launch); encadeia SEED→BASE→CARL→PAUL.

L6 – Generative Design (UI/protótipo a partir de prompt)

open-design (nexu-io/open-design)

- Axis1: studio local-first multi-surface; ~48.7K; ****Apache-2.0****; lineage frontend-design; [!] versão drift (brief 19/71 SUPERSEDED).
- PURPOSE: web/mobile/deck/img/vídeo; HTML/PDF/PPTX/MP4; BYOK 12 CLIs; persona = senior de
- TOPO: web+daemon delegando ao seu CLI; ****bypassPermissions total [!]****; ACP (Devin/Hermes
- MECH: {filesystem, git(só clone+flags), memory(SQLite .od/), rules(anti-slop), hooks(flag), ACP}.
- REL: ****BORROWS de open-codesign**** (citado: "UX north star"); ocupam a mesma vaga → escolhe

open-codesign (OpenCoworkAI/open-codesign)

- Axis1: studio desktop enxuto; ~7K; ****MIT****; Electron self-contained; v0.2.0; SBOM+SHA256SUMS.
- PURPOSE: prompt→protótipo HTML/JSX/slides/UI-kit; 5 exports; BYOK 20+; loop ****permission**
- TOPO: Electron self-contained (bundla pi-ai); GUI; sem delegação a CLIs externas.
- MECH: {filesystem(tools permissionados), memory(DESIGN.md+JSONL), rules(SKILL.md)}.
- REL: ****é a fonte de UX**** que open-design empresta; competem pela mesma vaga (lean+seguro v
- impeccable (pbakaus/impeccable)
- Axis1: LAYER de qualidade de design (não studio); ~30K; ****Apache-2.0****; lineage frontend-design.
- PURPOSE: combater "AI slop"; 1 skill, 23 commands, ****41 regras determinísticas key-free****; melhora UI que o host gera.
- TOPO: layer cross-harness (11 harnesses, a mais ampla); não produz artefato próprio; lin
- MECH: {filesystem(PRODUCT.md/DESIGN.md), git-repo}.
- REL: ****complementar a open-design/open-codesign**** (layer vs studio – não competem); menor risco do sublayer.

L7 – Slides-as-code (narrativa)
 slidev (slidevjs/slidev)
 · Axis1: framework slides-as-code; 47K; **MIT ©2021 antfu**;
 predata a era dos agentes (nativa).
 · PURPOSE: Markdown→deck (Vue/Vite/UnoCSS/Shiki); presenter mode/LaTeX/Mermaid; PDF/PNG/first.
 · TOPO: build tool standalone; sem agent loop / sem harness coverage.
 · MECH: {filesystem(Markdown+projeto), git-repo}.
 · REL: convergem só no OUTPUT com frontend-slides (framework vs skill); sem canibalização.
 frontend-slides (zarazhangrui/frontend-slides)
 · Axis1: skill agêntico de slides; 19K; **MIT**;
 só Claude Code; progressive-disclosure SKILL.md.
 · PURPOSE: content→HTML deck zero-dep; UX "3 previews"; anti-slop (12 presets); export PDF + deploy Vercel.
 · TOPO: skill dentro do Claude Code (não server); herda o modelo do Claude.
 · MECH: {filesystem(HTML único+scripts), git-repo(plugin marketplace)}.
 · REL: complementar a slidev (tiers diferentes); destino natural do pipeline L6→L7.

D. AMPLIFIER (L3 – opcional · meta-routing que envolve L1–L7)
 ruflor (ruvnet/ruflor, ex claude-flow)
 · Axis1: meta-harness/swarm runtime; ~54.6–61.7K; **MIT**;
 1.547 releases; 314 MCP tools; CoT.
 · PURPOSE: "sistema nervoso" plug-and-play (100+ agentes, AgentDB HNSW, learning loop, federated trust);
 persona = infra engineer.
 · TOPO: gating de 2 níveis – Topology Selection (Queen-Raft/Mesh-Gossip/Adaptive-Byzantine) → Agent Dispatch (MoE attention + HNSW);
 inflexão = roteamento adaptativo aprendido.
 · MECH: {MCP, filesystem, git-repo, git-branch, git-worktree, memory(AgentDB+ReasoningBank)}.
 · REL: **dependente de L8+L9** (DEVE provisionar memory+observability); amplifica L1–L7;
 benchmarks self-reported [UNVERIFIED].
 bmad-method (bmad-code-org/BMAD-METHOD)
 · Axis1: metodologia ágil persona-driven; ~47.7–49.6K; **MIT** (LICENSE refuta NOASSERTION).
 · PURPOSE: processo (Analyst→PM→Architect→Dev→QA→DevOps) via persona handoff; "Party Mode" domain-adaptive.
 · TOPO: gating **simbólico/declarativo** (fase do SDLC → persona); stateless; sem swarm runtime.
 · MECH: {filesystem, git-repo, memory(artefatos), rules(CLAUDE.md), hooks(husky), MCP(via plugin/)}.
 · REL: complementar/sobreposto a L1 (gera PRD/ADR – duplica se já há spec-kit/Linear); amplificador leve (sem L8 nativo).

2. EDGE LIST — from → to | mecanismo (Axis4) | relação (Axis5) | evidência

Arestas agrupadas por classe. “Evidência” cita o arquivo Phase-1 de origem. Mecanismo = o canal pelo qual a aresta de fato flui.

2.1 Substrate → Pipeline (sempre-ligado envolve a esteira)

from → to	mecanismo (Axis4)	relação	evidência
karpathy-claude-md → (todo pipeline L1–L7)	rules, filesystem	complementar	“qualquer harness que leia CLAUDE.md/AGENTS.md herda os princípios” (01a)

from → to	mecanismo (Axis4)	relação	evidência
base → spec-kit/superpowers	MCP, hooks, memory	complementar	injeta contexto compacto via UserPromptSubmit/SessionStart antes do build (01a)
mem0 → L1/L2 (recall de preferências)	MCP, memory, TCP	dependente (substrato)	“memória default... add/search isolado por tenant” (01a)
cognee → L4/L2 (raciocínio sobre relações)	MCP, hooks, memory	complementar	hooks de lifecycle Claude Code (SessionStart...PreCompact) (01a)
langfuse → (toda rota L1-L7)	TCP(OTLP), webhooks, MCP	complementar (cross-cut)	“ingere qualquer app que emita spans OTLP”; default obs (01a)

2.2 Knowledge → Spec (consultar o codebase antes de especificar)

from → to	mecanismo (Axis4)	relação	evidência
graphify → spec-kit/openspec	MCP, filesystem	complementar/dependente	grafo pré-indexado serve contexto barato ao planejar (01b)
understand-anything → graphify	filesystem, memory	complementar	“Understand-Anything roda 1º (compreensão); Graphify serve durante o trabalho” (01b)
graphify → obsidian-skills	filesystem, git-repo	dependente	“export Obsidian Vault + JSON Canvas... ponte L4→L5 mais concreta” (01b)
graphify ↔ graphiti (BRIDGE)	memory, MCP	complementar	“graphify extrai estrutura estática → graphiti adiciona camada temporal” (01b)

2.3 Spec → Workflow (a spec governa o comportamento de execução)

from → to	mecanismo (Axis4)	relação	evidência
spec-kit → superpowers	filesystem, git-branch, rules	colaborativo (inter-camada)	spec executável → TDD/worktree executa o plano (01c síntese de pipeline)
openspec → superpowers	filesystem, rules	colaborativo	change-folder vira plano que o workflow implementa (01c)

from → to	mecanismo (Axis4)	relação	evidência
gsd-core → (própria execução)	A2A, git-worktree, git-PR	colaborativo intra-nó	“Execute em waves; cada executor recebe 200k tokens limpos” (01c)
seed → paul	filesystem, git-repo	dependente	“/seed launch gradua p/ repo git + inicializa PAUL headless” (01c)
carl → paul	hooks, MCP, rules	dependente	“hook do CARL injeta as 14 regras de loop-enforcement JIT” (01c)

2.4 Workflow → Design → Slides (a esteira de entrega)

from → to	mecanismo (Axis4)	relação	evidência
superpowers → open-design	filesystem, git-worktree	colaborativo (inter-camada)	“spec-kit gera spec → superpowers executa → open-design produz UI” (01c síntese)
open-design → impeccable	filesystem(DSIGN.md)	complementar (studio→layer)	impeccable “melhora a UI que o host gera”; ambos descendem de frontend-design (01c)
open-codesign → impeccable	filesystem(DSIGN.md)	complementar	mesma lineage DESIGN.md; layer audita o studio (01c)
open-design → frontend-slides	filesystem(HTML)	colaborativo	deck/slides como destino do design (01c, tabela L7)
frontend-slides → slidev	filesystem(Markdown/HTML)	complementar	“convergem só no formato de saída”; tiers framework/skill (01c)

2.5 Amplifier → tudo (L3 envolve L1-L7 quando ligado)

from → to	mecanismo (Axis4)	relação	evidência
ruflo → (L1-L7 sub-experts)	MCP(314), hooks(27), A2A	dependente (amplifica)	“Queen instancia sub-agentes como workers e coordena via AgentDB” (01d)
ruflo → mem0/cognee (L8)	MCP, memory	dependente	“HUB com ruflo DEVE provisionar L8 (AgentDB ou equivalente)” (01d §5.3)
ruflo → langfuse (L9)	TCP(OTLP), webhooks	dependente	ruflo-observability emite logs/traces/metrics (01d §5.3)

from → to	mecanismo (Axis4)	relação	evidência
bmad-method → spec-kit	filesystem, rules	complementar/sobreposto	PRD/ADR de BMAD duplicam artefatos se já há spec formal (01d §5.2)
bmad-method → (personas internas)	filesystem, rules	colaborativo intra-nó	gating fase→persona; “Party Mode” multi-persona (01d §4.2)

2.6 Memory / Observability cross-cuts (eixos ortogonais)

from → to	mecanismo (Axis4)	relação	evidência
mem0 ↔ langfuse	memory / TCP(OTLP)	complementar	“par default×default recomendado pelo RUN 0” (01a §L9)
graphiti → langfuse	TCP(OTLP)	complementar	“ambos têm OTEL embutido; graphiti emite traces que langfuse ingere” (01a)
cognee → (qualquer OTEL backend)	TCP(OTLP)	complementar	“cognee tem OTEL collector embutido” (01a)
letta → langfuse	webhooks, TCP(OTLP)	complementar	“OTEL embutido; webhooks nativos” (01a)

2.7 Arestas COMPETITIVAS / INCOMPATÍVEIS (negativas — não compor)

from ↔ to	mecanismo em disputa	relação	evidência
base ↔ ECC	rules(CLAUDE.md), hooks	incompatível	RUN 0: base “conflicts with other CLAUDE.md managers e.g. ECC”; /base:audit-claude detecta hooks duplicados (01a)
gstack ↔ ECC	rules(CLAUDE.md), hooks, MCP(browser)	incompatível	gstack bane mcp__claude-in-chrome__* que ECC espera; ambos editam CLAUDE.md; “Duplicate hooks file detected” (01c §b)
superpowers ↔ gstack ↔ ECC	rules(always-on)	competitivo (escolher um)	“três camadas de instrução ‘mandatory’ = precedência imprevisível... escolher um, não empilhar” (01c §b)

from ↔ to	mecanismo em disputa	relação	evidência
carl ↔ ECC	hooks(per-prompt)	incompatível	“CARL+ECC empilhariam dois hook-engines” (01c §b nota)
mem0 ↔ letta ↔ cognee	memory backend (pgvector/Postgres)	competitivo	tabela “vs Competitors”; letta = “High (must use Letta)” lock-in (01a §L8)
spec-kit ↔ openspec	filesystem(specs/)	incompatível (mutuamente exclusivos)	“dois sistemas de spec concorrentes... escolha mutuamente exclusiva” (01c §a)
paul ↔ gsd-core	filesystem(STATE.md)	incompatível (filosofia oposta)	doc PAUL-VS-GSD.md: in-session+UNIFY vs subagentes+closure-opcional (01c §c)
open-design ↔ open-codesign	mesma vaga (studio L6)	competitivo (lineage)	“UX north star and our closest peer”; mesmo job → um só (01c §d)
open-design ↔ open-codesign (segurança)	bypassPermissions vs permissionado	incompatível de postura	open-design = bypass total; open-codesign = loop permissionado (01c §d)

3. MERMAID — grafo de roteamento MoE estratificado

Validade: testado mentalmente — IDs alfanuméricos (L0a, L1a...), labels entre aspas, sem caracteres soltos. Setas tracejadas (-.->) marcam arestas negativas (incompat/competição). Subgraphs com direction explícito. As setas que partem de um nó para um subgraph (HUB -.-> SUB, L9a -.-> PIPE, L3a -.-> PIPE) são sintaxe Mermaid válida (subgraph como destino). **Mermaid: VÁLIDO.**

4. RECEITAS DE COMPOSIÇÃO (substrate-first)

Stack ordenado: **sempre L0+L8+L9 primeiro** (substratos), depois a pipeline. “Risco/HITL” = ponto onde se recomenda gate humano.

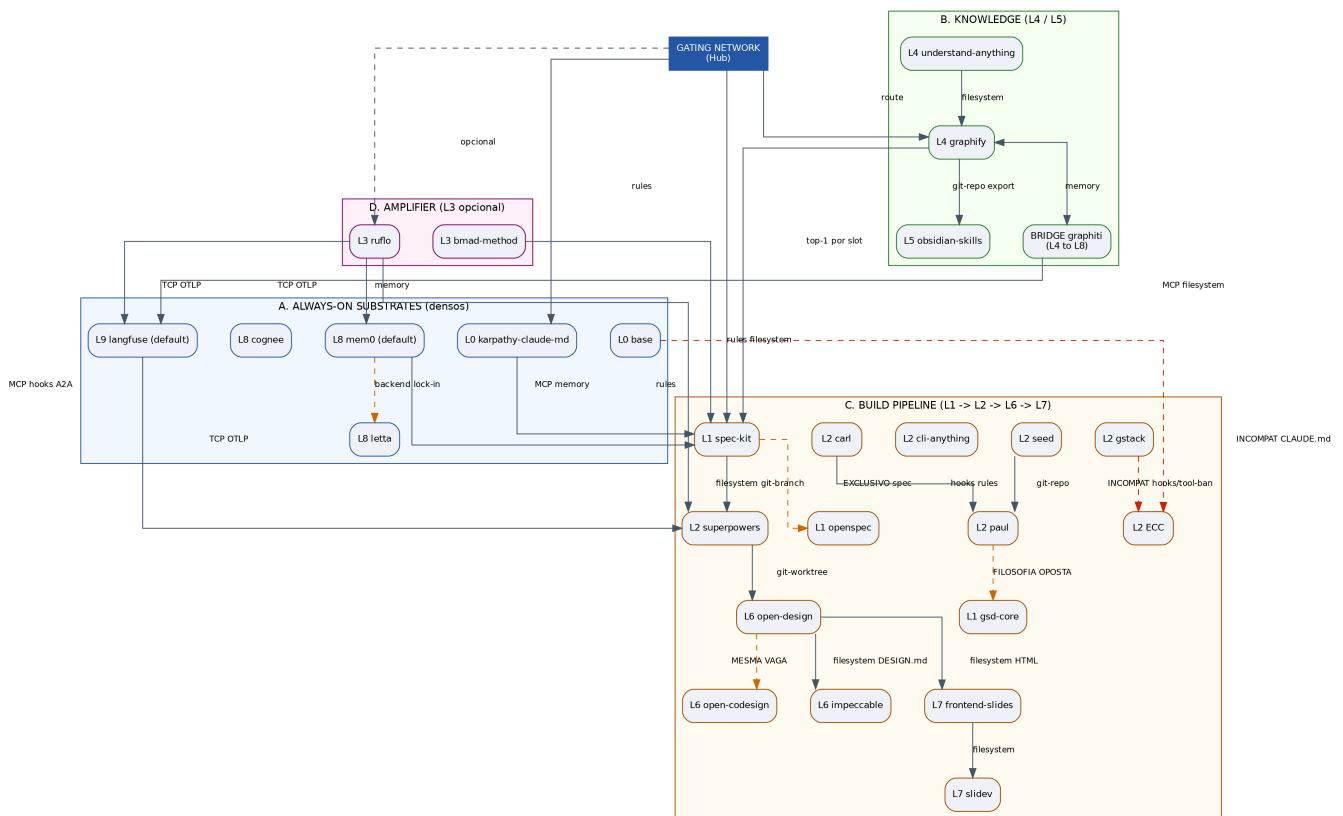


Figure 1: Diagrama: Grafo de Roteamento MoE Estratificado (N-Tree)

Caso de uso	Stack ordenado (L0+L8+L9 → pipeline)	Por quê	Risco / HITL
Nova ideia → MVP	karpathy-claude-md (L0) + mem0 (L8) + langfuse (L9) → seed (ideação) → spec-kit ou openspec (L1) → superpowers (L2, TDD) → open-codesign (L6) → frontend-slides (L7)	Substrato leve; seed estrutura a ideia; spec-kit p/ rigor (ou openspec p/ leve); superpowers garante TDD; studio enxuto+seguro p/ UI; deck p/ pitch.	Gate humano na escolha spec-kit vs openspec (mutuamente exclusivos) e na aprovação do plano (clarify).
Refactor de legado	karpathy-claude-md + base (L0) + cognee (L8, graph) + langfuse (L9) → understand-anything (L4, onboarding) → graphify (L4, contexto) → openspec (L1, brownfield) → superpowers (L2, worktree+review)	Legado exige compreensão antes de mudar (understand→graphify); cognee mapeia relações; openspec é brownfield-first; review gates obrigatórios.	HITL no code-review com severidade (superpowers “Critical issues block progress”); cuidado p/ não commitar grafo com segredos.

Caso de uso	Stack ordenado (L0+L8+L9 → pipeline)	Por quê	Risco / HITL
Feature enterprise	karpthy-claude-md (L0) + graphiti (L8 temporal) + langfuse (L9, HIPAA) → graphify (L4) → spec-kit (L1, constitution+compliance) → gstack ou superpowers (L2) → open-design (L6) → slidev (L7)	spec-kit é o canônico p/ governança/constitution; graphiti dá auditoria temporal; langfuse região HIPAA; gstack traz /cso (OWASP+STRIDE) + coverage gate.	HITL nos gates de compliance (/speckit.analyze, /cso); não empilhar gstack+ECC .
Deploy de produção	base (L0, drift-check) + mem0 (L8) + langfuse (L9, evals-gate) → gsd-core (L1, verify-before-ship) ou gstack (/ship coverage audit) → impeccable (L6, linter key-free em CI)	gsd-core e gstack têm verify-before-ship + PR auto ; impeccable é linter determinístico plugável em CI; langfuse evals como gate.	HITL na aprovação do PR e no canary; [!] se gsd-core: usar SÓ o fork open-gsd (npm original é perigoso).
Onboarding de codebase	karpthy-claude-md (L0) + cognee (L8) + langfuse (L9) → understand-anything (L4, dashboard+tour) → graphify (L4, export Obsidian) → obsidian-skills (L5, vault navegável)	understand-anything é “graphs that teach” (onboarding); graphify exporta vault; obsidian-skills deixa o agente enriquecer o vault → loop L4↔L5.	Risco baixo (read-mostly). HITL opcional na revisão do tour/domínio extraído.
Sessão longa com memória	karpthy-claude-md (L0) + letta (L8 runtime self-edit) ou mem0+graphiti (L8 layer+temporal) + langfuse (L9) → ECC ou gsd-core (L2/L1, memory cross-sessão) → ruflo (L3, learning loop opcional)	Sessão longa precisa de memória que sobrevive a sessões : letta (self-edit) ou mem0+graphiti; ECC/gsd-core persistem STATE/SQLite; ruflo adiciona RETRIEVE→JUDGE→DISTILL→CONSOLIDATE.	[!] letta = lock-in de runtime (não é backend plugável); ruflo é over-kill se solo/budget-limitado (27 hooks+daemon). HITL na decisão letta-vs-mem0.

5. INCOMPATIBILIDADES DE CAMADA

Colunas: par | camada | natureza do conflito | evidência | mitigação.

par	camada	natureza do conflito	evidência	mitigação
base vs ECC	L0 (gerência de CLAUDE.md)	Dois gerenciadores de CLAUDE.md/.claude → hooks em duplicata + injeção de contexto redundante (estrutural, não só sobreposição). Colisão de camada de instrução always-on: três workflows “mandatory” → precedência imprevisível; erosão de contexto empilhada (ECC documenta 200k→~70k).	RUN 0: base “conflicts with other CLAUDE.md managers e.g. ECC”; o próprio /base:audit-claude detecta “duplicated hooks running twice” (01a).	Escolher UM orquestrador de CLAUDE.md por workspace; rodar audit-claude p/ detectar sprawl.
superpowers vs gstack vs ECC	L2 (instruction-layer)	Colisão de camada de instrução always-on: três workflows “mandatory” → precedência imprevisível; erosão de contexto empilhada (ECC documenta 200k→~70k).	Tabela de mecanismos de injeção (01c §b): superpowers=skill-preamble; gstack=edita CLAUDE.md; ECC=CLAUDE.md+AGENTS.md+hooks+SessionSt	Escolher um, não empilhar. superpowers é o mais convivente (sem hooks/edição de CLAUDE.md).
gstack bane browser-MCP que ECC empacota	L2 (política de ferramenta)	gstack injeta no CLAUDE.md “Never use mcp__claude-in-chrome__*” enquanto ECC/outros esperam/empacotam tooling MCP de browser → conflito direto de política.	01c §b (ii): “gstack bane mcp__claude-in-chrome__* enquanto ECC e outros bundlam/esperam tooling MCP de browser”.	Não co-instalar; se gstack for o escolhido, remover stacks que dependem de browser-MCP.
mem0 vs letta vs cognee	L8 (memory backend / runtime)	Backends de memória concorrentes; letta = lock-in de runtime (“must use Letta”) vs mem0/cognee plugáveis; colisão de embeddings se compartilham pgvector/Postgres sem namespacing.	Tabela “vs Competitors” do agentmemory: Letta “High (must use Letta)” (01a §L8); “escolher UM substrato primário de memória por agente”.	Um substrato primário por agente; graphiti(temporal)+mem0(layer) é composição válida (eixos diferentes).

par	camada	natureza do conflito	evidência	mitigação
OpenSpec vs spec-kit	L1 (modelo de spec)	Modelos de spec mutuamente exclusivos: phase-gated .spec-ify/+specs/NNN/ vs git-agnóstico open-spec/changes/ → dois diretórios + duas convenções /speckit.* vs /opsx:.*	01c §a: “rodar ambos no mesmo repo gera dois sistemas de spec concorrentes... escolha mutuamente exclusiva”.	Escolher por contexto: spec-kit (enterprise/compliance/greenfield) · OpenSpec (brownfield iterativo leve).
open-design vs open-codesign	L6 (studio + postura de segurança)	Mesma vaga (alternativa OSS local-first a Claude Design) + postura de segurança incompatível : open-design = bypassPermissions total; open-codesign = loop permissionado.	01c §d: “UX north star and our closest peer”; open-design “bypassPermissions... builds não assinados” vs open-codesign loop permissionado+SBOM.	Um por job: open-codesign (lean+seguro+polido) · open-design (amplo+multi-harness, com sandbox extra).
paul vs gsd-core	L1/L2 (orquestração)	Heavy-orchestration em contexto de baixo risco + filosofias opostas: GSD = subagentes de contexto-fresco; PAUL = in-session + UNIFY obrigatório — não se compõem.	01c §c: doc PAUL-VS-GSD.md; “duas teses concorrentes sobre o mesmo mal”.	Escolher a filosofia; não rodar ru-flo/bmad/gsd pesados em tarefa solo/simples (over-kill — 01d §5.2).
carl vs ECC	L2 (hook engine)	Dois hook-engines per-prompt empilhados (carl-hook.py + Pre/PostToolUse do ECC) → dupla execução por prompt.	01c §b nota: “CARL+ECC empilhariam dois hook-engines”.	Não co-instalar; se ECC, dispensar carl (e vice-versa).

par	camada	natureza do conflito	evidência	mitigação
ruflo/bmad em contexto de baixo risco	L3 (amplifier overkill)	Orquestração pesada (27 hooks+daemon+MCP server / persona-overhead) sem ganho em tarefa solo, budget pequeno, CI efêmero, ou onde já há Linear/Jira.	01d §5.2: contextos onde ruflo e BMAD são “over-kill”.	Reservar L3 a alta-ambiguidade/paralelização/me cross-session; senão pular o amplificador.
langfuse ee/ non-OSS	L9 (licença)	Único default não-Apache-puro: MIT-core + pastas ee/ proprietárias (Enterprise Edition) — componentes não-livres.	01a §L9: “This repository is MIT licensed, except for the ee folders”; RUN 0 NOASSERTION.	Se exigência for OSS-puro: usar só o core MIT (sem ee/) ou trocar por openll-metry+backend Apache (opik).
seed / base license=null	L0/L2 (higiene legal)	LICENSE ausente na máquina (claim MIT só na prosa) → scanners tratam como all-rights-reserved (risco de reuso comercial).	RUN 0 §0.2 + 01c: “license: null na API... o maior risco de licença do sub-sub-grupo”.	Não redistribuir comercialmente sem clareza legal; preferir alternativas com LICENSE válida; pinar.
karpathy-claude-md license=NONE	L0 (licença)	README diz “MIT” mas sem arquivo LICENSE → não constitui concessão OSS (all-rights-reserved por padrão).	RUN 0 §0.2 + 01a: “Sem licença ≠ open-source para o filtro ‘state license’”.	Manter como política de baseline com flag; não tratar como OSS-permissivo até LICENSE válida.

Phase 2 · N-Tree/MoE · 27 nós (26 INCLUDED + BRIDGE graphiti) · observado 2026-06-27. Prosa pt-BR, identificadores en-US. Fundamentado em 00 + 01a-d; nenhuma nova pesquisa web. Estrelas/licenças = RUN 0 (ordem-de-magnitude datada). [UNVERIFIED] herdados: dígitos exatos de estrelas (superpowers/ECC/gsd-core/cli-anything via [sec]); string SPDX de phoenix (NEIGHBOR, fora do span); od mcp install de open-design; webhooks de ruflo; benchmarks self-reported de ruflo.

Parte IV — Fase 3: Orchestrator/Hub

Phase 3 — Orchestrator/Hub: the MoE Gating Network (middleware Agentic-Tool Orchestrator)

Título: Agentic MoE 2026 — desenho concreto do **HUB middleware** = a **gating network** Mixture-of-Experts que roteia um humano $\rightleftharpoons$ expert(s) dinamicamente selecionados, montada **sobre** os substratos always-on (L0 guardrails + L8 memory + L9 observability). **Fase:** 3 — Orchestrator/Hub (DESENHO; sem nova pesquisa web — fundamentado em 00 + 02, reusando padrões de 01a/01c/01d). **Observado:** 2026-06-27 (estrelas/licenças = ordem-de-magnitude datada do RUN 0; identidades canonicalizadas no arquivo 00). **Nota de idioma:** prosa em **pt-BR**; identificadores, comandos, nomes de repositório, YAML, fórmulas e tags de mecanismo em **en-US**. **Depende de:** 20260627-00-canonicalization.md (identidades/licenças/vereditos de segurança) + 20260627-02-ntree-moe.md (o grafo de roteamento MoE em que este HUB se apoia). **Reusa padrões** de 20260627-01a-substrates.md (defaults: mem0/graphiti/langfuse), 20260627-01c-pipeline.md (ECC/NanoClaw O(1) gating + colisão de instrução L2 + AgentShield) e 20260627-01d-amplifier.md (ruflo Queen→topology + Learning-Loop + behavioral-trust; BMAD persona→phase). **Princípio DRY:** LiteLLM (model gateway) e os MCP gateways são **primitivas de INFRA** — usadas, não reinventadas. O learning-loop, o trust-scoring e o O(1) routing são **importados** dos experts (não reimplementados).

(a) ARQUITETURA EM CAMADAS — o pipeline de gating sobre os substratos

O HUB é um **plano de controle (control plane) stateless** que materializa o gating network simbólico-estratificado descrito no arquivo 02 §0. Diferente de um MoE neural (gating denso aprendido), aqui o gating é **simbólico + estratificado**: os substratos (L0·L8·L9) **envelopam** densamente toda rota (residual sempre somado), enquanto a pipeline (L1→L2→L6→L7) e o knowledge (L4·L5) recebem **top-1 esparsos por slot**.

Pipeline de 5 estágios (cada estágio consome o anterior; substratos cross-cut todos):

1. **Intent Classification** — classifica o intent do operador em {task_family, scope, methodology, risk_hint}. Reusa o sinal de gating do **BMAD** (fase_do_SDLC → persona, 01d §4.2) como classificador simbólico barato de primeira linha; sobe para classificação semântica (embedding + lookup HNSW, padrão **ruflo**) só em alta ambiguidade.
2. **Multi-Criteria Scoring (CTS)** — pontua candidatos por **ISO × Eisenhower × risk × scope × methodology × reversibility** sob o rubric de 4 dimensões (Expert-fit / Authorization / Task-frame / Risk-frame). Detalhe em (c).
3. **Tool-Gating / Lazy-Schema-Loader** — atenção gateada sobre o **pool permanente de summaries** (≤ 60 tokens/tool); promove o schema completo só para os top-k tools gateados. Resolve o “MCP/Tools Tax”. Detalhe em (b).
4. **Model Router (Switchcraft-style)** — escolhe o modelo capaz mais barato por tool-call via **LiteLLM** (INFRA) + budgets por categoria. Detalhe em (g).
5. **Tool Registry** — a fonte-de-verdade declarativa (YAML por tool) que alimenta os estágios 2–4. Detalhe em (d).

Validade Mermaid: IDs alfanuméricos (0P,S1...S5,L0/L8/L9,KN,PIPE,AMP,GATE,WP); todos os labels entre aspas (parênteses e setas-texto escapados pelas aspas); subgraphs com direction explícito; setas para subgraph (L9 --> GATE, S4 --> GATE, GATE --> WP) são válidas. Aresta tracejada S2 -. -> S5 = leitura de scores. **Mermaid: VÁLIDO.**

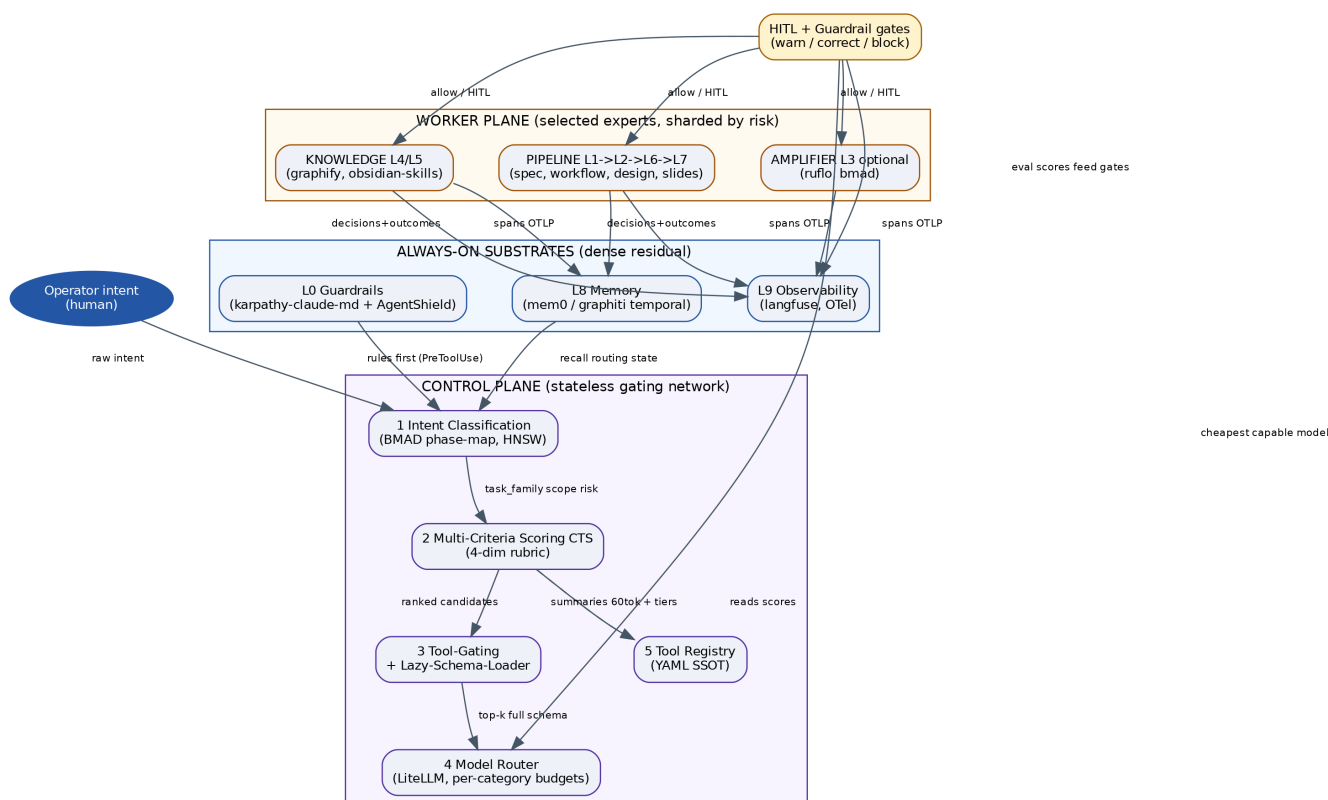


Figure 2: Diagrama: Control Plane — Stateless Gating Network

(b) TOOL ATTENTION / ISO — atenção gateada sobre um pool permanente de summaries

O problema: “MCP/Tools Tax” + context-fracture

Cada servidor MCP injeta o schema completo de **todas** as suas tools na janela a cada turno. O arquivo 01c documenta o sintoma no ECC: ~14 MCP servers + preâmbulo SessionStart de 8000 chars erodem a janela de **200k → ~70k tokens**. Em landscapes ricos (este HUB referencia 26 experts, e ECC sozinho expõe ~14 servers; rufo expõe **314 MCP tools**), o “Tools Tax” custa **10k-60k tokens/turn** só em descrições não-usadas. Acima de **~70% de utilização** da janela, o modelo sofre *context-fracture*: instruções always-on competem, precedência fica imprevisível (o mesmo achado da colisão L2 superpowers↔gstack↔ECC).

A solução: ISO (Information-Scent Optimization) — gated attention de 2 níveis

O HUB mantém um **pool permanente de summaries** — exatamente **um summary de ≤60 tokens por tool** (campo summary do registry, (d)). Esse pool é o **único** custo de contexto fixo. O schema completo (parâmetros, exemplos, JSONSchema) é **lazy-loaded** apenas para os top-k tools que passam o gate. Padrão idêntico ao **lazy-schema** do ToolSearch/progressive-disclosure e ao “skill hot-load” do **NanoClaw** (01c): a tool existe por *nome+scent* até ser *promovida* a schema completo.

Matematicamente é uma **atenção gateada** (sparse top-k softmax) sobre o pool de summaries:

```
# Para cada tool i no pool permanente (summary embeddings pré-computados):
score_iso(i)  = sim(q, summary_i)                # cosine(query, summary≤60tok)
gate(i)       = 1 if CTS(i) ≥ θ_layer AND hard_filters(i) = PASS else 0
               # CTS = score multi-critério de (c); hard_filters de (c)/(h)/(i)
weight(i)     = softmax_k( score_iso(i) · gate(i) ) # top-k mask
```

PROMOTE(i) = weight(i) > 0 # carrega schema completo só destes
context_cost = $\sum_i \text{len}(\text{summary}_i) + \sum_{\{\text{PROMOTE}(i)\}} \text{len}(\text{schema}_i)$
~60·N (fixo, barato) + k·|schema| (variável, top-k apenas)

- **Nível 1 (sempre):** todos os N summaries ≤60tok no contexto → custo fixo ~60·N tokens.
- **Nível 2 (gateado):** só os k tools com weight(i)>0 recebem schema completo. Tipicamente k ∈ [3,7] por slot de layer (top-1 esparsa na pipeline, top-k no knowledge).

Como isso mata o Tools Tax: em vez de $\sum |\text{schema}_i|$ (10k-60k tokens), paga-se 60·N + k·|schema|. Para N=80 tools e k=5, isso é ~4.8k + ~5k ≈ 10k no pior caso vs >40k sem gating — e mantém a utilização **abaixo de 70%**, evitando o context-fracture. O θ_{layer} é o threshold por camada (mais baixo para L0/L8/L9 que são densos/always-on; mais alto para L3 amplifier que é caro/opcional).

(c) MULTI-CRITERIA SCORING (CTS) — ISO × Eisenhower × risk × scope × methodology × reversibility

O CTS combina **seis critérios** sob o **rubric de gating de 4 dimensões** (mapeamento explícito abaixo). **Hard filters primeiro** (eliminam candidatos antes de qualquer soma ponderada — economia + segurança), depois a função ponderada ordena os sobreviventes.

Os 6 critérios → 4 dimensões do rubric

Critério (sinal)	Dimensão do rubric	O que mede	Fonte do padrão
ISO sim(q, summary)	Expert-fit	quão bem o scent da tool casa o intent	(b); HNSW lookup do ruflo
scope (file/module/repo/org)	Expert-fit	o tool opera no escopo certo?	grafo 02 (Axis2 PURPOSE)
methodology (greenfield/brownfield/compliance)	Task-frame	a metodologia casa a tarefa?	BMAD phase-map (01d)
Eisenhower (urgent×important)	Task-frame	priorização da tarefa	quadrante clássico
risk_tier (LOW/MED/HIGH)	Risk-frame	dano potencial da ação	AgentShield tiers (01c)
reversibility (reversible/irreversible)	Risk-frame	dá para desfazer?	RUN 0 precedents (00)
authorization (read-only/scoped/broad)	Authorization	o tool tem permissão p/ isto?	ECC autorização (01c)

Hard filters (executam ANTES da soma — qualquer falha = candidato eliminado)

HF1 security_status ∈ {EXCLUDED, FLAGGED} → REJECT (ex.: GSD original, MemPalace)
HF2 license ∈ {none, NOASSERTION} AND context = "commercial-redistribution" → REJECT (ex.: seed/base/karpathy)
HF3 authorization_required > authorization_granted → REJECT (escala de privilégio)
HF4 layer_incompatible(candidate, already_selected) → REJECT (ex.: spec-kit XOR openspec; base XOR ECC)
HF5 risk_tier = HIGH AND HITL_token ausente → HOLD (vai p/ gate humano, não roteia)
HF6 reversibility = irreversible AND approval ausente → HOLD (idem)

Função de scoring ponderada (sobreviventes dos hard filters)

CTS(i) = w_iso · iso(i) # Expert-fit
+ w_scope · scope_match(i) # Expert-fit

```

+ w_meth · methodology_match(i)          # Task-frame
+ w_eis   · eisenhower(i)                 # Task-frame
- w_risk  · risk_penalty(i)               # Risk-frame (PENALIDADE: subtrai)
- w_rev   · irreversibility_penalty(i)    # Risk-frame (PENALIDADE: subtrai)
+ w_trust · trust_score(i)                # Authorization (behavioral-
trust importado)

# trust_score importado VERBATIM do ruflo (01d §4.5), reusado sem reimplementar:
trust_score(i) = 0.4·success_rate + 0.2·uptime + 0.2·threat_clearance + 0.2·integrity

# Pesos default (operador pode sobrescrever por contexto; soma dos positivos = 1.0):
w_iso=0.30 w_scope=0.15 w_meth=0.15 w_eis=0.10 w_trust=0.30 # positivos
w_risk=0.50 w_rev=0.40 # penalidades (escala maior: segurança domi

```

Worked example — intent: “refatore o módulo de auth deste repo legado, sem quebrar a API pública”

Classificação (estágio 1): task_family=refactor, scope=module, methodology=brownfield, risk_hint=MED, reversibility=reversible (worktree), eisenhower=important_not_urgent.

Candidatos L1 (spec): **openspec** vs **spec-kit** (mutuamente exclusivos, HF4 deixa passar só na ausência do outro).

Sinal	openspec	spec-kit	nota
HF (filtros)	PASS	PASS	ambos MIT/clean
iso (Expert-fit)	0.88	0.55	“brownfield/iterativo” casa o intent
scope_match	1.0	1.0	ambos operam em module/repo
methodology_match	1.0	0.4	openspec é brownfield-first; spec-kit é greenfield/compliance
eisenhower	0.6	0.6	igual
risk_penalty	0.2	0.2	igual (escrita de spec, baixo)
irreversibility_penalty	0.0	0.0	reversível
trust_score	0.85	0.85	ambos maduros

```

CTS(openspec) = 0.30·0.88 + 0.15·1.0 + 0.15·1.0 + 0.10·0.6 + 0.30·0.85 - 0.50·0.2 - 0.40·0.0
               = 0.264 + 0.15 + 0.15 + 0.06 + 0.255 - 0.10 - 0.0 = 0.779
CTS(spec-kit) = 0.30·0.55 + 0.15·1.0 + 0.15·0.4 + 0.10·0.6 + 0.30·0.85 - 0.50·0.2 - 0.40·0.0
               = 0.165 + 0.15 + 0.06 + 0.06 + 0.255 - 0.10 - 0.0 = 0.590

```

Decisão: openspec (0.779 > 0.590) vence o slot L1 — coerente com a receita “Refactor de legado” do arquivo 02 §4. O gating é **interpretável** (o delta vem inteiramente de iso + methodology_match, ou seja, “brownfield casa melhor”).

(d) TOOL REGISTRY SCHEMA — YAML por tool (SSOT do gating)

Cada tool é um nó do grafo 02 declarado como um registro YAML. Campos obrigatórios: layer, role, summary (≤60 tok — alimenta o pool ISO de (b)), preconditions, risk_tier, authorization, harness_coverage, security_status, mechanisms[]. Quatro exemplos preenchidos a partir dos dados **verificados** (00/01a/01c):

```

# ---- L8 MEMORY (default) ----
- id: mem0
  repo: mem0ai/mem0
  layer: L8
  role: memory-substrate-default
  summary: "Universal pluggable memory layer; add/search User/Session/Agent state; tenant-iso

```

```

preconditions: ["substrate slot L8 empty OR mem0 already primary", "store reachable (lib/se
risk_tier: LOW
authorization: read-write # scoped to memory store, per-tenant
harness_coverage: ["claude-code", "codex", "cursor", "windsurf", "opencode", "openclaw"]
security_status: INCLUDED # Apache-2.0; clean
mechanisms: ["MCP", "memory", "filesystem", "git-repo", "rules", "TCP"]

# ---- L0 GUARDRAIL / SECURITY (ECC's AgentShield as the L0 scanner) ----
- id: ecc-agentshield
  repo: affaan-m/ECC # AgentShield ships within ECC + standalone ecc-agentshield
  layer: L0
  role: supply-chain-security-scanner
  summary: "Static-analysis gate: 102 rules across secrets(14 patterns), permission audit, ho
  preconditions: ["runs in PreToolUse / CI before any write", "target = CLAUDE.md/settings/MC
  risk_tier: LOW # the scanner itself is read-only over config
  authorization: read-only
  harness_coverage: ["claude-code", "codex", "cursor", "opencode", "gemini", "zed", "copilot"]
  security_status: INCLUDED # MIT; note ECC-vs-base/gstack incompatibility (HF4)
  mechanisms: ["rules", "hooks", "filesystem", "git-repo"]

# ---- L6 GENERATIVE DESIGN (broad studio, security caveat) ----
- id: open-design
  repo: nexu-io/open-design
  layer: L6
  role: generative-design-studio
  summary: "Local-first multi-surface studio (web/mobile/deck/img/video); HTML/PDF/PPTX/MP4;
  preconditions: ["L2 produced a plan/spec to design from", "EXTRA sandbox+egress policy appl
  risk_tier: HIGH # bypassPermissions total -> needs HITL per (h)/(i)
  authorization: broad # full filesystem; unsigned builds
  harness_coverage: ["claude-code", "devin", "hermes", "kimi", "kiro"] # ACP
  security_status: INCLUDED # Apache-2.0; flag: bypassPermissions (sandbox required)
  mechanisms: ["filesystem", "git", "memory", "rules", "hooks", "ACP"]

# ---- L3 AMPLIFIER (optional meta-routing; requires L8+L9) ----
- id: ruflo
  repo: ruvnet/ruflo
  layer: L3
  role: swarm-runtime-meta-router
  summary: "Optional 2-level meta-router: Queen topology select then MoE+HNSW agent dispatch;
  preconditions: ["high ambiguity OR real parallelization OR cross-session memory needed", "L
  risk_tier: MED # broad orchestration surface; 27 hooks
  authorization: broad
  harness_coverage: ["claude-code", "codex"]
  security_status: INCLUDED # MIT; benchmarks self-reported [UNVERIFIED]
  mechanisms: ["MCP", "filesystem", "git-worktree", "git-branch", "memory", "hooks", "TCP"]

```

O registry inteiro (26 INCLUDED + BRIDGE graphiti) é gerado a partir da N-Tree do 02 §1; só os 4 acima estão expandidos por brevidade. O campo summary é deliberadamente **≤60 tokens** porque é o que vai no pool permanente de (b).

(e) MEMORY SUBSTRATE — como o HUB persiste/recorda estado e decisões de roteamento

O HUB é stateless **no plano de controle**, mas sua *memória de roteamento* (quais experts foram escolhidos, com que score, com que outcome) é persistente em L8. Modelo **tiered L0..Ln** (espelha

o Multi-Level Memory do mem0 + o ReasoningBank do ruflo):

Tier	Conteúdo	Backend default	TTL/escopo
L0-mem (working)	janela atual: candidatos, scores CTS, gate decisions	em-memória (efêmero)	turno
L1-mem (session)	trajetória de rotas da sessão; {intent, chosen_expert, CTS, outcome}	mem0 (Session)	sessão
L2-mem (operator)	preferências do operador (pesos CTS sobrescritos, experts banidos)	mem0 (User)	persistente
L3-mem (episodic)	ReasoningBank de rotas: {task, input, output, reward[0-1], success, tokensUsed, latencyMs}	mem0 + opção graphiti	persistente, com decay
Ln-mem (temporal)	fatos de roteamento com janela de validade ("expert X regrediu desde release Y")	graphiti (bi-temporal, BRIDGE L4↔L8)	invalidação, não delete

Consolidação = reuso do Learning-Loop do ruflo (DRY, 01d §4.5) — o HUB **não reimplementa**; consome via MCP o ciclo de 4 fases como primitivo de L8:

RETRIEVE (top-k MMR sobre L3-mem: rotas passadas similares ao intent atual)
→ JUDGE (LLM-as-judge pontua se a rota escolhida foi boa – alimenta L9 evals)
→ DISTILL (extraí a regra: "para task_family=X em scope=Y, expert=Z venceu")
→ CONSOLIDATE(dedup + pruning; promove regra a fast-path de gating; aplica decay)

Resultado: rotas frequentes viram **fast-paths** (cache de gating) — na próxima vez que task_family=refactor ∧ scope=module ∧ methodology=brownfield aparecer, o HUB recupera "openspec venceu (0.779)" sem re-pontuar tudo. A **temporalidade graphiti** garante que, se um expert for rebaixado (ex.: novo CVE, regressão), o fato antigo é *invalidado* em vez de apagado — preservando auditoria.

(f) OBSERVABILITY SUBSTRATE — todo route traçado OTel-native; evals alimentam os gates

Cada rota emite spans **OTLP** para **langfuse** (default, 01a §L9) — LangfuseSpanProcessor no @opentelemetry/sdk-node. Trace hierárquico por rota:

```
span: route.{trace_id}
  span: intent.classify      attrs={task_family, scope, methodology, risk}
  span: cts.score            attrs={candidates[], scores[], hard_filters_rejected[]}
  span: gate.tool            attrs={promoted_tools[], context_cost_tokens}
  span: model.route          attrs={model_chosen, category, budget_remaining}
  span: hitl.gate            attrs={verdict: allow|warn|block, approval_id?}
  span: expert.exec.{layer}  attrs={tool_id, latency_ms, tokens, error?, output_taint}
```

Eval scores → gates warn→correct→cure (cadeia de remediação): o LLM-as-judge do langfuse (ou Opik se evals-as-gate forte) pontua cada rota; o score realimenta os gates de (h):

eval_score ≥ θ_ok -> CURE (rota validada; consolida no Learning-Loop como sucesso)
θ_warn ≤ score < θ_ok -> WARN (loga aviso; sugere correção; não bloqueia)
score < θ_warn -> CORRECT (força replan: volta ao estágio 2 com penalidade no expert)
unsafe_call detectado -> BLOCK (curto-circuita; ver (i))

Telemetria por camada: cost/latency/error por layer e por tool_id (atributos do span). Isso fecha o loop com o Model Router de (g) — budgets por categoria são medidos aqui — e com a memória de (e) — tokensUsed/latencyMs do ReasoningBank vêm destes spans.

[!] **Caveat de licença (00/01a):** langfuse é **NOASSERTION** — MIT-core **exceto as pastas ee/** (Enterprise Edition, proprietárias). Se o requisito for OSS-puro, usar só o core MIT (sem ee/) ou trocar por **openllmetry + backend Apache** (opik). É o único default não-Apache-puro.

(g) MODEL ROUTER (Switchcraft-style) — modelo capaz mais barato por tool-call via LiteLLM

O Model Router escolhe o **modelo mais barato que ainda é capaz** para cada tool-call, via **LiteLLM** como gateway (INFRA — NOASSERTION, “Phase 3 only” no 00; usado, não reinventado). Cada categoria de tool tem um budget e um tier mínimo de capacidade:

Tool category	Min capability tier	Default model (cheapest capable)	Per-category budget
classify/route (estágio 1)	small	haiku-class / SLM local	very-low (alto volume)
summary/scent (pool ISO)	small	SLM local (embeddings)	minimal (pré-computado)
spec/plan (L1)	medium	sonnet-class	medium
code/workflow (L2)	high	sonnet/opus-class (TDD precisa rigor)	high
design (L6)	medium	sonnet-class (BYOK do studio)	medium
judge/eval (L9)	medium	sonnet-class (consistência de juiz)	low-medium
amplifier (L3)	high	opus-class + multi-provider (ruflo)	high (gated, opcional)

Roteamento (sketch): `model = litellm.pick(category.min_tier, budget_remaining, latency_slo)`. Se o budget da categoria estourar, o router **rebaixa** (downgrade) o tier ou enfileira para HITL — fechando o loop com a telemetria de custo de (f). Multi-provider (Claude/GPT/Gemini/Ollama) é nativo do LiteLLM; o HUB só declara a política, não a mecânica.

(h) GUARDRAIL & HITL GATES — warn → correct → block; L0 sempre primeiro

L0 guardrails executam SEMPRE primeiro (PreToolUse — antes de qualquer escrita), espelhando o ECC (rules always-on + PreToolUse hooks, 01c). A cadeia de severidade é progressiva:

WARN -> aviso no trace + sugestão; a rota PROSSEGUE (ex.: tool fora do scope ideal)
CORRECT -> força replan no estágio 2 com penalidade no expert (ex.: eval_score baixo de (f))
BLOCK -> curto-circuita a rota; nada é executado (ex.: hard filter HF1/HF3 de (c), ou unsafe-call de (i))

HITL obrigatório (a rota fica em HOLD, não roteia, até aprovação humana) quando: - risk_tier = HIGH (ex.: open-design com bypassPermissions; deploy de produção), **ou** - reversibility = irreversible (HF6), **ou** - escolha mutuamente-exclusiva de alto impacto (ex.: spec-kit vs openspec — a receita do 02 marca isso como gate humano).

Mapa para o rubric de 4 dimensões (qual dimensão dispara qual gate):

Dimensão (rubric)	Gate disparado	Exemplo
Authorization	BLOCK (HF3)	tool pede privilégio > concedido
Risk-frame	HITL HOLD (HF5/HF6)	risk=HIGH ou irreversível
Task-frame	CORRECT	metodologia errada → replan
Expert-fit	WARN	scope/scent subótimo, mas seguro

(i) SECURITY MODEL — supply-chain, tainting, sandbox/egress, failover

Reusa **AgentShield** (ECC, 01c) como o scanner L0 de supply-chain (registry: ecc-agentshield), executado em **PreToolUse** e em CI (exit 2 gateia builds):

Supply-chain scanning (PreToolUse style — bloqueia antes de executar)

- BLOCK ``git ... --no-verify`` # burla hooks de segurança/pre-commit
- DETECT secrets por regex (14 patterns do AgentShield): ``sk-...``, ``ghp-...``, ``AKIA...`` -
- > BLOCK + redact
- PREVENT exfiltração de CLAUDE.md/.claude/ via tool-output ou egress não-autorizado
- PROFILE risco de MCP server (hook-injection analysis + MCP risk profiling, 102 rules)

Tool-output tainting (modelo de 3 estados)

Toda saída de tool carrega um rótulo de taint propagado pelo grafo de rotas:

trusted – fonte interna verificada (filesystem do repo, mem0 do próprio tenant)
untrusted – fonte externa (web fetch, repo de terceiro, output de sub-agente externo)
derived – produto de mistura: `derived = trusted * untrusted` (herda untrusted: pessimista)

untrusted/derived **não podem** alimentar um tool de authorization=broad sem passar por um gate de sanitização (regra: *taint não escala privilégio*). Isso é o análogo de roteamento do AIDefence/PII-pipeline do ruflor.

Sandbox + egress por tool

Cada tool roda no seu **sandbox** com **egress policy** declarada (campo derivável do registry). Tools `risk_tier=HIGH/authorization=broad` (ex.: open-design bypassPermissions) exigem sandbox reforçado + egress allowlist. **Failover**: se um expert falhar/timeout, o HUB cai para o segundo-colocado do CTS (degradação graciosa), nunca para um expert FLAGGED.

Amarração aos precedentes do RUN 0 (00 \$0.1) — por que o gate é necessário

- **GSD rug-pull** (\$GSD Solana, saque ~US\$500K, npm abandonado): `security_status=EXCLUDED` via **HF1**; o sucessor seguro open-gsd/gsd-core é o único roteável. O HUB **nunca** roteia para o pacote original.
- **MemPalace** (star-manip + benchmark não-auditável): EXCLUDED/FLAGGED via **HF1**; mem0 (benchmark reproduzível) é o default justamente por auditabilidade.
- **License=none** (karpathy/base/seed): **HF2** rejeita redistribuição comercial; mantidos como política de baseline com flag, nunca tratados como OSS-permissivo.

(j) COMPOSITION RECIPES embutidas no roteamento (presets, substrate-first)

As **6 receitas** do arquivo 02 §4 viram **routing presets** — atalhos de gating que pré-selecionam o stack (sempre L0+L8+L9 primeiro). Cada preset é um conjunto de pesos CTS + experts pré-gateados:

Preset (routing)	Stack (substrate-first → pipeline)	Gate / HITL embutido
new-idea-mvp	karpathy(L0)+mem0(L8)+langfuse(L9) → seed → spec-kit openspec(L1) → superpowers(L2) → open-codesign(L6) → frontend-slides(L7)	HITL na escolha spec-kit↔openspec (HF4) + aprovação do plano (clarify)
legacy-refactor	karpathy+base(L0)+cognee(L8)+langfuse(L9) → understand-anything(L4) → graphify(L4) → openspec(L1) → superpowers(L2)	HITL no review com severidade; cuidado p/ não commitar grafo com segredos
enterprise-feature	karpathy(L0)+graphiti(L8 temporal)+langfuse(L9 HIPAA) → graphify(L4) → spec-kit(L1 constitution) → gstack superpowers(L2) → open-design(L6) → slidev(L7)	HITL nos gates de compliance (/speckit.analyze, /cso); não empilhar gstack+ECC (HF4)
prod-deploy	base(L0 drift)+mem0(L8)+langfuse(L9 evals-gate) → gsd-core(L1 verify-before-ship) gstack(/ship) → impeccable(L6 linter CI)	HITL na aprovação do PR + canary; [!] gsd-core = SÓ o fork open-gsd (HF1)
codebase-onboarding	karpathy(L0)+cognee(L8)+langfuse(L9) → understand-anything(L4 tour) → graphify(L4 export) → obsidian-skills(L5 vault)	HITL baixo (read-mostly); HITL opcional na revisão do domínio extraído
long-session-memory	karpathy(L0)+ letta mem0+graphiti(L8)+langfuse(L9) → ECC gsd-core(L1/L2) → ruflo(L3 learning-loop, opcional)	[!] letta = lock-in de runtime; ruflo = overkill se solo (HF: skip amplifier)

\| = escolha mutuamente-exclusiva resolvida pelo CTS/HITL no momento da rota. Os presets são **defaults**, não trilhos: o operador pode sobrescrever pesos, e o Learning-Loop (e) ajusta os fast-paths com o uso real.

(k) EVALUATION — métricas, plano de testes, deployment/scaling

Métricas

Métrica	Definição	Alvo
routing precision	rotas corretas / rotas escolhidas	↑ (>0.9)
routing recall	experts certos ativados / experts que deveriam	↑
task completion rate	tarefas concluídas sem replan manual	↑

Métrica	Definição	Alvo
unsafe-call rate	tool-calls bloqueados por (i) / total	↓ (→0)
token cost / route	$60 \cdot N + \sum \text{schema promovido} + \text{exec (de (f))}$	↓ vs baseline sem gating
latency p50/p95 / route	tempo do estágio 1 ao expert.exec	↓
approval-fatigue	nº de HITL holds / sessão	↓ (gates calibrados, não ruidoso)

Plano de testes

- **6 task families** × **{low, med, high} risk** (matriz 6×3 = 18 células): new-idea, refactor, enterprise-feature, deploy, onboarding, long-session (espelham os presets de (j)).
- **A/B with vs without gating** — medir token cost e context-fracture (% acima de 70% de utilização) com e sem o ISO de (b).
- **Security injection tests** — payloads --no-verify, secrets sk-/ghp-/AKIA, tentativa de exfiltração de CLAUDE.md, MCP malicioso (deve disparar BLOCK/HF1); taint-escalation (untrusted→broad deve ser barrado).
- **Approval-fatigue calibration** — varrer thresholds $\theta_{\text{warn}}/\theta_{\text{ok}}$ e contar HITL holds para achar o ponto onde segurança não vira ruído.

Deployment / scaling

- **Control plane stateless** (estágios 1-5): escala horizontal trivial (sem estado entre requests; o estado vive em L8/L9). Réplicas atrás de um LB.
- **Worker pools shardados por risk_tier**: pool LOW (read-mostly, sem sandbox pesado, alta densidade), pool MED, pool HIGH (sandbox reforçado + egress allowlist + HITL obrigatório de (h)/(i)). Sharding por risco isola blast-radius e dimensiona custo por tier.
- **Substrates como serviços compartilhados**: mem0/graphiti/langfuse rodam como backends multi-tenant (já são, por design) — um por workspace/tenant, não por rota.

(I) REFERENCE IMPLEMENTATION — MCP-server sketch + sequência de roteamento

Tool names expostos pelo HUB (MCP server maos-hub)

hub.route(intent → rota), hub.score(debug CTS), hub.registry.get (lê YAML), hub.gate.approve (HITL token), hub.trace(link langfuse da rota).

Loop de gating dirigido pelo registry (~40 linhas, ilustrativo)

```
# maos-hub: registry-driven gating loop (illustrative – LiteLLM + MCP gateways are INFRA, use
def hub_route(intent: str, ctx: Context) -> Route:
    # ESTÁGIO 0 – substratos always-on (dense residual)
    guardrails_preflight(ctx) # L0: AgentShield PreToolUse (block --no-
    prior = mem0.recall(intent, ctx.tenant) # L8: fast-path lookup (Learning-Loop con
    span = langfuse.start_span("route", intent=intent) # L9: OTeI trace begins

    if prior and prior.confidence > FASTPATH_THETA: # consolidated fast-path hit -> skip full
        return finalize(prior.route, span)

    # ESTÁGIO 1 – intent classification (BMAD phase-map cheap; HNSW only if ambiguous)
    frame = classify(intent) # {task_family, scope, methodology, risk,

    # ESTÁGIO 2 – CTS: hard filters FIRST, then weighted score
    pool = registry.load() # YAML SSOT (d); summaries already ≤60 to
```

```

survivors = [t for t in pool if hard_filters(t, frame, ctx) == "PASS"] # HF1..HF6 (c)
holds     = [t for t in pool if hard_filters(t, frame, ctx) == "HOLD"] # risk=HIGH / in
scored    = sorted(survivors, key=lambda t: cts(t, frame, ctx), reverse=True) # (c) weigh

# ESTÁGIO 3 – tool-gating + lazy schema (ISO attention (b)): promote only top-k
gated = topk_by_layer(scored, k_per_layer=K) # top-1 pipeline slot, top-k knowledge
for t in gated: t.schema = registry.load_full_schema(t.id) # lazy promote (NanoClaw hot)

# ESTÁGIO 4 – model router (cheapest capable via LiteLLM, per-category budget (g))
for t in gated: t.model = litellm.pick(t.category.min_tier, ctx.budget, ctx.slo)

# GATE – warn/correct/block + HITL (h); taint check (i)
verdict = gate(gated, holds, span) # BLOCK/HF -> raise; HOLD -> await hub.gate
if verdict.needs_hitl: return await_hitl(holds, span)

# EXEC – selected experts (sharded worker pool by risk); spans + outcomes back to L8/L9
route = execute(gated, ctx, span) # worker plane
langfuse.judge(route, span) # L9 eval -> warn/correct/cure (f)
mem0.consolidate(intent, route, span.outcome) # L8 Learning-Loop: RETRIEVE->JUDGE->DISTILL
return finalize(route, span)

```

Routing SEQUENCE — brownfield refactor (graphify → openspec → ECC com HITL gate)

Sequência de Roteamento: Brownfield Refactor (graphify → openspec → ECC com HITL gate)

#	De	Para	Mensagem / Ação
1	Operator	Hub	"refactor auth module (legacy), keep public API"
2	Hub	L0 AgentShield	preflight (secrets / -no-verify / exfil)
3	L0 AgentShield	Hub	PASS (clean)
4	Hub	L8 mem0	recall similar routes
5	L8 mem0	Hub	no fast-path (score CTS)
6	Hub	L9 langfuse	start trace route.{id}
7	Hub	L4 graphify	gate L4 (build-once graph, blast-radius)
8	L4 graphify	Hub	code graph + impact set
9	Hub	L1 openspec	gate L1 (CTS 0.779 > spec-kit 0.590)
10	L1 openspec	Hub	change-folder (brownfield spec)
11	Hub	Hub	CTS picks L2 = ECC (memory cross-session)
12	Hub	Operator	HITL gate (risk=MED on broad-auth write)
13	Operator	Hub	approve (hub.gate.approve)
14	Hub	L2 ECC	execute refactor in worktree (HIGH-risk shard)
15	L2 ECC	L9 langfuse	spans (latency, tokens, taint=trusted)
16	L9 langfuse	Hub	eval_score >= theta_ok (CURE)
17	Hub	L8 mem0	consolidate route (Learning-Loop)
18	Hub	Operator	done + trace link

Validade Mermaid: sequenceDiagram com actor + participant ... as "label" (aliases alfanuméricos Hub,L0,L8,L9,Gfy,0sp,ECC); mensagens com ->> (sync) e -->> (return); labels e textos de mensagem livres de caracteres que quebrem o parser. **Mermaid: VÁLIDO.**

(m) PRAGMATISM FILTER — alternativas over-engineered REJEITADAS (explícito)

O HUB é deliberadamente **simbólico-estratificado e barato**, não um sistema distribuído especulativo. Rejeitado, com motivo:

Rejeitado	Por quê (não roteável / não construído)
Blockchain audit trail	A proveniência temporal do graphiti (bi-temporal, invalidação) já é trilha de auditoria nativa; blockchain adiciona custo/latência sem ganho. O precedente \$GSD (00) é exatamente o <i>anti-padrão</i> de “crypto em dev-tooling”.
Quantum / quantum-routing	Zero base no landscape; o gating é top-k softmax sobre embeddings — clássico, O(N) no pool. Nenhuma tarefa exige.
Self-replicating agent swarms	ruflo (L3) já cobre swarm gateado e opcional ; auto-replicação sem bound é justamente o que os gates HF5/HF6 + HITL existem para impedir. Over-kill documentado (01d §5.2).
Distributed design sem use-case real	Control plane stateless + worker pools shardados (k) cobrem o scaling real. “Distribuído” só onde há tenant multi-máquina — a federação do ruflo é opcional, não default.
“L10 gateway layer”	Os MCP gateways e o LiteLLM são INFRA (00 marca litellm “Phase 3 only / infra-primitive), não uma camada de experts. Promovê-los a “L10” inventaria uma camada que o grafo 02 não tem — viola DRY e a estratificação canônica (L0–L9).
Gating neural denso aprendido	O arquivo 02 §0 é explícito: este gating é simbólico + estratificado , não MoE neural. Treinar um roteador denso é custo sem retorno quando BMAD-phase + ISO + CTS já são interpretáveis e auditáveis.

Phase 3 · Orchestrator/Hub · MoE gating network sobre substratos always-on (L0·L8·L9) · observado 2026-06-27. Prosa pt-BR, identificadores/YAML/fórmulas en-US. Depende de 00 + 02; reusa padrões de 01a/01c/01d. Infra (LiteLLM + MCP gateways) usada, não reinventada. [UNVERIFIED] herdados do upstream: dígitos exatos de estrelas (ECC/superpowers/etc. via [sec]); benchmarks self-reported do ruflo (“89% routing accuracy”, AgentDB speedup); webhooks do ruflo; string SPDX exata de phoenix. [UNVERIFIED] desta fase (extrapolação de design): valores numéricos default dos pesos CTS, thresholds $\theta_{\text{layer}}/\theta_{\text{warn}}/\theta_{\text{ok}}$ e a contagem de tokens do pool ISO são propostas de engenharia a calibrar empiricamente no plano de testes (k), não medições.

Parte V — Síntese Final

Agentic MoE 2026 — RELATÓRIO FINAL (Síntese das Fases 0-3)

Fase: 4 — Final Report (SÍNTESE; sem nova pesquisa web — funde + reconcilia os arquivos existentes). **Observado:** 2026-06-27 (toda observação de estrela é datada deste dia;

valores = ordem-de-magnitude, não precisão). **Nota de idioma:** prosa em **pt-BR**; identificadores, comandos, nomes de repositório, fórmulas e tags de mecanismo em **en-US**. **Funde:** 20260627-00-canonicalization.md + 20260627-01a-substrates.md + 01b-knowledge.md + 01c-pipeline.md + 01d-amplifier.md + 20260627-02-ntree-moe.md + 20260627-03-orchestrator-hub.md. **Disciplina epistêmica:** flags [UNVERIFIED] / [sec] herdadas dos upstreams são **preservadas**, não apagadas. Estrelas extraordinárias são reportadas como valor de API datado, **sem editorializar**, com aviso de magnitude.

1. SUMÁRIO EXECUTIVO (pt-BR, ~1 página)

Tese central. O ecossistema agentic/AI-native de engenharia de software em 2026 **não é vendido por uma única ferramenta**. Ele é uma **STACK COMPONENTE DE CAMADAS FUNCIONAIS AGRUPADAS POR PAPEL** (substratos · conhecimento · pipeline · amplificador), roteada como um **Mixture-of-Experts (MoE)**: um *gating network* simbólico-estratificado recebe o intent do operador e decide **quais experts ativar e em que ordem**. O que muda a economia da escolha é que **três substratos ficam sempre ligados** (“always-on”), envelopando densamente toda rota como um *residual* somado: **L0 guardrails + L8 memory + L9 observability**. Sobre esse colchão, a pipeline (L1 spec → L2 workflow → L6 design → L7 slides) e o conhecimento (L4 codebase / L5 PKM) recebem **top-1 esparsos por slot** — porque dois specs, dois injetores de instrução ou dois studios **não convergem** no mesmo repo. O L3 (amplifier) é um *gating* de segundo nível, **opcional e caro**, que só dispara em alta ambiguidade/paralelização.

Achados de SEGURANÇA (headline) — o que mudou o set INCLUDED. O gate de supply-chain do RUN 0 **alterou o conjunto** com três achados decisivos: 1. **GSD \$GSD rug-pull → EXCLUDED (confirmado)**. *gsd-build/get-shit-done* (original, ~63K): token Solana \$GSD, saque de liquidez (~US\$500K), fundador “TÂCHES” deletou contas ~2026-04. **A ameaça é viva:** os pacotes npm ORIGINAIS seguem publicados sob chaves do golpista; um update malicioso comprometeria máquinas locais (GSD pede permissões shell/bash profundas). Sucessor seguro auditado: **open-gsd/gsd-core** (sem backdoors no fork; risco está nos pacotes originais). 2. **MemPalace → EXCLUDED/FLAGGED em alegações SECUNDÁRIAS NÃO PROVADAS**. Alegações (não-auditadas) de compra de ~42K em repo de ~2 meses, “wrapper de ChromaDB” sob nome de celebridade (milla-jovovich/mempalace) e benchmark “96.6% LongMemEval” não-reproduzível. **Não se afirma fraude como fato** — pelo gate, sai do INCLUDED até auditoria independente. Substituído por **mem0** (benchmark memory-benchmarks reproduzível) como default de memória. 3. **Contexto de vulnerabilidade de skills**. O ambiente de pacotes agent-skill carrega risco material: relatos do período apontam **~13% de pacotes agent-skill com vulnerabilidades críticas** [UNVERIFIED – re-verificar]. Isso fundamenta tratar “velocidade de estrela” como **sinal, não prova** (vide MemPalace), e por que o HUB roda AgentShield em PreToolUse.

Ressalva de MAGNITUDE de estrelas. Vários counts são **extraordinários** para repos de 2025-2026 — rivalizam os maiores do GitHub (ECC ~188K, karpathy ~154-183K, superpowers ~147K, spec-kit ~106K, gstack ~101K). São reportados como **valor de API ao vivo datado (2026-06-27)**, fonte primária, **sem editorializar** — mas com **aviso de magnitude** para verificação humana. Não são precisão; são o melhor sinal verificável hoje.

O HUB em 3-4 frases. O Hub (multi-agent-os) é o **gating network**: um control plane *stateless* que roteia intent → CTS scoring → ISO tool-gating → model-router, sobre os substratos always-on, com **HITL** (human-in-the-loop) obrigatório em ações **irreversíveis** ou de **risco HIGH**. O *Intent Classification* usa o mapa simbólico barato do BMAD (fase do SDLC → persona) e sobe para lookup HNSW (padrão ruflo) só em ambiguidade. O CTS (Composite Tool Score) aplica **hard filters de segurança/licença primeiro** e depois pontua candidatos por ISO × Eisenhower × risk × scope × methodology × reversibility, importando o *trust_score* do ruflo verbatim. O *ISO tool-gating* mantém um pool permanente de summaries ≤60 tokens/tool e só promove o schema completo dos top-k — matando o “MCP/Tools Tax” e mantendo a janela abaixo de ~70% de utilização.

2. PANORAMA MERGED (Fases 0-3, condensado)

2.1 Veredito final por categoria (do arquivo 00)

Veredito	Itens
INCLUDED (26 experts)	karpthy-claude-md[!] · base (cluster)[!] · mem0 · letta · cognee · langfuse[!] · graphify · understand-anything · obsidian-skills · spec-kit · openspec · gsd-core · superpowers · gstack · ECC · cli-anything · paul · carl · seed[!] · open-design · open-codesign · impeccable · slidev · frontend-slides · ruflo · bmad-method
BRIDGE	getzep/graphiti (L4↔L8)
NEIGHBOR	agentmemory · openllmetry · lmn · opik · phoenix · agentic-flow · parruda/swarm · claude-squad
BASELINE	zep · obsidian app · claude-design/frontend-design · MetaGPT · LangGraph/AutoGen/CrewAI · claude-code
LENS	SpecDD (rótulo de metodologia SDD; coberto por spec-kit + openspec)
EXCLUDED	gsd-build/get-shit-done ([FLAG] \$GSD rug-pull) · MemPalace ([FLAG] star-manip/claims não-provadas) · forrestchang/andrey-karpthy-skills (404/mis-attr) · klaviyo/graphiti_mcp (fork) · goabstract/Awesome-Design-Tools (stale/non-agentic)
INFRA (não-Target)	BerriAI/litellm (model gateway, usado em Phase 3)

[!] = flag de licença/segurança (ver §3). O span da árvore N-Tree é os **26 INCLUDED + BRIDGE graphiti = 27 nós**; NEIGHBOR/BASELINE entram só quando uma aresta os exige.

2.2 Landscape compacto por PAPEL (o “para que serve” de cada INCLUDED)

SUBSTRATOS always-on (01a) — envelopam toda rota, não competem pela tarefa: - **L0 guardrails:** karpthy-claude-md — força deliberação antes de codar (Think-Before-Coding, Simplicity, Surgical-Changes, Goal-Driven). · base — anti-staleness do workspace (drift score, PSMM) via hooks. - **L8 memory:** mem0 (**DEFAULT**) — memória plugável universal, isolada por tenant, benchmark reproduzível. · letta — runtime de agente stateful com memória OS-tiered self-editing (lock-in: usar a memória = usar o agente). · cognee — memória graph-native + ontologia (“Company Brain”). · graphiti (**BRIDGE L4↔L8**) — grafo de contexto bi-temporal (fatos com janela de validade; invalida em vez de deletar). - **L9 observability:** langfuse (**DEFAULT**) — tracing + evals + prompt-mgmt + datasets, OTel-native.

CONHECIMENTO (01b) — consultado antes de especificar: - **L4 codebase:** graphify — indexa o repo 1x → grafo queryable (token-efficiency ~71x); exporta Obsidian Vault (ponte L4→L5). · understand-anything — grafo interativo/educativo (“graphs that teach”); onboarding + exploração visual. - **L5 PKM:** obsidian-skills — 5 skills oficiais do Obsidian (kepano) que ensinam o agente a operar vaults (wikilinks, Bases, Canvas).

PIPELINE (01c) — esteira ordenada L1→L2→L6→L7: - **L1 spec:** spec-kit — SDD canônico/template-rich da GitHub (constitution→specify→plan→tasks→implement; compliance-ready). · openspec — SDD leve/brownfield-first/iterativo (change-folders). · gsd-core — SDD anti-context-rot via sub-agentes de contexto-fresco (fork SEGURO). - **L2 workflow:** superpowers — metodologia SDLC via

skills (TDD obrigatório; menor footprint do trio). · gstack — “virtual eng team” de 23 personas (Garry Tan; reescreve CLAUDE.md). · ECC — harness operator system maximalista (232 skills + instincts + hooks + AgentShield). · cli-anything — gera CLIs agent-native a partir do código-fonte (“making ALL software agent-native”). · paul/carl/seed (cluster Kahler) — Plan-Apply-Unify in-session / injeção JIT de regras por hook / incubadora de ideação tipada. - **L6 design:** open-design — studio local-first multi-surface (amplo; bypassPermissions). · open-codesign — studio desktop enxuto (loop permissionado, mais seguro). · impeccable — LAYER de qualidade de design key-free (41 regras determinísticas; não é studio). - **L7 slides:** slidev — framework slides-as-code maduro (2021, human-first, não agent-native). · frontend-slides — skill agêntico de slides (Claude Code; anti-slop).

AMPLIFICADOR (01d) — L3 opcional, meta-routing: - ruflo — meta-harness/swarm runtime (Queen→topology + MoE+HNSW dispatch; 314 MCP tools; learning loop; **exige L8+L9**). · bmad-method — metodologia ágil persona-driven (gating simbólico fase→persona; “Party Mode”; sem L8 nativo).

2.3 SPINE de roteamento N-Tree + top-5 incompatibilidades (do arquivo 02)

Spine (uma frase): substrate-check (L0 guardrails + L8 memory recall + L9 trace-on) → KNOWLEDGE (L4 grafo de codebase / L5 PKM) → L1 spec → L2 workflow (TDD/worktree) → L6 design → L7 slides, com **L3 como amplificador opcional** que envolve L1-L7 quando a tarefa exige swarm/persona-handoff.

Top-5 incompatibilidades (arestas negativas — NÃO compor): 1. **spec-kit [x] openspec** (L1) — modelos de spec **mutuamente exclusivos**; dois diretórios + duas convenções no mesmo repo não convergem. 2. **superpowers [x] gstack [x] ECC** (L2) — colisão de instrução always-on; três workflows “mandatory” = precedência imprevisível + erosão de contexto (ECC: 200k→~70k). Escolher **um**. 3. **base [x] ECC** (L0) — dois gerenciadores de CLAUDE.md → hooks em duplicata (o próprio /base:audit-claude detecta o sprawl). 4. **mem0 [x] letta [x] coggnee** (L8) — backends concorrentes; **letta = lock-in de runtime** (“must use Letta”); colisão de embeddings se compartilham pgvector sem namespacing. 5. **open-design [x] open-codesign** (L6) — mesma vaga e postura de segurança incompatível (bypassPermissions total vs loop permissionado). (Bônus: paul [x] gsd-core — filosofia oposta in-session vs subagentes; gstack bane mcp__claude-in-chrome__* que ECC empacota; carl [x] ECC — dois hook-engines per-prompt.)

2.4 Arquitetura do HUB em ~8 bullets (essência (a)-(m) do arquivo 03)

- **(a) Control plane stateless de 5 estágios:** Intent Classification → CTS Scoring → Tool-Gating/Lazy-Schema → Model Router → Tool Registry, com substratos cross-cutting todos. Reusa o gating de fase **BMAD** como classificador barato; sobe para **HNSW (ruflo)** em ambiguidade.
- **(b) ISO tool-gating (Tool-Attention):** pool permanente de summaries ≤60 tok/tool; promove schema completo só dos top-k via *gated attention* (sparse top-k softmax). Padrão **NanoClaw hot-load** / lazy-schema. Mata o “Tools Tax” ($60 \cdot N + k \cdot |\text{schema}|$ vs $\sum |\text{schema}|$); mantém utilização <70%.
- **(c) CTS (Composite Tool Score): hard filters de segurança/licença PRIMEIRO** (HF1 EXCLUDED/FLAGGED, HF2 license none/NOASSERTION em redistribuição comercial, HF3 escala de privilégio, HF4 incompatibilidade de camada, HF5/HF6 risk=HIGH/irreversível → HOLD), depois soma ponderada. Importa o **trust_score do ruflo verbatim** ($0.4 \cdot \text{success} + 0.2 \cdot \text{uptime} + 0.2 \cdot \text{threat} + 0.2 \cdot \text{integrity}$).
- **(d) Tool Registry YAML (SSOT):** cada nó do grafo 02 = um registro com layer, role, summary≤60tok, preconditions, risk_tier, authorization, harness_coverage, security_status, mechanisms[]. Alimenta (b)-(c).
- **(e) Memory substrate (L8):** memória de roteamento tiered (working→session→operator→episodic→temporal) em **mem0 + graphiti** (temporal). Consolidação = **Learning-Loop do ruflo** (RETRIEVE→JUDGE→DISTILL→C — rotas frequentes viram fast-paths).

- **(f)+(h) Observability + gates (L9):** todo route emite spans OTLP para **langfuse**; LLM-as-judge realimenta a cadeia **warn→correct→block**. L0 sempre primeiro (PreToolUse); HITL obrigatório em risk=HIGH/irreversível/escolha mutuamente-exclusiva de alto impacto.
- **(g) Model Router (Switchcraft-style):** modelo capaz **mais barato** por tool-call via **LiteLLM (INFRA)** + budgets por categoria; downgrade ou HITL se estourar.
- **(i)+(m) Security model + pragmatism filter: AgentShield (ECC)** como scanner L0 de supply-chain (block --no-verify, secrets, exfil de CLAUDE.md); **tool-output tainting** de 3 estados (trusted/untrusted/derived — taint não escala privilégio); sandbox+egress por tool. **Rejeitados explicitamente:** blockchain audit (graphiti já dá trilha), quantum-routing, swarms auto-replicantes, “L10 gateway layer” (LiteLLM/MCP são INFRA, não camada de experts), gating neural denso (este é simbólico+estratificado).

3. RECONCILIAÇÃO DE ESTRELAS & LICENÇAS (com datas)

Todas as observações são de **2026-06-27**. “kit v5” = valor citado no prompt-objeto v5. “RUN0 API” = valor da API ao vivo do RUN 0 (sem tag) ou [sec] (secundário, API vazia). “Phase-1 page-obs” = contagem da página de repositório observada no arquivo 01a-d. **Valor canônico adotado** = o que o HUB usa, com nota de reconciliação.

expert	kit v5 (citado)	RUN0 API 2026-06-27	Phase-1 page-obs 2026-06-27	valor canônico adotado + nota
karpathy- claude-md	~183K	154,186	183k (01a)	~ 154-183K · divergência grande: API 154K vs página 183K; reportar faixa, [!] magnitude extraordinária
base	—	87	86 (01a)	~ 86-87 · cluster <1K, estável
mem0	—	~57-59K [sec]	58.2k (01a)	~ 58K · [sec] no RUN0 (API vazia); página corrobora
letta	—	22,891	23.5k (01a)	~ 23K · crescimento normal API→página
cognee	—	17,453	23.3k (01a)	~ 17-23K · divergência: API 17.4K vs página 23.3K; reportar faixa
graphiti (BRIDGE)	~26K	26,380	27.8k (01a)	~ 27K · consistente
langfuse	—	28,219	27.2k (01a)	~ 28K · consistente

expert	kit v5 (citado)	RUN0 API 2026-06-27	Phase-1 page-obs 2026-06-27	valor canônico adotado + nota
graphify	~73K	52,657	~56-69k [sec] (01b)	~ 52-57K · v5 73K [SU- PERSEDED]; adoto API 52.7K, faixa secundária acima
understand- anything	~68K	~55-66K [sec]	64.9k [sec] (01b)	~ 55-65K · [sec]; migrou Lum1104→Egonex- AI
obsidian- skills	~36K	33,438	35k (01b, CONFIRMADO)	~ 35K · página confirmou “35k stars”
spec-kit	~90-97K	106,332	dinâmico (01c)	~ 106K · ↑ vs v5; [!] magnitude (GitHub- owned, crescimento rápido)
openspec	~56K	51,175	dinâmico (01c)	~ 51K · ↓ vs v5
gsd-core	—	~4.2K [sec]	—	~ 4.2K · [sec]; fork SEGURO (original tinha 65K+)
superpowers	~240K	~ 147K [sec]	dinâmico (01c)	~ 147K [sec] · ↓ ↓ vs v5 240K; [!] magnitude + [sec]
gstack	~117K	101,289	dinâmico (01c)	~ 101K · ↓ vs v5; [!] magnitude
ECC	~163K (abr)	187,680 [sec]	README diz “182K+”; busca live 222K+ (01c)	~ 188K [sec] · maior divergência do conjunto; ver nota abaixo; [!] magnitude extrema
cli-anything	~42-44K	~42-44K [sec]	40K em 2026-05-24 (01c)	~ 42-44K [sec]
paul	—	976	976 (01c, API exato)	976 · cluster, API autoritativa

expert	kit v5 (citado)	RUN0 API 2026-06-27	Phase-1 page-obs 2026-06-27	valor canônico adotado + nota
carl	—	353	353 (01c, API exato)	353 · cluster
seed	—	288	288 (01c, API exato)	288 · cluster
open-design	~70K	48,662	só badge (01c)	~ 49K · ↓ vs v5; [!] version-drift (19/71 do brief SUPERSEDED)
open- codesign	—	6,993	só badge (01c)	~ 7K
impeccable	—	30,054	só badge (01c)	~ 30K
slidev	—	47,037	só badge (01c)	~ 47K · ©2021 antfu
frontend- slides	—	19,093	—	~ 19K
ruflo	~38-62K	54,569	~61.7k (01d)	~ 54-62K · v5 amplo; API 54.6K, página 61.7K; reportar faixa
bmad- method	~43K	47,753	~49.6k (01d)	~ 48-50K · ↑ vs v5

A maior divergência de estrela reconciliada: ECC. Três fontes contraditórias num único dia — **RUN0 API ~188K [sec]**, **README hardcoda “182K+”** (prosa de marketing), **busca live dá 222K+**. Adoto **~188K [sec]** (valor de API do RUN0) como canônico, marco [UNVERIFIED] para o dígito exato, e sinalizo magnitude extrema. (*Runner-up: superpowers v5 240K → API ~147K [sec], queda de ~93K.*)

3.1 Sub-tabela de correções de LICENÇA (gate “state license”)

expert	licença citada (v5/page)	realidade verificada 2026-06-27	ação
karpathy-claude-md / base	MIT (README/prosa)	none — README diz MIT mas sem arquivo LICENSE → all-rights-reserved por default	flag OSS-posture; HF2 bloqueia redistribuição comercial; manter como baseline

expert	licença citada (v5/page)	realidade verificada 2026-06-27	ação
BMAD-METHOD	NOASSERTION (RUN0)	MIT (CONFIRMADO) — leitura direta do LICENSE: “MIT License, Copyright (c) 2025 BMad Code, LLC”. O NOASSERTION do RUN0 era artefato de parsing SPDX do GitHub (trademark rider BMad™ + MIT)	corrigido para MIT ; trademark separado sobre “BMad™”
langfuse	MIT (page)	NOASSERTION / source-available — MIT-core exceto pastas ee/ (Enterprise Edition proprietárias)	flag: único default não-Apache-puro; se OSS-puro, usar só core ou trocar por openllmetry+opik
phoenix (NEIGHBOR)	OSS	NOASSERTION / source-available — string SPDX não confirmável do README [UNVERIFIED]	flag cautela; fora do span INCLUDED
CLI-Anything	Apache-2.0 [confirm]	Apache-2.0 (CONFIRMADO) — arquivo LICENSE completo lido; “Copyright [2026] [HKUDS CLI-Anything Team]”; grant de patente	confirmado; era load-bearing; redistribuição comercial OK
seed / base	MIT (prosa)	license=null na API — LICENSE ausente/vazio; claim MIT só na prosa	maior risco legal do cluster; HF2; não redistribuir sem clareza
litellm (INFRA)	MIT	NOASSERTION — cláusula custom	flag infra; usado em Phase 3, não-Target

Flags [sec]/[UNVERIFIED] herdadas (todas preservadas): estrelas via secundário (API vazia) em mem0, superpowers, understand-anything, cli-anything, gsd-core, lmnr, par-ruda/swarm; dígitos exatos de ECC; string SPDX de phoenix; acrônimo “ECL/control-plane” de cognee; od mcp install de open-design; webhooks de ruflo; benchmarks self-reported de ruflo (“89% routing accuracy”, AgentDB speedup, “1.953× faster than LangGraph”).

4. CHEAT-SHEET DE ROTEAMENTO (“qual expert, quando”)

Eixo: **quadrante de Eisenhower × risco × escopo → stack recomendado + nota HITL**. Regra de ouro: **substrate-first SEMPRE** (L0 karpthy/base + L8 mem0/graphiti + L9 langfuse) antes de qualquer pipeline. ** | ** = escolha mutuamente-exclusiva resolvida pelo CTS/HITL.

Quadrante	Cenário (risco · escopo)	Stack recomendado (substrate-first → pipeline)	Nota HITL
Q1 urgente+importante	Incidente em prod, risco HIGH, escopo repo/org	L0 base(drift)+karpathy · L8 mem0 · L9 langfuse(evals) → graphify(L4 blast-radius) → gsd-core gstack(/ship,/cso OWASP+STRIDE) → impeccable(linter CI)	HITL obrigatório (risk=HIGH): aprovação de PR + canary; [!] gsd-core = SÓ o fork open-gsd (HF1); taint-check em qualquer fonte externa
Q1 urgente+importante	Hotfix pontual, risco MED, escopo file/module	L0 karpathy · L8 mem0 · L9 langfuse → openspec (brownfield leve) → superpow- ers (TDD+worktree, menor footprint)	HITL no code-review com severidade ("Critical issues block progress"); pular L3 (over-kill)
Q2 importante, não-urgente	Greenfield/feature nova, risco LOW-MED, escopo module→repo	L0 karpathy · L8 mem0 · L9 langfuse → seed(ideação) → spec-kit openspec(L1) → superpowers(L2 TDD) → open-codesign(L6 enxuto+seguro) → frontend-slides(L7 pitch)	HITL na escolha spec-kit↔openspec (HF4 mutuamente exclusivos) + aprovação do plano (clarify)
Q2 importante, não-urgente	Refactor de legado, risco MED, escopo module	L0 karpathy+base · L8 cognee (graph) · L9 langfuse → understand- anything(L4 onboarding) → graphify(L4 contexto) → open- spec (brownfield) → superpowers(worktree+review)	HITL no code-review; [!] não commitar grafo com segredos extraídos; CTS escolhe openspec (0.779>0.590 vs spec-kit)
Q2 importante, não-urgente	Feature enter- prise/compliance, risco MED-HIGH, escopo org	L0 karpathy · L8 graphiti (temporal/auditoria) · L9 langfuse(HIPAA) → graphify(L4) → spec- kit (constitution+compliance) → gstack superpowers → open-design(L6) → slidev(L7)	HITL nos gates de compliance (/speckit.analyze, /cso); não empilhar gstack+ECC (HF4)
Q3 urgente, não-importante	Slides/design rápido, risco LOW, escopo file	L0 karpathy · L8 mem0 · L9 langfuse → open-codesign open-design(L6) → frontend-slides slidev(L7)	HITL baixo/opcional; se open-design: sandbox reforçado (bypassPermissions é risk=HIGH)

Quadrante	Cenário (risco · escopo)	Stack recomendado (substrate-first → pipeline)	Nota HITL
Q3 urgente, não-importante	Onboarding de codebase, risco LOW (read-mostly), escopo repo	L0 karpathy · L8 cognee · L9 langfuse → understand-anything(L4 dashboard+tour) → graphify(L4 export Obsidian) → obsidian-skills(L5 vault)	Risco baixo (read-mostly); HITL opcional na revisão do domínio extraído
Q3 / sessão longa	Sessão longa com memória cross-session, risco MED, escopo repo	L0 karpathy · L8 letta (self-edit) mem0+graphiti · L9 langfuse → ECC gsd-core(L1/L2 STATE persistente) → ruflo (L3 learning-loop, opcional)	[!] letta = lock-in de runtime ; ruflo = over-kill se solo/budget-limitado (27 hooks+daemon); HITL na decisão letta-vs-mem0
Q4 nem-nem	Experimento/PoC descartável, risco LOW, escopo file	L0 karpathy (só) · L8 mem0(efêmero) · L9 langfuse(opcional) → cli-anything superpowers leve	Pular L3 inteiramente (amplifier overkill em CI efêmero/PoC); sem HITL; não persistir aprendizado

Cobertura confirmada: **Q1, Q2, Q3, Q4** todos presentes. Inclui as linhas-exemplo pedidas (Q1 incidente prod HIGH-risk; Q2 greenfield; Q2 legacy refactor; Q3 slides/design quick-turn; sessão longa com memória).

5. LIMITAÇÕES & PRÓXIMOS PASSOS

Nota epistêmica honesta. - **Estrelas datadas/divergentes/extraordinárias.** Todo count é de 2026-06-27 e é **ordem-de-magnitude, não precisão**. Vários divergem materialmente entre API/página/v5 (ECC 188K vs 222K vs “182K+”; superpowers 240K→147K; karpathy 154K↔183K; cognee 17.4K↔23.3K; graphify 52.7K↔69K). Counts extraordinários (ECC/karpathy/superpowers/spec-kit/gstack) são reportados sem editorializar mas sinalizados — repos rivalizando os maiores do GitHub em ~1 ano merecem ceticismo de magnitude. - **Itens [sec] a re-verificar com token autenticado.** mem0, superpowers, understand-anything, cli-anything, gsd-core, lmnrr, parruda/swarm tiveram estrelas via fonte **secundária** (API retornou corpo vazio). Re-tentar com retry/token autenticado para confirmar dígitos. - **MemPalace: alegações NÃO PROVADAS.** A exclusão é por **gate de cautela**, não por fraude comprovada. As alegações de star-manip e benchmark não-auditável são **secundárias** — exigir auditoria independente antes de qualquer reinclusão. - **Pesos de design da Fase 3 são PROPOSTAS.** Os valores numéricos default dos pesos CTS (w_iso=0.30, w_trust=0.30, w_risk=0.50...), os thresholds θ_layer/θ_warn/θ_ok e a contagem de tokens do pool ISO **não são medições** — são extrapolações de engenharia a calibrar empiricamente no plano de testes (k). Os benchmarks do ruflo herdados (“89% routing accuracy”, “1.953× faster”) são **self-reported, [UNVERIFIED]**.

Próximos passos concretos (3-5): 1. **Re-rodar a coleta de estrelas/licenças com token GitHub autenticado** — eliminar os [sec] (API vazia), confirmar dígitos de ECC/superpowers e resolver as divergências API↔página de cognee/karpathy/graphify/ruflo. 2. **Auditoria de**

supply-chain pendente — verificar a alegação de ~13% de pacotes agent-skill com vulns críticas [UNVERIFIED]; rodar AgentShield (102 rules) sobre os experts INCLUDED com canal de instrução pesado (ECC, gstack, carl, base) para mapear hooks/MCP-risk reais. 3. **Calibrar os pesos CTS e thresholds** empiricamente via a matriz 6×3 (6 task-families × {low,med,high} risk) do plano de testes (k); medir A/B com vs sem o ISO tool-gating (token cost + % context-fracture acima de 70%). 4. **Confirmar termos das licenças NOASSERTION** (langfuse ee/, phoenix SPDX, litellm cláusula custom) e os license=null (seed/base/karpathy) antes de qualquer redistribuição comercial. 5. **Implementar o Tool Registry YAML (SSOT)** para os 26 INCLUDED + graphiti (só 4 estão expandidos no arquivo 03) e o sketch do MCP server maos-hub (hub.route/hub.score/hub.registry.get/hub.gate.approve/hub.trace), revisando o MemPalace e re-validando os [UNVERIFIED] a cada TTL (revisão recomendada: 2026-09-27).

Phase 4 · Final Report · funde 00 + 01a-d + 02 + 03 · 26 experts INCLUDED + BRIDGE graphiti · observado 2026-06-27. Prosa pt-BR, identificadores/fórmulas en-US. Sem nova pesquisa web — merge + reconciliação. Flags [UNVERIFIED]/[sec] preservadas. Estrelas = ordem-de-magnitude datada, não precisão.